
**Estudio de complejidad de circuitos para el
problema CLIQUE**
**Study of circuit complexity for the CLIQUE
problem**



Trabajo de Fin de Grado
Curso 2024–2025

Autor

Daniel González Arbelo

Directores

Ismael Rodríguez Laguna

Narciso Martí Olié

Calificación: 10

Doble Grado en Ingeniería Informática y Matemáticas

Facultad de Informática

Universidad Complutense de Madrid

19 de junio de 2025

Agradecimientos

A mis amigos y compañeros de estos últimos cinco años, porque este doble grado sin ellos de mi lado habría sido inimaginable. Gracias por ser un fondo de agua en esta enorme piscina en la que nos hemos metido.

A mis tutores, Ismael y Narciso, por haber depositado su confianza en mí desde el inicio y por haber hecho posible realizar un trabajo del que estar orgulloso como informático.

A la gente de mi asociación, UCppM, y a todas las personas cercanas que he conocido a través de la programación competitiva, por haberme motivado y llevado a hacer tantas cosas con las que hacer el paso por la carrera mucho más memorable. En especial, a Jaime Jacobo Romo González, por sugerirme continuar con su línea de investigación, revisar mi último borrador y ser una persona con quien poder hablar fácilmente de cuestiones tan específicas.

A mis amigos y familia de Gran Canaria, por haber estado conmigo siempre y hacer que con ellos sea todo siempre tan fácil.

Y sobre todo, a mis padres, por ser la mayor fuente de apoyo que uno puede tener.

Muchas gracias a todos.

Resumen

Estudio de complejidad de circuitos para el problema CLIQUE

En lo referente al problema $\mathcal{P}$ vs $\mathcal{NP}$, uno de los campos de estudio que más útil ha demostrado ser en las últimas décadas es el de los circuitos booleanos. En este texto buscamos adentrarnos en esta vía de estudio, concretamente abordando el problema de $Clique_{n,k}$. Estudiamos detalladamente uno de los resultados más importantes que se han desarrollado en este campo, que enuncia que cualquier sucesión de circuitos de profundidad constante que compute $Clique_{n,k}$ debe tener tamaño superpolinómico. Después de esto, estudiamos este problema de forma experimental, continuando con la vía de estudio propuesta en un trabajo anterior, cuyo objeto de estudio principal es la métrica sintáctica. Contrastamos ciertas hipótesis acerca del posible crecimiento de la puntuación y proponemos de forma teórica varias medidas que describen comportamientos que pueden diferenciar las funciones más productivas de las funciones fácilmente computables. Finalmente, ponemos a prueba estas medidas para el caso de $n = 8$ y $k = 4$, exponiendo las diferencias que presentan los distintos tipos de funciones, así como desarrollando un modelo de aprendizaje supervisado que, con base en los valores de las distintas medidas, sea capaz de predecir el tipo de función del que provienen.

Palabras clave

$\mathcal{P}$ vs $\mathcal{NP}$, $\mathcal{P}_{poly}$, función booleana, circuito booleano, clique, métrica, forma normal disyuntiva, medida, función simulada, función pseudoaleatoria.

Abstract

Study of circuit complexity for the CLIQUE problem

With regard to the problem $\mathcal{P}$ vs $\mathcal{NP}$, one of the fields of study that has proven most useful in recent decades is that of boolean circuits. In this text, we seek to delve deeper into this line of study, specifically addressing the problem of $\text{Clique}_{n,k}$. We study in detail one of the most important results that has been developed in this field, which states that any sequence of constant depth circuits that computes $\text{Clique}_{n,k}$ must be superpolynomial in size. After this, we study this problem experimentally, continuing with the line of study proposed in a previous work, whose main object of study is the syntactic metric. We contrast certain hypotheses about the possible growth of the score, and we propose several theoretical measures which describe behaviors that can differentiate the most productive functions from easily computable functions. Finally, we test these measures for the case of $n = 8$ and $k = 4$, exposing the differences between the different types of functions, as well as developing a supervised learning model that, based on the values of the different measures, is capable of predicting the type of function from which they originate.

Keywords

$\mathcal{P}$ vs $\mathcal{NP}$, $\mathcal{P}_{poly}$, boolean function, boolean circuit, clique, metric, disjunctive normal form, measure, simulated function, pseudorandom function.

Índice

1. Introducción	1
Introduction	3
2. Estado de la cuestión	5
2.1. El problema de $\mathcal{P}$ vs $\mathcal{NP}$	5
2.2. Circuitos booleanos	7
2.3. Antecedentes	9
3. Cota inferior para circuitos de profundidad constante que calculan $\text{Clique}_{n,k}$	11
3.1. Circuitos con profundidad constante	11
3.2. Preliminares al estudio de Rossman	13
3.3. La técnica de estudio	15
3.3.1. Sensibilidad a clique	15
3.3.2. El lema del circuito	18
3.3.3. Demostración de las versiones débiles de los teoremas principales	20
3.4. Restricciones aleatorias y AC^0	23
3.5. Demostración de las versiones fuertes de los teoremas principales . . .	29
4. Estudio experimental	35
4.1. Introducción a las métricas	35
4.1.1. Métrica sintáctica	36
4.1.2. Consideraciones previas a los experimentos	40
4.2. Experimentos iniciales	42
4.2.1. Conjeturas acerca de las funciones pseudoaleatorias	49
4.3. Medidas	52
4.3.1. Medición del solapamiento	52
4.3.2. Medición de la homogeneidad	57
4.3.3. Medición de grados de libertad	60
4.4. Experimentos con las medidas	67
4.4.1. Resultados con la medida de solapamiento	67

4.4.2. Resultados con la medida de homogeneidad	70
4.4.3. Resultados con la medida de grados de libertad	72
4.5. Predicción de funciones según el valor de las medidas	74
5. Conclusiones y trabajo futuro	77
Conclusions and Future Work	79
Bibliografía	81

Índice de figuras

2.1. Ejemplo de circuitos booleanos equivalentes	8
3.1. Circuito que computa PARIDAD con profundidad lineal	12
3.2. Circuito que computa PARIDAD con profundidad logarítmica	12
3.3. Circuito con niveles de AND y OR alternos	25
3.4. Circuito de niveles alternos tras conmutación de niveles de AND y OR	26
3.5. Circuito tras colapso de niveles de OR	27
3.6. Ejemplos de grafos de tipo $H_{a,b,c}$	30
4.1. Asignaciones óptimas compatibles y de suma máxima	38
4.2. Asignaciones óptimas múltiples de sumas no máximas	38
4.3. Asignaciones óptimas incompatibles	38
4.4. Crecimiento de μ_x con funciones aleatorias	42
4.5. Simulación de 300 iteraciones	44
4.6. Crecimiento de μ_x con funciones simuladas y pseudoaleatorias	48
4.7. Solapamiento de funciones pseudoaleatorias, aleatorias y simuladas . .	67
4.8. Solapamiento de funciones pseudoaleatorias y simuladas	68
4.9. Histograma de solapamiento de funciones pseudoaleatorias y simuladas	68
4.10. Matriz de confusión del umbral de solapamiento	69
4.11. Variación de solapamiento de funciones simuladas	70
4.12. Homogeneidad de funciones pseudoaleatorias, aleatorias y simuladas .	71
4.13. Variación de homogeneidad de funciones simuladas	71
4.14. Grados de libertad de funciones pseudoaleatorias, aleatorias y simuladas	72
4.15. Variación de grados de libertad de funciones simuladas	73
4.16. Matriz de confusión de predicción de funciones simuladas con k-NN .	75

Introducción

En el ámbito de la informática, uno de los problemas más importantes que siguen abiertos a día de hoy es el problema de $\mathcal{P}$ vs $\mathcal{NP}$, el problema del milenio que se mantiene como una gran barrera entre aquello que sabemos resolver rápidamente y aquello que únicamente sabemos verificar rápidamente. A pesar de que casi todas las intuiciones vayan en la misma dirección, la investigación acerca de este problema demuestra una y otra vez ser realmente complicada, lo que lleva a avances muy lentos a lo largo de los años.

Uno de los campos de estudio que parecen haber sido más fructíferos en el pasado es el de los circuitos booleanos. Este campo de estudio resulta de gran interés, entre otras cosas, por permitirnos entender con mucho detalle comportamientos relacionados con los problemas de decisión y las funciones booleanas. Uno de los principales objetos de estudio en este campo es la clase de problemas de decisión $\mathcal{P}_{/poly}$, que recoge todos aquellos problemas de decisión computables con circuitos “pequeños”. Aprovechando que se conoce la inclusión $\mathcal{P} \subset \mathcal{P}_{/poly}$ [4], con el objetivo de demostrar $\mathcal{P} \neq \mathcal{NP}$ buscamos encontrar un problema que se encuentre en la clase $\mathcal{NP}$ que no se encuentre en la clase $\mathcal{P}_{/poly}$.

Un problema que se ha estudiado mucho en el pasado es el problema de $Clique_{n,k}$, el cual consiste en determinar si un grafo contiene un subgrafo completo (clique) de tamaño k , y que será en el que nos centremos en todo este estudio. En relación con este, múltiples estudios pasados han demostrado resultados muy fuertes que parecen “haberse quedado a una restricción” de demostrar $\mathcal{P} \neq \mathcal{NP}$ (véanse los estudios de Aleksandr Razborov [11] y Benjamin Rossman [13], de los cuales hablaremos más adelante). Sin embargo, en todos estos casos, levantar esa última restricción ha sido imposible hasta la fecha.

A través de vías experimentales, buscamos indagar más en este problema, tratando de aportar nuevas ideas e intuiciones que permitan avanzar en esta rama de estudio. En un trabajo previo en esta materia, realizado por el estudiante Jaime Jacobo Romo González [12], se propuso una vía experimental muy interesante, que dio lugar a diversas intuiciones y conjeturas sobre las que trabajar. Concretamente, se propuso un concepto de métrica y se vio que obtener la puntuación máxima sobre dicha métrica empleando una cantidad polinómica de puertas lógicas es condición necesaria para que el problema de $Clique_{n,k}$ esté en $\mathcal{P}_{/poly}$. Además, se observó que

los resultados experimentales parecen indicar que esto no es fácil.

Siguiendo esta línea, buscamos estudiar con más detalle por qué ocurre esto. Creemos que, a través del avance por esta línea de investigación, es posible que encontremos ideas que nos permitan entender mejor el conjunto $\mathcal{P}_{poly}$, concretamente los circuitos orientados a computar $Clique_{n,k}$, que lleven a herramientas que nos acerquen más a caracterizarlo.

En el capítulo 2 estudiaremos conceptos que emplearemos a lo largo de todo el trabajo. Principalmente, explicaremos con detalle los conceptos de la clase $\mathcal{P}_{poly}$ y las funciones y circuitos booleanos, y expondremos ciertos resultados importantes que involucren a estos últimos y motivan su uso como objeto de estudio. Buena parte de la información presentada en este capítulo proviene del estudio del capítulo 6 del libro *Computational Complexity: A Modern Approach* [4], obra de referencia en la materia.

Dedicaremos el capítulo 3 a estudiar con detalle uno de los resultados más relevantes que se han desarrollado en el ámbito de los circuitos booleanos y el problema de $Clique_{n,k}$. En él se demuestra que cualquier sucesión de circuitos de profundidad constante que compute el problema de $Clique_{n,k}$ debe ser de tamaño superpolinómico [13]. En la sección 3.1 daremos ejemplos de por qué la noción de la profundidad de los circuitos es relevante. En las secciones siguientes estudiaremos con detalle los resultados y demostraciones, con la intención de facilitar su comprensión, así como para desarrollar intuiciones de cara a nuestro estudio experimental.

En las secciones 4.1 y 4.2 sentaremos las bases de nuestro estudio experimental. Explicaremos la línea de investigación propuesta por Jaime Jacobo Romo González [12] y expondremos los que serán nuestros objetos de estudio principales: las funciones simuladas y las funciones pseudoaleatorias. Las funciones simuladas recogen la idea de “funciones computables con circuitos pequeños”, mientras que las funciones pseudoaleatorias son un conjunto de funciones que destacan por ser muy productivas bajo cierta métrica. A través de sus definiciones, expondremos varias conjeturas acerca de las diferencias entre estos dos tipos de funciones, que motivarán los desarrollos posteriores.

En la sección 4.3 expondremos nuestras aportaciones teóricas al estudio del problema, acompañadas de ciertos desarrollos acerca de sus propiedades. Desarrollaremos una serie de conceptos bajo el nombre de *medidas*, con la esperanza de que resulten útiles de cara a intentar caracterizar el conjunto de las funciones simuladas, aportando algo más de intuición acerca de las hipótesis formuladas a partir del trabajo [12].

En las secciones 4.4 y 4.5 pondremos a prueba las medidas definidas a través de experimentos, determinando si los resultados coinciden con las conjeturas planteadas anteriormente, y generaremos una herramienta de aprendizaje automático que, con base en las medidas, prediga si una función es simulada o no. Idealmente, si este modelo alcanza una precisión muy alta, reforzaría la idea de que las medidas podrían ser útiles para caracterizar el conjunto de las funciones simuladas.

Finalmente, en el capítulo 5 resumiremos los resultados obtenidos y comentaremos ciertas ideas acerca de posibles formas de avanzar con esta línea de investigación.

Introduction

In the field of computer science, one of the most important problems that remains unsolved today is the $\mathcal{P}$ vs $\mathcal{NP}$ problem, the millennium problem that remains a major barrier between what we know how to solve quickly and what we only know how to verify quickly. Although almost all intuitions point in the same direction, research on this problem has proven time and again to be extremely complicated, leading to very slow progress over the years.

One of the fields of study that seems to have been most fruitful in the past is that of boolean circuits. This field of study is of great interest, among other things, because it allows us to understand in great detail behaviors related to decision problems and boolean functions. One of the main objects of study in this field is the class of decision problems $\mathcal{P}_{/poly}$, which includes all decision problems that can be computed with “small” circuits. Taking advantage of the fact that the inclusion $\mathcal{P} \subset \mathcal{P}_{/poly}$ [4] is known, with the aim of proving $\mathcal{P} \neq \mathcal{NP}$, we seek to find a problem that is in the class $\mathcal{NP}$ but not in the class $\mathcal{P}_{/poly}$.

A problem that has been studied extensively in the past is the $Clique_{n,k}$ problem, which consists of determining whether a graph contains a complete subgraph (clique) of size k , and which will be the focus of this study. In relation to this, multiple past studies have demonstrated very strong results that seem to be “one restriction away” from proving $\mathcal{P} \neq \mathcal{NP}$ (see the studies by Aleksandr Razborov [11] and Benjamin Rossman [13], which we will discuss later). However, in all these cases, lifting that last restriction has been impossible to date.

Through experimental approaches, we seek to delve deeper into this problem, trying to contribute new ideas and insights that will advance this branch of study. In previous work on this subject, carried out by student Jaime Jacobo Romo González [12], a very interesting experimental approach was proposed, which gave rise to various insights and conjectures to work on. Specifically, a metric concept was proposed, and it was seen that obtaining the maximum score on this metric using a polynomial number of logic gates is a necessary condition for the $Clique_{n,k}$ problem to be in $\mathcal{P}_{/poly}$. Furthermore, it was observed that the experimental results seem to indicate that this is not easy.

Following this line of inquiry, we seek to study in more detail why this occurs. We believe that, by advancing along this line of research, we may find ideas that allow us to better understand the set $\mathcal{P}_{/poly}$, specifically circuits designed to compute $Clique_{n,k}$, leading to tools that bring us closer to characterizing it.

In Chapter 2, we will study concepts that we will use throughout this work. Mainly, we will explain in detail the concepts of the class $\mathcal{P}_{/poly}$, boolean functions and circuits, and we will present certain important results involving the latter that motivate their use as objects of study. Much of the information presented in this chapter comes from the study of Chapter 6 of the book *Computational Complexity: A Modern Approach* [4], a reference work on the subject.

We will devote Chapter 3 to studying in detail one of the most relevant results that has been developed in the field of boolean circuits and the $\text{Clique}_{n,k}$ problem. It shows that any sequence of circuits of constant depth that computes the $\text{Clique}_{n,k}$ problem must be of superpolynomial size [13]. In Section 3.1, we will give examples of why the notion of circuit depth is relevant. In the following sections, we will study the results and proofs in detail, with the aim of facilitating their understanding and developing intuitions for our experimental study.

In Sections 4.1 and 4.2, we will lay the foundations for our experimental study. We will explain the line of research proposed by Jaime Jacobo Romo González [12], and we will present what will be our main objects of study: simulated functions and pseudorandom functions. Simulated functions capture the idea of “functions computable with small circuits,” while pseudorandom functions are a set of functions that stand out for being very productive under a certain metric. Through their definitions, we will present some conjectures about the differences between these two types of functions, which will motivate further developments.

In Section 4.3, we will present our theoretical contributions to the study of the problem, accompanied by certain developments regarding its properties. We will develop a series of concepts under the name *measures*, in the hope that they will be useful in attempting to characterize the set of simulated functions, providing some more insight into the hypotheses formulated in the work [12].

In Sections 4.4 and 4.5, we will test the measures defined through experiments, determining whether the results match the conjectures made above, and we will generate a machine learning tool that, based on the measures, predicts whether a function is simulated or not. Ideally, if this model achieves very high accuracy, it would reinforce the idea that the measures are useful for characterizing the set of simulated functions.

Finally, in Chapter 5, we will summarize the results obtained and discuss some ideas about possible ways to advance this line of research.

Estado de la cuestión

2.1. El problema de $\mathcal{P}$ vs $\mathcal{NP}$

En el ámbito de la informática teórica, uno de los retos sin resolver más importantes que hay es el problema de $\mathcal{P}$ vs $\mathcal{NP}$, uno de los problemas del milenio.

De manera divulgativa se puede decir que este problema intenta abordar la cuestión de determinar si aquellos problemas que parecen ser muy difíciles de resolver pero fáciles de verificar son realmente tan difíciles de resolver. Veamos definiciones más formales para abordar estos conceptos.

Definición 2.1.1. Decimos que $\mathcal{A}$ es un **problema de decisión** cuando la salida asociada a cada posible entrada es 1 o 0, que representan “sí” y “no”, respectivamente. En caso de ser 1, decimos que $\mathcal{A}$ acepta la entrada, y en caso contrario decimos que la rechaza.

Este tipo de problemas son a los que se refieren las clases $\mathcal{P}$ y $\mathcal{NP}$.

Definición 2.1.2. La clase $\mathcal{P}$ es la que contiene a todos los problemas de decisión que pueden resolverse con una máquina de Turing determinista en tiempo polinómico.

En general, es habitual referenciar más a los problemas de esta clase como aquellos que se pueden resolver con un algoritmo de coste polinómico. Es común hablar de estos problemas como “tratables”, pues generalmente los problemas de esta clase que se encuentran en la práctica suelen tener ordenes de complejidad en $O(n^k)$ con k relativamente pequeño. No obstante, esta clase abarca también a los problemas con k arbitrariamente grande, que aunque suelen ser problemas poco comunes en la práctica y “artificiales”, existen y se pueden generar [14].

Definición 2.1.3. La clase $\mathcal{NP}$ es la que contiene a todos los problemas de decisión que pueden resolverse con una máquina de Turing no determinista en tiempo polinómico.

De forma alternativa, esta clase se puede definir como aquella que contiene a todos los problemas de decisión tales que, dada cualquier posible solución, se puede verificar en tiempo polinómico si esta es efectivamente una solución.

De las definiciones se deriva inmediatamente que $\mathcal{P} \subseteq \mathcal{NP}$. La duda que queda por resolverse es si $\mathcal{NP} \subseteq \mathcal{P}$, lo que, en caso de ser cierto, implicaría $\mathcal{P} = \mathcal{NP}$. Esta cuestión es la que abarca el problema de $\mathcal{P}$ vs $\mathcal{NP}$.

A pesar de no saberse a ciencia cierta si este será el caso o no, la gran mayoría de personas entendidas en el asunto sospechan que $\mathcal{P} \neq \mathcal{NP}$. Las consecuencias de que se diera la igualdad serían muy rompedoras, tanto a nivel de informática teórica como a nivel cotidiano. Por ejemplo, implicaría que problemas para los que solo sabemos cómo abordar eficientemente la tarea de verificar sus soluciones, pero no la tarea de encontrarlas, como el diseño óptimo de circuitos, o la planificación de rutas y horarios, también podrían resolverse eficientemente, lo cual tendría un impacto inmenso en campos como la criptografía, el aprendizaje automático o incluso en la resolución automática de demostraciones matemáticas.

Ahora, para ver que la inclusión no se cumple en el sentido contrario, bastaría encontrar un problema que esté en $\mathcal{NP}$ y no esté en $\mathcal{P}$. Para ello, hay ciertos problemas que parecen ser buenos candidatos.

Definición 2.1.4. Decimos que un problema $\mathcal{A}$ se puede **reducir polinómicamente** a otro problema $\mathcal{B}$ si la entrada de $\mathcal{A}$ se puede transformar, mediante un algoritmo polinómico, en una entrada de $\mathcal{B}$, de forma que la salida de $\mathcal{B}$ sea la misma que hubiera obtenido $\mathcal{A}$ con la entrada original. Esta propiedad se denota por $\mathcal{A} \leq_{\mathcal{P}} \mathcal{B}$.

Definición 2.1.5. Decimos que un problema $\mathcal{B}$ es **$\mathcal{NP}$ -completo** si cumple las dos siguientes condiciones:

1. $\mathcal{B} \in \mathcal{NP}$.
2. $\forall \mathcal{A} \in \mathcal{NP}, \mathcal{A} \leq_{\mathcal{P}} \mathcal{B}$.

En vista de esto, una forma de determinar si un problema $\mathcal{B}$ es $\mathcal{NP}$ -completo es probar $\mathcal{B} \in \mathcal{NP}$ y después escoger otro problema $\mathcal{NP}$ -completo $\mathcal{A}$ y ver que $\mathcal{A} \leq_{\mathcal{P}} \mathcal{B}$.

La utilidad de estos problemas, de manera informal, es que “si cae uno, caen todos”. Con esto lo que se pretende decir es que, si se encontrara un problema $\mathcal{NP}$ -completo $\mathcal{A}$ tal que $\mathcal{A} \in \mathcal{P}$, entonces cualquier otro problema en $\mathcal{NP}$ pertenecería también a $\mathcal{P}$.

Uno de los problemas más conocidos de este tipo es *SAT-FNC*. Este problema consiste en determinar si una fórmula en forma normal conjuntiva puede satisfacerse, y fue el primer problema cuya $\mathcal{NP}$ -completitud fue demostrada [6].

No obstante, en este estudio nos vamos a centrar en otro problema $\mathcal{NP}$ -completo, que es el problema de *Clique_{n,k}*. El problema consiste en, dado un grafo G de n vértices y cierto entero positivo k , determinar si G contiene un subgrafo completo (clique) de tamaño k . Los valores n y k son parámetros de la instancia a resolver, y no valores fijos.

En [7] se demuestra que *Clique_{n,k}* es $\mathcal{NP}$ -completo. Para ello se emplean otros problemas $\mathcal{NP}$ -completos entre medias. A partir de la $\mathcal{NP}$ -completitud de *SAT-FNC* se demuestra la $\mathcal{NP}$ -completitud de 3-SAT (una versión de *SAT-FNC* en la que todas las cláusulas tienen como mucho tres literales). Después, se ve que 3-SAT

se puede reducir al problema de recubrimiento de grafos, que consiste en determinar, para cierto k , si existe un subconjunto de k vértices tal que todas las aristas del grafo incidan en, al menos, uno de estos vértices. Finalmente, se ve que el problema de recubrimiento de vértices es equivalente al problema de $Clique_{n,k}$.

Una versión particular de este problema es $Half-Clique_n$, en el que, con la notación anterior, se toma $k = \lfloor \frac{n}{2} \rfloor$. Este problema también es $\mathcal{NP}$ -completo, y nos interesará para el estudio experimental.

Podemos demostrar fácilmente su $\mathcal{NP}$ -completitud a partir de una reducción del problema de $Clique_{n,k}$. Es decir, dado un grafo $G = (V, E)$ con $|V| = n$, queremos encontrar una transformación de coste polinómico tal que el grafo resultante $G' = (V', E')$ tenga un clique de tamaño $\frac{|V'|}{2}$ si y solo si G tiene un clique de tamaño k . Basta con hacer la siguiente reducción:

- Si $k = \frac{\lfloor n \rfloor}{2}$, basta tomar $G = G'$.
- Si $k < \frac{\lfloor n \rfloor}{2}$, añadimos $n - 2k$ vértices a V conectados con aristas a todos los vértices del grafo.
- Si $k > \frac{\lfloor n \rfloor}{2}$, añadimos $2k - n$ vértices aislados.

Así, con esta reducción, es fácil comprobar que se cumple lo que pretendíamos.

Pero ahora, teniendo estas nociones en mente, ¿cómo se puede abordar un problema como $\mathcal{P}$ vs $\mathcal{NP}$? Hay distintas vías de estudio, algunas de las cuáles han resultado considerablemente útiles en el pasado. En nuestro caso, vamos a explorar el campo de los circuitos booleanos.

2.2. Circuitos booleanos

Definición 2.2.1. Decimos que una función f es una **función booleana** si es de la forma $f : \{0, 1\}^n \rightarrow \{0, 1\}$, donde n denota el número de variables de entrada de la función. De forma análoga a los problemas de decisión, decimos que f acepta a x si $f(x) = 1$, y decimos que rechaza a x si $f(x) = 0$.

El siguiente resultado acerca de funciones booleanas, a pesar de ser básico, nos será útil en el futuro.

Proposición 2.2.1. Para cada tamaño de entrada n , hay 2^{2^n} funciones booleanas distintas.

Demostración. Fijado n , como las entradas son elementos de $\{0, 1\}^n$, hay 2^n entradas posibles. Puesto que para cada una de ellas hay que determinar la salida correspondiente, que puede ser 0 o 1, hay en total 2^{2^n} funciones booleanas, una por cada selección que se haga de la salida correspondiente a las entradas. $\square$

Con esta definición, $\mathcal{A}_f = \{x \in \{0, 1\}^n : f(x) = 1\}$ define un problema de decisión. En general, nos va a interesar tratar con problemas que dependen del tamaño de la entrada (como es el caso de $Clique_{n,k}$), de forma que para cada n

tendremos una función f_n , por lo que expresaremos problemas de decisión como $\mathcal{A} = \bigcup_{n \in \mathbb{N}} \{x \in \{0, 1\}^n : f_n(x) = 1\} = \bigcup_{n \in \mathbb{N}} \mathcal{A}_{f_n}$.

Ahora sí, estamos en posición para definir a lo que será el protagonista principal de este estudio.

Definición 2.2.2. Un **circuito booleano** C de n entradas y m salidas es un multigrafo dirigido acíclico con n fuentes y m sumideros en el que los nodos internos están etiquetados con puertas lógicas. Denotaremos por $|C|$ el número de nodos internos del circuito, a lo que nos referiremos como **tamaño del circuito**. También nos referiremos a la longitud máxima de un camino desde cualquier sumidero hasta cualquier fuente como la **profundidad del circuito**.

En la gran mayoría de los casos trabajaremos con circuitos booleanos con un único sumidero, es decir, $m = 1$, y siempre será este el caso a menos que se especifique lo contrario, lo cual nos interesará más adelante.

De manera intuitiva, un circuito booleano se puede evaluar tomando el valor de las entradas y propagándolas por el circuito, de forma que el valor que se propaga a partir de un nodo interno es el resultante de aplicar la puerta lógica con la que está etiquetado a los valores que inciden en él. Dados un circuito C y una entrada x , denotaremos el valor de evaluar el circuito en dicha entrada como $C(x)$. En el caso en que la entrada sea la representación de un grafo G , denotaremos la evaluación del circuito en dicha entrada como $C(G)$.

Así, diremos que un circuito C computa una función f si $C(x) = f(x), \forall x \in \{0, 1\}^n$.

Veamos un ejemplo de esto:

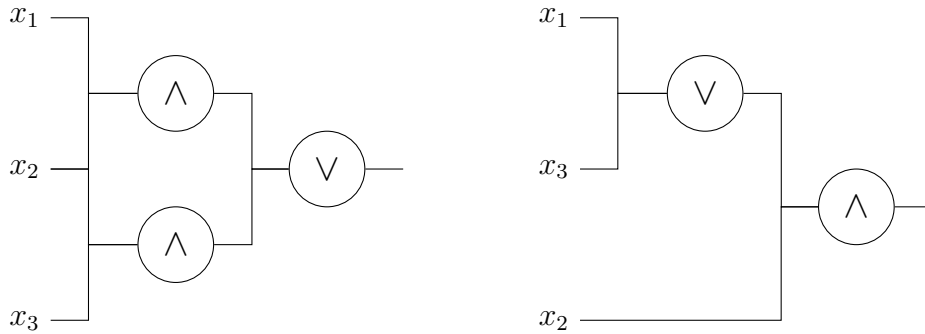


Figura 2.1: Ejemplo de circuitos booleanos equivalentes

El circuito de la izquierda computa la función $(x_1 \wedge x_2) \vee (x_2 \wedge x_3)$, mientras que el de la derecha computa la función $(x_1 \vee x_3) \wedge x_2$. La cuestión importante a notar es que las dos funciones son la misma, a pesar de ser computadas por circuitos distintos y de tamaños distintos.

Esto lleva a una noción que será importante en el futuro, que es la de **circuito mínimo**, con lo que nos referiremos a cualquier circuito minimal (de tamaño mínimo) que compute cierta función booleana f .

En esta línea, podemos definir el tamaño de una familia de circuitos.

Definición 2.2.3. Dada una familia de circuitos $\mathcal{C} = \{C_n\}_{n \in \mathbb{N}}$, donde cada C_n tiene n variables de entrada, y una función $T : \mathbb{N} \rightarrow \mathbb{N}$, decimos que $\mathcal{C}$ es una familia de circuitos **de tamaño acotado por T** si $|C_n| \leq T(n), \forall n \in \mathbb{N}$.

Previamente hablamos de las funciones que están en $\mathcal{P}$ como funciones “tratables”, por poderse computar en una cantidad polinómica de instrucciones. Como siguiente paso, resulta natural definir una noción similar para el contexto de los circuitos booleanos.

Definición 2.2.4. Dada una familia de funciones booleanas $\{f_n\}_{n \in \mathbb{N}}$, diremos que el problema de decisión $\mathcal{A} = \bigcup_{n \in \mathbb{N}} \mathcal{A}_{f_n}$ pertenece a la clase $\mathcal{P}_{/poly}$ si existen un polinomio $p(n)$ y una familia de circuitos $\{C_n\}_{n \in \mathbb{N}}$ de tamaño acotado por p tales que C_n computa f_n para cada n .

Pero, con todo esto definido, ¿de qué manera nos acerca todo esto a poder abordar el problema? Pues es el teorema siguiente el que precisamente nos va a hacer ver la utilidad en esta línea de estudio.

Teorema 2.2.2. [4] $\mathcal{P} \subset \mathcal{P}_{/poly}$.

Este teorema, aunque escueto, tiene una implicación muy importante. Más allá de decirnos que toda función computable en tiempo polinómico se puede también computar con circuitos de tamaño polinómico, nos da una forma nueva de abarcar el problema. Esta es, que si pudiéramos encontrar un problema en $\mathcal{NP}$ que no esté en $\mathcal{P}_{/poly}$, entonces tampoco estará en $\mathcal{P}$, con lo que quedaría demostrado que $\mathcal{P} \neq \mathcal{NP}$.

Además, el siguiente resultado da indicios de que esta aproximación podría resultar útil de cara a abordar el problema.

Teorema 2.2.3. [4] Para $n \geq 100$, casi todas las funciones booleanas de n entradas requieren de circuitos booleanos de tamaño, al menos, $\frac{2^n}{10n}$.

Aunque no es de excesivo interés, al revisar la demostración de este resultado se observa que tomar $n \geq 100$ no es imprescindible, sino que se pueden tomar valores significativamente menores. Aunque la noción de “casi todas las funciones” resulta algo vaga, la idea a extraer de este resultado es que las funciones computables con circuitos booleanos de tamaño polinómico son muy minoritarias.

A pesar de que no se haya logrado hasta el momento, este resultado puede dar a uno la esperanza de pensar que debería ser posible dar con un problema de decisión que requiera de circuitos de tamaño superpolinómico para ser computado.

Cabe mencionar que esto resulta útil bajo la sospecha de que $\mathcal{P} \neq \mathcal{NP}$. Aunque parezca extremadamente improbable, si se demostrara que no hay ningún problema en $\mathcal{NP}$ que no esté en $\mathcal{P}_{/poly}$, aún así no quedaría demostrado que $\mathcal{P} = \mathcal{NP}$, pues la inclusión del teorema 2.2.2 es estricta. Es decir, existen problemas en $\mathcal{P}_{/poly}$ que no están en $\mathcal{P}$. Por ejemplo, el problema de parada para máquinas de Turing denotadas en unario está en $\mathcal{P}_{/poly}$, y no está en $\mathcal{P}$ [4].

2.3. Antecedentes

El campo de los circuitos booleanos se viene estudiando desde hace ya décadas. En lo referente específicamente al problema de $Clique_{n,k}$, hay varios resultados muy

interesantes que se han descubierto en el pasado.

Por un lado, en 1985, Aleksandr Razborov demostró que, limitándonos al campo de los circuitos monótonos (aquellos en los que se permite usar puertas AND y OR, pero no puertas NOT), cualquier sucesión de circuitos que compute $Clique_{n,k}$ debe ser de tamaño exponencial [11].

Por otro lado, en 2008, Benjamin Rossman demostró que cualquier sucesión de circuitos que compute $Clique_{n,k}$ con profundidad $O(1)$ tiene que ser también de tamaño superpolinómico. En el siguiente capítulo estudiaremos con detalle este resultado.

En ambos casos parece que ambos autores se han quedado a una restricción de demostrar $\mathcal{P} \neq \mathcal{NP}$. No obstante, “levantar” estas restricciones es un reto que nadie ha logrado resolver de momento. Viendo el estancamiento que hay por la parte teórica, resulta interesante intentar abordar esta cuestión también desde una aproximación experimental.

A pesar de que los resultados experimentales generalmente no puedan constituir una demostración formal, sí que pueden llegar a resultar de enorme utilidad a la hora de obtener intuiciones, descartar ideas e intentar encauzar las vías de estudio teóricas.

En esta línea, dedicaremos una gran parte de este trabajo a hacer un estudio experimental del problema. Para abordar esta cuestión, vamos a continuar la línea de estudio del trabajo de fin de grado de Jaime Jacobo Romo González [12]. Aunque en la parte experimental explicaremos con detalle todo lo que resulte relevante de dicho trabajo para nuestro estudio, recomendamos encarecidamente su lectura, pues desarrolla otros conceptos y obtiene resultados interesantes sobre ellos.

Cota inferior para circuitos de profundidad constante que calculan $\text{Clique}_{n,k}$

En este capítulo estudiaremos el problema de $\text{Clique}_{n,k}$ desde el punto de vista de los circuitos de profundidad constante ($O(1)$). Estos circuitos $\{\mathbf{C}_n\}_{n \in \mathbb{N}}$ son aquellos para los cuales existe cierta constante entera $M > 0$ tales que la profundidad de cada $\mathbf{C}_n$ es menor o igual que M .

Nuestro objetivo con este capítulo es estudiar y explicar con todo detalle los resultados del estudio de Benjamin Rossman [13], en el cual demuestra que, dada cualquier secuencia de circuitos que compute $\text{Clique}_{n,k}$ con profundidad $O(1)$, el tamaño de estos circuitos es superpolinómico.

Reiteramos antes de nada que todos los resultados que se expondrán a continuación, salvo que se indique expresamente lo contrario, han sido tomados de [13]. No obstante, dada la complejidad del artículo original, se ha realizado un esfuerzo significativo para comentar y complementar los argumentos presentados, con el objetivo de facilitar su comprensión. Para ello, se han incluido muchas explicaciones adicionales, ejemplos ilustrativos y aclaraciones que no figuran en el texto original.

3.1. Circuitos con profundidad constante

Antes de empezar a estudiar el artículo, es interesante que nos paremos a comentar brevemente por qué la cuestión de la profundidad constante es relevante. Para ello, veamos el ejemplo de la función de PARIDAD.

La función de PARIDAD es aquella que determina si la cantidad de entradas con valor 1 es par o impar. A nivel algorítmico, esta función es evidentemente muy fácil de computar.

En el campo de los circuitos booleanos, es también sencilla de computar. Hay dos formas de hacerlo intuitivas: una de profundidad lineal, y otra de profundidad logarítmica. En ambos casos, se computa fácilmente con puertas XOR.

El caso lineal consiste en unir los dos primeros inputs con una puerta XOR, unir la salida de dicha puerta con el tercer input con otra puerta XOR, la salida de esta puerta con el cuarto input, . . . Véase en el siguiente ejemplo, para el caso de 4 inputs:

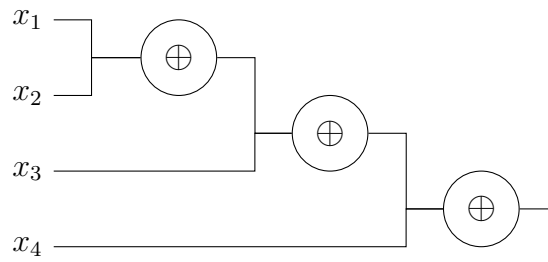


Figura 3.1: Circuito que computa PARIDAD con profundidad lineal

Vemos que en este caso la profundidad del circuito es 3. En general, para tamaño de entrada n se puede generar un circuito como este de profundidad $n - 1 \in O(n)$.

Ahora, veamos la otra forma de computar fácilmente PARIDAD:

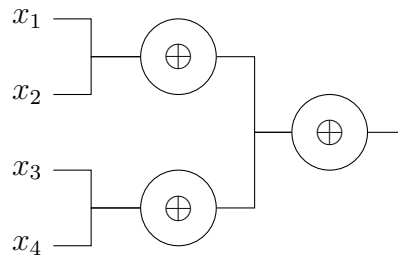


Figura 3.2: Circuito que computa PARIDAD con profundidad logarítmica

En este caso el circuito tiene profundidad 2. Este circuito tiene estructura de árbol binario equilibrado, y es fácil extender esta estructura a cualquier tamaño de entrada. Así, en general se obtiene un circuito de profundidad $\lceil \log_2(n) \rceil \in O(\log(n))$.

En cualquier caso, ambos circuitos se han obtenido empleando 3 puertas XOR, y en general hacen falta $n - 1 \in O(n)$. Así, ambas estructuras dan lugar a circuitos de tamaño polinómico.

Ahora, ¿qué pasaría si nos limitamos a profundidad constante? A fin de cuentas, cualquier función booleana se puede computar con un circuito de profundidad 2, si permitimos que las puertas tengan aridad arbitraria: basta tomar todas las entradas que tengan 1 como output en la tabla de verdad de la función, utilizar para cada una de ellas una puerta AND que tenga por entrada las variables negadas o sin negar, y unir la salida de todas las puertas AND con una puerta OR. Esto esencialmente se puede pensar como computar la función en forma normal disyuntiva. La cuestión es que no tenemos ninguna garantía de que este circuito vaya a ser “pequeño” o, más concretamente, de tamaño polinómico.

En el caso de la función de PARIDAD, está demostrado que cualquier sucesión de circuitos que compute dicha función con profundidad constante debe ser de tamaño exponencial [3]. Es decir, es muy fácil computar PARIDAD con profundidad logarítmica y tamaño polinómico, pero al reducirnos al caso de profundidad constante, no solo no es fácil, sino que es imposible.

Esto nos ilustra que la profundidad de los circuitos es un elemento realmente importante de cara a computar funciones booleanas con circuitos booleanos.

3.2. Preliminares al estudio de Rossman

Antes de pasar a los resultados, es relevante que establezcamos primero la notación que vamos a usar en lo que sigue:

- Definimos $[n] = \{1, 2, \dots, n\}$.
- Dados un conjunto A y un entero k , denotamos con $\binom{A}{k}$ al conjunto de todos los subconjuntos de A de tamaño k .
- Dado un conjunto de vértices A , denotamos con K_A el clique formado por todos los vértices de A .
- Denotamos con $\mathcal{G}_n$ al conjunto de todos los grafos de n vértices.
- Denotamos por $\text{ER}(n, q)$ la distribución *Erdős-Rényi*. Esta es una distribución de grafos con conjunto de vértices $[n]$ tales que cada arista está incluida de forma independiente con probabilidad q . Por ejemplo, con esta definición la probabilidad de obtener un grafo completo es $q^{n(n-1)}$ y la de obtener un grafo sin aristas es $(1 - q)^{n(n-1)}$.
- Dados dos grafos G y H , denotamos con $\text{sub}(G, H)$ al número de subgrafos de G isomorfos a H .
- Se define la clase AC^0 como la clase de problemas que se pueden resolver con circuitos de tamaño polinómico y profundidad constante, donde las puertas lógicas pueden tener cualquier aridad.
- Diremos que un suceso se cumple *asintóticamente casi seguro* cuando la probabilidad del suceso contrario tiende a 0 a medida que n tiende a ∞ .
- Dado un grafo G , denotamos con $H \subseteq G$ que H es un subgrafo de G . Con esto, vamos a llamar *umbral exponencial de G* al valor $\min_{H \subseteq G} \frac{|V_H|}{|E_H|}$, donde $|V_H|$ y $|E_H|$ son el número de vértices y aristas en H , respectivamente. Denotaremos a este valor como $\text{thres}(G)$. En caso de que G no tenga aristas diremos que $\text{thres}(G) = \infty$.

En cuanto a este último término, es importante que veamos un ejemplo, que es el caso particular de los cliques.

Ejemplo 3.2.1 (Umbral exponencial de un clique). *Sea K_n un clique de tamaño n . Dado cualquier subgrafo $H \subseteq K_n$ de tamaño k , este es un clique de tamaño k . Así,*

$$\frac{|V_H|}{|E_H|} = \frac{k}{\binom{k}{2}} = \frac{2}{k-1}.$$

Por tanto,

$$\text{thres}(K_n) = \min_{H \subseteq K_n} \frac{|V_H|}{|E_H|} = \min_{k \in \{1, \dots, n\}} \frac{2}{k-1} = \frac{2}{n-1}$$

pues $\frac{2}{k-1}$ es decreciente con respecto a k .

También es importante que mencionemos un resultado que se usa en este estudio, a pesar de que su demostración se omite:

Lema 3.2.1. *Dado un grafo H y un valor $\alpha > 0$, se cumple lo siguiente para cualquier grafo $G \in \text{ER}(n, n^{-\alpha})$:*

1. Si $\alpha > \text{thres}(H) \Rightarrow \Pr[\text{sub}(G, H) > 0] = \exp(-n^{\Omega(1)})$.
2. Si $\alpha < \text{thres}(H) \Rightarrow \forall \varepsilon > 0, \Pr[\text{sub}(G, H) < n^{|V_H| - \alpha|E_H| - \varepsilon}] = \exp(-n^{\Omega(1)})$.

Aunque este lema parece realmente complejo y fuera de contexto, nos resultará imprescindible más adelante.

El estudio de Rossman es considerablemente complicado y técnico. Para entender mejor lo que se hace en él, vamos a explicar cuál va a ser el objetivo de los resultados, pues consideramos que puede facilitar enormemente el entendimiento del estudio.

Empleando el lema 3.2.1 que acabamos de presentar, tomando como subgrafo H un clique de tamaño k , si logramos situarnos en las condiciones del primer apartado del lema, lo que nos viene a decir este resultado es que, a medida que crece n (el tamaño de los grafos), la probabilidad de que un grafo contenga un clique de tamaño k es ridículamente baja.

Ahora, ¿cómo relacionamos esto con la función que computa $Clique_{n,k}$? La cuestión es que, durante la mayoría del estudio, no nos vamos a fijar en esta función, sino que vamos a fijarnos en funciones aleatorias que, por hipótesis, se puedan computar con circuitos de tamaño polinómico y profundidad constante. Al fijarnos en estas funciones, el objetivo final va a ser demostrar que, a medida que crece n , es casi seguro que el output de las funciones **no depende de la presencia de un clique**. Es decir, que si tenemos un grafo G y un clique K , es casi seguro que, dada una de estas funciones f , se cumple que $f(G) = f(G \cup K)$.

¿Y a qué nos llevaría ver esto? Pues, por un lado, tenemos que los grafos “grandes” casi nunca tienen cliques de tamaño k . Por otro lado, las funciones computables con circuitos de tamaño polinómico y profundidad constante casi siempre generan un output independiente de la presencia de un clique. Lo que se deduce de esto es que ninguna de estas funciones puede estar computando $Clique_{n,k}$ pues, si tomamos un grafo G aleatorio, es casi seguro que este no contiene un clique de tamaño k , y, sin embargo, $G \cup K$ contiene claramente un clique (pues “se lo hemos añadido”), por lo que obtener $f(G) = f(G \cup K)$ nos dice que f no puede estar reflejando la propiedad de que un grafo contenga o no un clique de tamaño k .

Puesto que esto se habrá obtenido bajo la premisa de que las funciones sean computables con profundidad constante y tamaño polinómico, se deducirá que cualquier circuito que compute $Clique_{n,k}$ no podrá cumplir estas condiciones. Así, limitándonos a profundidad constante, los circuitos deberán ser de tamaño superpolinómico.

La demostración de estos resultados será lo último que se obtenga en el estudio, pero los enunciamos a continuación:

Teorema 3.2.2. Sea $f_n : \{0, 1\}^{\binom{n}{2}} \rightarrow \{0, 1\}$ una secuencia de funciones computables con circuitos de profundidad constante y tamaño $O(n^t)$, con $t > \frac{1}{2}$. Para cualquier $k \in \mathbb{N}$ y $0 < \alpha \leq \frac{1}{2t-1}$, sea $G = \text{ER}(n, n^{-\alpha})$ un grafo aleatorio, y sea A un conjunto aleatorio de k vértices. Entonces, se cumple asintóticamente casi seguro que $f_n(G) = f_n(G \cup K_A)$.

Teorema 3.2.3. Para cualquier k , cualquier sucesión de circuitos de profundidad constante que compute $\text{Clique}_{n,k}$ debe tener tamaño $\omega(n^{\frac{k}{4}})$.

Teniendo este mapa mental de la dirección en la que se orienta el estudio, podemos pasar ya a estudiarlo con detalle.

3.3. La técnica de estudio

En este apartado demostraremos unas versiones algo más débiles de los resultados finales, pero que, aparte de ser suficiente por sí mismos, ilustran muy bien la técnica que se usará a lo largo de las explicaciones y simplificará las demostraciones de las versiones fuertes de los teoremas principales.

3.3.1. Sensibilidad a clique

Para empezar, presentamos una definición de lo que será el principal objeto de estudio a lo largo de todo el trabajo.

Definición 3.3.1. Sea f una función de grafos (de dominio $\mathcal{G}_n$ e imagen arbitraria) y sea G un grafo de n vértices. Dado $A \subseteq [n]$ y $s \in \mathbb{N}$, definimos los siguientes conjuntos:

$$\mathbb{T}^{f,G}(A) = \{a \in A \mid \exists B \subseteq A \text{ tal que } f(G \cup K_B) \neq f(G \cup K_{B \setminus \{a\}})\}$$

$$\mathbb{T}_{<s>}^{f,G}(A) = \bigcup_{B \subseteq A: |B| \leq s} \mathbb{T}^{f,G}(B)$$

Llamaremos **núcleo sensible a clique** de A a $\mathbb{T}^{f,G}(A)$ y **núcleo sensible a clique acotado a s** de A a $\mathbb{T}_{<s>}^{f,G}(A)$.

Aparte, diremos que A es **totalmente sensible a clique** si $\mathbb{T}^{f,G}(A) = A$, y diremos que A es **totalmente sensible a cliques de tamaño s** si $\mathbb{T}_{<s>}^{f,G}(A) = A$.

De forma intuitiva, se puede pensar en $\mathbb{T}^{f,G}(A)$ como el subconjunto de vértices de A tales que, al añadir el clique apoyado en dichos vértices, la función f deja de ser sensible a la adición de cualquier otro clique apoyado en vértices de A . De forma similar, $\mathbb{T}_{<s>}^{f,G}(A)$ refleja esto mismo pero refiriéndose a cliques de tamaño menor o igual que s .

Esta noción es ciertamente difícil de visualizar, así que veamos un ejemplo para intentar darle forma:

Ejemplo 3.3.1. Sea f_{clique} la función que computa $\text{Clique}_{n,k}$, es decir, la función que determina si un grafo de tamaño n contiene un clique de tamaño k . Sea G el grafo de n vértices sin ninguna arista.

Dado $A \subseteq [n]$ tenemos las dos siguientes igualdades:

$$\mathbb{T}^{f_{\text{clique}}, G}(A) = \begin{cases} A, & \text{si } |A| \geq k \\ \emptyset, & \text{en caso contrario} \end{cases} \quad \mathbb{T}_{<s>}^{f_{\text{clique}}, G}(A) = \begin{cases} A, & \text{si } |A| \geq k \text{ y } s \geq k \\ \emptyset, & \text{en caso contrario} \end{cases}$$

En el primer caso, si $|A| \geq k$, entonces, dado cualquier vértice a , podemos añadir a G un clique de tamaño exactamente k que contenga al vértice a , de forma que al eliminar dicho vértice, el clique pasa a tener tamaño $k - 1$. Así, el grafo pasa de tener un clique de tamaño k a no tenerlo, por lo que cambia la salida de f .

El segundo caso es casi idéntico, excepto porque para poder usar el mismo razonamiento, necesitamos que $s \geq k$, puesto que, en caso contrario, ningún clique que añadamos a G hará que cambie la salida de f .

En lo que resta de este apartado vamos a ver ciertos lemas que nos van a permitir manejar mucho mejor estos conjuntos que acabamos de definir, pero antes de llegar a eso viene bien exponer varias propiedades que estos conjuntos cumplen por su propia definición.

Observación 3.3.1. Se cumplen las siguientes propiedades:

1. Si $|A| \leq s$, entonces $\mathbb{T}_{<s>}^{f, G}(A) = \mathbb{T}^{f, G}(A)$.
2. $\mathbb{T}_{<s>}^{f, G}(A) \subseteq \mathbb{T}^{f, G}(A)$, para todo $A \subseteq [n]$.
3. $\mathbb{T}^{f, G}(A)$ y $\mathbb{T}_{<s>}^{f, G}(A)$ son monótonos. Es decir, $A \subseteq B$ implica que $\mathbb{T}^{f, G}(A) \subseteq \mathbb{T}^{f, G}(B)$ y $\mathbb{T}_{<s>}^{f, G}(A) \subseteq \mathbb{T}_{<s>}^{f, G}(B)$.
4. Si f es una función con una salida de más de un bit, es decir, $f : \mathcal{G}_n \rightarrow \{0, 1\}^m$, siendo $f_1, \dots, f_m : \mathcal{G}_n \rightarrow \{0, 1\}$ las funciones en cada coordenada, entonces $\mathbb{T}^{f, G}(A) = \bigcup_{i=1}^m \mathbb{T}^{f_i, G}(A)$ y $\mathbb{T}_{<s>}^{f, G}(A) = \bigcup_{i=1}^m \mathbb{T}_{<s>}^{f_i, G}(A)$.
5. Si dos conjuntos A y B son totalmente sensibles a clique bajo f y G , entonces la unión $A \cup B$ también lo es.

Ahora sí, podemos pasar a ver ciertas propiedades un poco más complicadas de $\mathbb{T}^{f, G}(A)$ y $\mathbb{T}_{<s>}^{f, G}(A)$.

Lema 3.3.1. Sea $T = \mathbb{T}_{<s>}^{f, G}(A)$ y sea B un conjunto tal que $T \subseteq B \subseteq A$ y $|B| \leq s$. Entonces $f(G \cup K_T) = f(G \cup K_B)$.

Demostración. Denotemos $B \setminus T = \{b_1, \dots, b_m\}$, y sea $i \in \{1, \dots, m\}$. Veamos que

$$f(G \cup K_{T \cup \{b_1, \dots, b_i\}}) = f(G \cup K_{T \cup \{b_1, \dots, b_{i-1}\}}).$$

Supongamos que no. En ese caso, $b_i \in \mathbb{T}_{<s>}^{f, G}(B)$, por definición. Ahora, como $|B| \leq s$, se tiene que $\mathbb{T}_{<s>}^{f, G}(B) = \mathbb{T}^{f, G}(B)$. Aparte, como $B \subseteq A$, por la monotonía

de $\mathbb{T}$ tenemos que $\mathbb{T}_{<s>}^{f,G}(B) \subseteq \mathbb{T}_{<s>}^{f,G}(A) = T$. Por tanto, $b_i \in T$, pero $b_i \in B \setminus T$, lo cual es una contradicción.

Por tanto, se cumple la igualdad anterior. Aplicando esa igualdad reiteradamente, llegamos a que $f(G \cup K_T) = f(G \cup K_B)$, como queríamos probar. $\square$

Con el siguiente resultado lo que vamos a ver es que, dado cualquier conjunto A , podemos descomponerlo en pequeños “trozos” (no necesariamente disjuntos), tales que cada uno de ellos sea totalmente sensible a cliques de tamaño s .

Lema 3.3.2. *El conjunto $\mathbb{T}_{<s>}^{f,G}(A)$ es la unión de todos los subconjuntos totalmente sensibles a cliques de tamaño s de A . Esto es,*

$$\mathbb{T}_{<s>}^{f,G}(A) = \bigcup \{B \subseteq A : \mathbb{T}_{<s>}^{f,G}(B) = B \text{ y } |B| \leq s\}$$

Demostración. La inclusión $\supseteq$ es obvia, por lo que solo necesitamos demostrar $\subseteq$. Para lograr esto, dado cualquier $a \in \mathbb{T}_{<s>}^{f,G}(A)$, queremos encontrar un subconjunto totalmente sensible a cliques de tamaño s $B \subseteq A$ tal que $|B| \leq s$ y $a \in B$.

Como $a \in \mathbb{T}_{<s>}^{f,G}(A)$, sabemos que existe cierto $B \subset A$ tal que $f(G \cup K_B) \neq f(G \cup K_{B \setminus \{a\}})$ y $|B| \leq s$. De entre todos los subconjuntos que cumplan esto, tomemos un subconjunto B minimal. Lo que queremos ver es que este subconjunto es efectivamente totalmente sensible a cliques de tamaño s .

Supongamos lo contrario. En tal caso, existe $b \in B \setminus \mathbb{T}_{<s>}^{f,G}(B)$. Para empezar, debido a que hemos tomado B minimal, se debe cumplir que

$$f(G \cup K_{B \setminus \{b\}}) = f(G \cup K_{B \setminus \{a,b\}}).$$

Además, como $b \notin \mathbb{T}_{<s>}^{f,G}(B)$, entonces

$$f(G \cup K_B) = f(G \cup K_{B \setminus \{b\}}).$$

Finalmente, puesto que $\mathbb{T}_{<s>}^{f,G}(B \setminus \{a\}) \subseteq \mathbb{T}_{<s>}^{f,G}(B)$ por la monotonía de $\mathbb{T}$,

$$b \notin \mathbb{T}_{<s>}^{f,G}(B) \Rightarrow b \notin \mathbb{T}_{<s>}^{f,G}(B \setminus \{a\}) \Rightarrow f(G \cup K_{B \setminus \{a\}}) = f(G \cup K_{B \setminus \{a,b\}})$$

Juntando todas estas igualdades obtenemos $f(G \cup K_B) = f(G \cup K_{B \setminus \{a\}})$, lo cual contradice la elección inicial de B .

Por tanto, la suposición de que B no era totalmente sensible a cliques de tamaño s era errónea, con lo que queda demostrado el resultado. $\square$

El último resultado de este apartado trata acerca de la existencia de subconjuntos totalmente sensibles a cliques de tamaño s de tamaños no triviales. De cara a las demostraciones de los teoremas principales, el segundo apartado del siguiente resultado será fundamental. Para demostrar unas versiones más débiles, nos centraremos más en el tercer apartado.

Lema 3.3.3. *Se cumplen las siguientes propiedades:*

1. $\mathbb{T}_{<s>}^{f,G}(A)$ es no vacío $\Leftrightarrow A$ tiene un subconjunto B totalmente sensible a clique tal que $2 \leq |B| \leq s$.

2. $|\mathbb{T}_{<s>}^{f,G}(A)| > s \Leftrightarrow A$ tiene subconjuntos B y C totalmente sensibles a clique de tamaños $\leq s$ tales que $|B \cup C| > s + 1$.
3. Si $|\mathbb{T}_{<s>}^{f,G}(A)| > s$, entonces A tiene un subconjunto D totalmente sensible a clique tal que $s + 1 \leq |D| \leq 2s$.

Demostración. De nuevo, las implicaciones $\Leftarrow$ en (1) y (2) son evidentes. Veamos las implicaciones $\Rightarrow$ de (1), (2) y (3):

Sean $A_1, \dots, A_m, m \in \mathbb{N}$ todos los subconjuntos totalmente sensibles a clique de A no vacíos y de tamaño $\leq s$ (estos subconjuntos no tienen por qué ser disjuntos). Por lo visto en el lema 3.3.2, $\mathbb{T}_{<s>}^{f,G}(A) = A_1 \cup \dots \cup A_m$.

Para demostrar (1), supongamos que $\mathbb{T}_{<s>}^{f,G}(A)$ es no vacío, es decir, $m \geq 1$. Sea $B = A_1$. Queremos ver que $|B| \geq 2$. Supongamos, en cambio, que $|B| < 2$. En tal caso, como B es no vacío, $B = \{b\}$, para cierto b , y consecuentemente $\mathbb{T}^{f,G}(\{b\}) = \{b\}$. Por definición de $\mathbb{T}$, existe $C \subseteq \{b\}$ tal que $f(G \cup K_C) \neq f(G \cup K_{C \setminus \{b\}})$, que en este caso claramente debe ser $C = \{b\}$. Pero entonces $f(G \cup K_{\{b\}}) \neq f(G \cup K_\emptyset)$, lo cual es absurdo, pues $K_{\{b\}}$ es el clique apoyado únicamente sobre el vértice b , y este no tiene aristas, por lo que $G \cup K_{\{b\}} = G \cup K_\emptyset = G$. Por tanto, $|B| \geq 2$, quedando demostrada la implicación $\Rightarrow$ de (1).

Ahora, para demostrar (2), supongamos que $|\mathbb{T}_{<s>}^{f,G}(A)| > s$. Sea $l \in \{1, \dots, m\}$ el menor índice tal que $|A_1 \cup \dots \cup A_l| > s$. Vemos que l está bien definido, pues $|A| = |A_1 \cup \dots \cup A_m| > s$, y $l > 1$, pues $|A_i| \leq s, \forall i = 1, \dots, m$. Así, sea $B = A_1 \cup \dots \cup A_{l-1}$ y sea $C = A_l$. Por la elección de l se cumple que $|B|, |C| \leq s$ y $|B \cup C| = |A| \geq s + 1$. Además, B es totalmente sensible a clique por la observación 3.3.1 (5), pues es la unión de conjuntos totalmente sensibles a clique, y también lo es C , con lo que queda probada la implicación $\Rightarrow$ de (2).

Finalmente, (3) sale de forma inmediata tomando $D = B \cup C$. $\square$

3.3.2. El lema del circuito

Antes de seguir, debemos pararnos de nuevo a establecer ciertas notaciones:

Sea C un circuito con un único nodo de salida y $\binom{n}{2}$ entradas. Para cada nodo ν en C denotamos por C_ν al circuito cuyo nodo de salida es ν . Denotamos también por $C_\nu^\bullet$ el circuito C_ν en el cual no solo ν sino todos los hijos se designan como salidas. Así, C y C_ν computan funciones $\mathcal{G}_n \rightarrow \{0, 1\}$, mientras que $C_\nu^\bullet$ computa una función $\mathcal{G}_n \rightarrow \{0, 1\}^{\text{aridad}(\nu)+1}$.

Para simplificar la notación, sean $\mathcal{T}(\nu) = \mathbb{T}_{<s>}^{C_\nu, G}(A)$ y $\mathcal{T}^\bullet(\nu) = \mathbb{T}_{<s>}^{C_\nu^\bullet, G}(A)$. Ha de hacerse notar que, por el lema 3.3.1 (4), $\mathcal{T}^\bullet(\nu) = \mathcal{T}(\nu) \cup \bigcup_{\text{hijos } \mu \text{ de } \nu} \mathcal{T}(\mu)$.

Con esto estamos ya en posición para enunciar el resultado principal de este apartado.

Lema 3.3.4 (Lema del circuito). *Sea G un grafo de n vértices y sea $A \subseteq [n]$. Supongamos que $\mathbb{T}_{<s>}^{C, G}(A) = \emptyset$ y que $|\mathbb{T}_{<s>}^{C_\nu^\bullet, G}(A)| \leq s$ para todos los nodos ν en C , con $s \geq 1$. Entonces $C(G) = C(G \cup K_A)$.*

Aunque los resultados que vimos anteriormente eran cuestiones meramente utilitarias para emplear en otras demostraciones, este resultado ya se acerca mucho a la

dirección que describimos antes de empezar a estudiar con detalle los resultados. Es la primera instancia donde nos encontramos la idea de que los cliques no afecten al output de un circuito. Teniendo esto, lo que buscaremos conseguir posteriormente será situarnos en las condiciones que requiere este resultado para ser aplicado.

Ahora, antes de demostrar este resultado, nos viene bien demostrar antes otro que nos va a simplificar la tarea después:

Proposición 3.3.5. *Bajo las condiciones anteriores, para todo nodo ν en $\mathcal{C}$,*

$$\mathcal{C}_\nu(G \cup K_{\mathcal{T}^\bullet(\nu)}) = \mathcal{C}_\nu(G \cup K_A).$$

Demostración. Vamos a trabajar empleando inducción sobre la profundidad del nodo ν .

Veamos que cuando ν es un nodo de entrada del circuito, el resultado es inmediato. En este caso, ν se corresponde con cierta arista e , por lo que $\mathcal{T}^\bullet(\nu) = \{e\}$ si $e \subseteq A$ y $\emptyset$ si $e \not\subseteq A$. Así, $\mathcal{C}_\nu$ (que sencillamente determina si la arista e está o no presente en el grafo de entrada) tiene el mismo valor en los grafos $G \cup K_{\mathcal{T}^\bullet(\nu)}$ y $G \cup K_A$, pues la arista o bien está en ambos o en ninguno.

Para el paso inductivo, tomemos ν un nodo interno del circuito (que no sea una entrada). Por hipótesis de inducción, $\mathcal{C}_\mu(G \cup K_{\mathcal{T}^\bullet(\mu)}) = \mathcal{C}_\mu(G \cup K_A)$ para cada hijo μ de ν . Fijándonos en algún hijo concreto μ , dado que $\mathcal{T}(\mu) \subseteq \mathcal{T}^\bullet(\nu) \subseteq A$ y $|\mathcal{T}^\bullet(\nu)| \leq s$, el lema 3.3.1 implica que

$$\mathcal{C}_\mu(G \cup K_{\mathcal{T}(\mu)}) = \mathcal{C}_\mu(G \cup K_{\mathcal{T}^\bullet(\nu)}).$$

Teniendo en cuenta también que $\mathcal{T}(\mu) \subseteq \mathcal{T}^\bullet(\mu) \subseteq A$ y $|\mathcal{T}^\bullet(\mu)| \leq s$, el mismo lema nos dice que

$$\mathcal{C}_\mu(G \cup K_{\mathcal{T}(\mu)}) = \mathcal{C}_\mu(G \cup K_{\mathcal{T}^\bullet(\mu)}).$$

Además, por hipótesis de inducción se cumple que

$$\mathcal{C}_\mu(G \cup K_{\mathcal{T}^\bullet(\mu)}) = \mathcal{C}_\mu(G \cup K_A).$$

Juntando todas estas igualdades obtenemos que $\mathcal{C}_\mu(G \cup K_{\mathcal{T}^\bullet(\nu)}) = \mathcal{C}_\mu(G \cup K_A)$. Finalmente, como el valor de $\mathcal{C}_\nu$ viene determinado por los valores de $\mathcal{C}_\mu$ para cada hijo μ , concluimos que $\mathcal{C}_\nu(G \cup K_{\mathcal{T}^\bullet(\nu)}) = \mathcal{C}_\nu(G \cup K_A)$. $\square$

Ahora sí, podemos demostrar el resultado anterior.

Demostración del lema 3.3.4. Denotemos por ν_{out} el nodo de salida de $\mathcal{C}$ (es decir, $\mathcal{C} = \mathcal{C}_{\nu_{out}}$). Aplicando el resultado anterior a ν_{out} tenemos que $\mathcal{C}(G \cup K_{\mathcal{T}^\bullet(\nu_{out})}) = \mathcal{C}(G \cup K_A)$. Además, como $\mathcal{T}(\nu_{out}) \subseteq \mathcal{T}^\bullet(\nu_{out}) \subseteq A$ y $|\mathcal{T}^\bullet(\nu_{out})| \leq s$, el lema 3.3.1 nos dice que $\mathcal{C}(G \cup K_{\mathcal{T}(\nu_{out})}) = \mathcal{C}(G \cup K_{\mathcal{T}^\bullet(\nu_{out})})$. Finalmente, usando la hipótesis de que $\mathbb{T}_{<s>}^{\mathcal{C}, G}(A) = \emptyset$, obtenemos que $G \cup K_{\mathcal{T}(\nu_{out})} = G \cup K_\emptyset = G$, por lo que $\mathcal{C}(G) = \mathcal{C}(G \cup K_A)$, concluyendo así la demostración. $\square$

3.3.3. Demostración de las versiones débiles de los teoremas principales

En esta sección vamos a demostrar unas versiones un poco más débiles de los teoremas principales del artículo.

El primer resultado se demostrará más adelante, pero lo enunciamos a continuación:

Lema 3.3.6 (Lema técnico principal). *Sea $f = (f_n)_{n \in \mathbb{N}}$ una secuencia de funciones $f_n : \{0, 1\}^{\binom{n}{2}} \rightarrow \{0, 1\}^{n^\beta}$ computables con circuitos de profundidad constante y tamaño polinómico (AC^0), con cierto $\beta \geq 0$. Sea $\alpha > 0$ y $k \in \mathbb{N}$. Entonces, dado $G \in \text{ER}(n, n^{-\alpha})$ y A un subconjunto aleatorio de $[n]$ de tamaño k , se cumple que*

$$\Pr [\mathbb{T}^{f,G}(A) = A] = n^{\alpha \binom{k}{2} + (\beta-1)k + o(1)}.$$

Aunque no sea excesivamente relevante de momento, fijémonos en una cuestión: este resultado es el primer punto del estudio en el que hemos empleado la hipótesis de la profundidad constante.

Este resultado, a falta de demostración, nos interesa inmediatamente para el siguiente corolario:

Corolario 3.3.7. *Tomando f , β , α , k , G y A como en el lema anterior, se verifican las siguientes igualdades:*

1. $\Pr [\mathbb{T}_{<s>}^{f,G}(A) \neq \emptyset] = n^{\max(\alpha+2(\beta-1), \alpha \binom{s}{2} + s(\beta-1)) + o(1)}.$
2. $\Pr [|\mathbb{T}_{<s>}^{f,G}(A)| > s] = n^{\max(\alpha \binom{s+1}{2} + (\beta-1)(s+1), \alpha \binom{2s}{2} + 2s(\beta-1)) + o(1)}.$

Demostración. Vamos a demostrar primero el segundo resultado, pues es el único que está demostrado en el documento original, aunque a partir de esta demostración haremos fácilmente la del primer resultado.

Para empezar, por el lema 3.3.3 (3):

$$\Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ A \in \binom{[n]}{k}}} [|\mathbb{T}_{<s>}^{f,G}(A)| > s] \leq \Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ A \in \binom{[n]}{k}}} \left[\bigvee_{B \subseteq A: s < |B| \leq 2s} \mathbb{T}^{f,G}(B) = B \right].$$

Por ser esto una unión de sucesos,

$$\Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ A \in \binom{[n]}{k}}} \left[\bigvee_{B \subseteq A: s < |B| \leq 2s} \mathbb{T}^{f,G}(B) = B \right] \leq \sum_{r=s+1}^{2s} \binom{k}{r} \Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ B \in \binom{[n]}{r}}} [\mathbb{T}^{f,G}(B) = B].$$

Ahora, aplicando el lema 3.3.6,

$$\sum_{r=s+1}^{2s} \binom{k}{r} \Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ B \in \binom{[n]}{r}}} [\mathbb{T}^{f,G}(B) = B] = \sum_{r=s+1}^{2s} \binom{k}{r} n^{\alpha \binom{r}{2} + (\beta-1)r + o(1)}.$$

Finalmente, tenemos una suma de potencias. Así, sin entrar en excesivos detalles, el sumando de exponente máximo domina sobre el resto, de forma que se pueden obtener los demás con el $o(1)$ del exponente. Con eso, dado que en la fórmula que empleamos del lema 3.3.6 el exponente es monótono con respecto a k , el exponente máximo de este sumatorio debe ser o el primero o el último. Así,

$$\sum_{r=s+1}^{2s} \binom{k}{r} n^{\alpha \binom{r}{2} + (\beta-1)r + o(1)} = n^{\max(\alpha \binom{s+1}{2} + (\beta-1)(s+1), \alpha \binom{2s}{2} + 2s(\beta-1)) + o(1)}.$$

Aparte, este mismo término $o(1)$ nos permite que el resultado sea una igualdad, en vez de una desigualdad.

Hagamos ahora la demostración del segundo resultado, que no está incluída en el documento original, pero es análoga a la que acabamos de hacer.

Para empezar, por el lema 3.3.3 (1),

$$\Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ A \in \binom{[n]}{k}}} [\mathbb{T}_{<s>}^{f,G}(A) \neq \emptyset] \leq \Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ A \in \binom{[n]}{k}}} \left[\bigvee_{B \subseteq A: 2 \leq |B| \leq s} \mathbb{T}^{f,G}(B) = B \right]$$

A partir de aquí el resto de pasos son completamente análogos. Por ser esto último una unión de sucesos,

$$\Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ A \in \binom{[n]}{k}}} \left[\bigvee_{B \subseteq A: 2 \leq |B| \leq s} \mathbb{T}^{f,G}(B) = B \right] \leq \sum_{r=2}^s \binom{k}{r} \Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ B \in \binom{[n]}{r}}} [\mathbb{T}^{f,G}(B) = B]$$

Empleando una vez más el lema 3.3.3,

$$\sum_{r=2}^s \binom{k}{r} \Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ B \in \binom{[n]}{r}}} [\mathbb{T}^{f,G}(B) = B] = \sum_{r=2}^s \binom{k}{r} n^{\alpha \binom{r}{2} + (\beta-1)r + o(1)}.$$

Finalmente, por el mismo argumento que antes,

$$\sum_{r=2}^s \binom{k}{r} n^{\alpha \binom{r}{2} + (\beta-1)r + o(1)} = n^{\max(\alpha \binom{s+1}{2} + (\beta-1)(s+1), \alpha \binom{2s}{2} + 2s(\beta-1)) + o(1)}.$$

De nuevo, el término $o(1)$ del exponente convierte el resultado en una igualdad. $\square$

Una cuestión interesante de este corolario es que la fórmula que obtenemos no depende de k más allá del término $o(1)$.

Previamente, en el lema 3.3.4 vimos ya la idea de que el output de un circuito no dependa de la presencia de un clique, y mencionamos que ahora el objetivo sería situarnos en las condiciones necesarias para aplicar dicho lema. Si nos fijamos, las condiciones son precisamente los sucesos opuestos a los que se analizan en el corolario que acabamos de demostrar. Así, este último resultado se utilizará para ver que las condiciones necesarias para aplicar 3.3.4 se cumplen *casi seguro*.

Con todo esto, ahora sí, vamos con los resultados principales de este apartado:

Teorema 3.3.8 (Versión débil del teorema 3.2.2). *Supongamos que $f = (f_n)_{n \in \mathbb{N}}$ es una secuencia de funciones $f_n : \{0, 1\}^{\binom{n}{2}} \rightarrow \{0, 1\}$ computables por circuitos de profundidad $O(1)$ y tamaño $O(n^t)$ para cierta constante $t > \frac{2}{3}$. Sea $G = \text{ER}(n, n^{-\alpha})$ un grafo de Erdős-Rényi aleatorio y sea $A \in \binom{[n]}{k}$ un subconjunto aleatorio de tamaño k , para ciertas constantes $0 < \alpha \leq \frac{4}{9t-3}$ y $k \in \mathbb{N}$. Entonces $f_n(G) = f_n(G \cup K_A)$ asintóticamente casi seguro, a medida que $n \rightarrow \infty$.*

Demostración. Sea $s = \lfloor \frac{3t}{2} \rfloor$. Se puede comprobar que,

$$\begin{aligned} \max \left(\alpha - 2, \alpha \binom{s}{2} - s \right) &< 0, \\ \max \left(\alpha \binom{s+1}{2} - (s+1), \alpha \binom{2s}{2} - 2s \right) &< -t. \end{aligned}$$

Dado que estas desigualdades son estrictas (en particular, la segunda), se cumple, por continuidad, que existe cierto $\beta > 0$ tal que

$$\max \left(\alpha \binom{s+1}{2} + (\beta - 1)(s+1), \alpha \binom{2s}{2} + 2s(\beta - 1) \right) < -t.$$

Ahora, sea $\mathbf{C} = (\mathbf{C}_n)_{n \in \mathbb{N}}$ una sucesión de circuitos que computa f con profundidad $O(1)$ y tamaño $O(n^t)$. Hasta el momento no hemos puesto ninguna restricción a la aridad de las puertas del circuito. En este punto, y sin pérdida de generalidad, supondremos que todas las puertas de los circuitos tienen aridad, como máximo, n^β . Asumir esto solo incrementa la profundidad de $\mathbf{C}$ por un factor de $\lceil \frac{t}{\beta} \rceil$, y el tamaño no llega ni a duplicarse (en cualquier caso, lo relevante es que el tamaño sigue siendo polinómico).

Ahora, para cada nodo ν de $\mathbf{C}$, tomemos $\mathbf{C}_\nu^\bullet$ con la misma definición que usamos en el apartado 3.3.2. Así, $\mathbf{C}_\nu^\bullet$ computa una función $\mathcal{G}_n \rightarrow \{0, 1\}^{n^{\beta+1}}$ con profundidad $O(1)$ y tamaño $O(n^t)$. Por tanto, por el corolario 3.3.7 (2),

$$\Pr \left[\left| \mathbb{T}_{<s>}^{\mathbf{C}_\nu^\bullet, G}(A) \right| > s \right] = n^{\max(\alpha \binom{s+1}{2} + (\beta-1)(s+1), \alpha \binom{2s}{2} + 2s(\beta-1)) + o(1)} = n^{-t - \Omega(1)}.$$

Por ello, tomando la unión de todos los nodos de $\mathbf{C}$,

$$\Pr \left[\bigvee_{\nu} \left| \mathbb{T}_{<s>}^{\mathbf{C}_\nu^\bullet, G}(A) \right| > s \right] = O(n^t) \cdot n^{-t - \Omega(1)} = n^{-\Omega(1)}.$$

Aparte, por el corolario 3.3.7 (1) tenemos que

$$\Pr \left[\mathbb{T}_{<s>}^{\mathbf{C}, G}(A) \neq \emptyset \right] = n^{\max(\alpha+2, \alpha \binom{s}{2} + s) + o(1)} = n^{-\Omega(1)}.$$

Como ambas probabilidades son de la forma de n elevado a un exponente negativo y constante, ambas tienden a 0 a medida que $n \rightarrow \infty$, por lo que es asintóticamente casi seguro que $\mathbb{T}_{<s>}^{\mathbf{C}, G}(A) = \emptyset$ y $\left| \mathbb{T}_{<s>}^{\mathbf{C}_\nu^\bullet, G}(A) \right| \leq s$ para todos los nodos ν de $\mathbf{C}$. Por tanto, aplicando el lema 3.3.4 obtenemos que $\mathbf{C}(G) = \mathbf{C}(G \cup K_A)$ asintóticamente casi seguro, tal y como queríamos probar. $\square$

Finalmente, llegamos al teorema final de esta sección, y el más importante:

Teorema 3.3.9 (Versión débil del teorema 3.2.3). *Para cualquier constante k , cualquier sucesión de circuitos de profundidad constante que compute $\text{Clique}_{n,k}$ tiene tamaño $\omega\left(n^{\frac{2k}{9}}\right)$. Es decir, no hay circuitos de profundidad constante y tamaño polinómico que compute $\text{Clique}_{n,k}$.*

Demostración. El resultado es evidente para $k \leq 2$, así que nos centramos en el caso $k \geq 3$.

Sea $\mathbf{C} = (\mathbf{C}_n)_{n \in \mathbb{N}}$ una sucesión de circuitos de $\binom{n}{2}$ entradas con profundidad $O(1)$ y tamaño $O(n^t)$ para cierta constante $t = \frac{2k}{9}$, que es mayor que $\frac{2}{3}$. Sea también $\alpha = \frac{4}{9t-3}$. Dado un grafo aleatorio $G \in \text{EG}(n, n^{-\alpha})$ y $A \in \binom{[n]}{k}$, el teorema 3.3.8 nos dice que es asintóticamente casi seguro que $\mathbf{C}(G) = \mathbf{C}(G \cup K_A)$.

Por otro lado, como $\alpha = \frac{2}{k-1.5} > \frac{2}{k-1} = \text{thres}(K_k)$, siendo K_k un clique de tamaño k , el lema 3.2.1 nos dice que (tomando $H = K_k$) es asintóticamente casi seguro que G no contiene un clique de tamaño k .

Pero fijémonos en lo siguiente: con esto último hemos visto que es casi seguro que G no contiene ningún clique de tamaño k . Sin embargo, no es que sea probable, sino seguro, que $G \cup K_A$ contiene un clique de tamaño k . Por tanto, por esta probabilidad tan alta (que esencialmente es casos favorables entre casos posibles), para cada n “suficientemente grande” debe existir un grafo G que no tiene un clique de tamaño k y, sin embargo, verifica $\mathbf{C}(G) = \mathbf{C}(G \cup K_A)$. Así, esta última igualdad nos dice que, tomando n lo suficientemente grande, $\mathbf{C}$ no puede estar definiendo la propiedad de que un grafo contenga un clique de tamaño k .

Por tanto, el problema de $\text{Clique}_{n,k}$ no puede ser computado con circuitos de profundidad constante y tamaño $O\left(n^{\frac{2k}{9}}\right)$, por lo que estos tienen que ser de tamaño $\omega\left(n^{\frac{2k}{9}}\right)$. $\square$

Con esto, a falta de demostrar el lema 3.3.6, ya está demostrada la necesidad de una cantidad de puertas superpolinómica para computar $\text{Clique}_{n,k}$ con circuitos de profundidad constante. En lo que queda de capítulo trabajaremos en mejorar ligeramente la cota en el número de puertas necesaria, para ver que hacen falta aún más.

Los teoremas fuertes se demostrarán en el último apartado del estudio. En el siguiente, principalmente lo que queremos es ampliar nuestra metodología, y dejar demostrado el lema 3.3.6, que habíamos dejado pendiente.

3.4. Restricciones aleatorias y AC^0

Esta sección la dedicaremos a ampliar nuestra metodología de estudio. Advertimos ya de que esta es la sección más técnica de todo el estudio. Por una cuestión de comprensión, omitiremos la mayoría de las demostraciones, pues varias de ellas no son demasiado relevantes para la comprensión de los resultados. Todas ellas se pueden encontrar en [13].

La metodología que se introducirá a continuación se relaciona con el estudio del comportamiento de las funciones en AC^0 al verse sometidas a restricciones aleatorias. Aunque queda bastante para llegar a ello, emplearemos algunas ideas relacionadas con esto en la parte experimental del trabajo.

Ahora, recordemos que no hemos empleado la restricción de la profundidad constante en ninguna demostración. Sin embargo, el lema 3.3.6 se quedó sin demostrar, y sí emplea esta restricción en su enunciado. Como ya mencionamos, el objetivo final de esta sección será demostrar dicho resultado.

Empecemos con ciertas definiciones, que reflejan ciertos conceptos relativamente conocidos dentro del campo de estudio de los circuitos booleanos.

Definición 3.4.1. Sea I un conjunto de entradas, generalmente $[n]$ o $\binom{[n]}{2}$, y sea f una función con dominio $\{0, 1\}^I$. Definimos el conjunto de **inputs** o **entradas sensibles** de f al conjunto

$$\mathbb{S}(f) = \left\{ i \in I : \exists x, y \in \{0, 1\}^I \text{ tal que } f(x) \neq f(y), \text{ con } x_j = y_j, \forall j \in I \setminus \{i\} \right\}.$$

En cierto modo, se puede pensar en $\mathbb{S}(f)$ como “los inputs que importan”. Cualquier índice que no esté en este conjunto esencialmente da igual, pues cambiar el valor de la entrada asociada a dicho índice no modifica el output de la función por sí mismo (haría falta cambiar más inputs).

Este conjunto representa, más o menos, una noción más genérica a la presentada previamente con $\mathbb{T}^{f,G}$. Aquí nos alejamos de los cliques y nos fijamos en los inputs de forma más general. Cabe notar que, para $f = f_{clique}$, claramente todos los inputs son sensibles. Basta tomar como entrada la representación de un grafo de n vértices con un subgrafo completo de k vértices y ninguna arista adicional. Quitar cualquiera de las aristas haría que el grafo dejara de contener un clique de tamaño k , por lo que todos los inputs asociados a dichas aristas son sensibles. Haciendo esto con distintos subgrafos completos, se obtiene que todos los inputs son sensibles.

Vamos ahora con el concepto de restricciones que se menciona en el título del apartado.

Definición 3.4.2. Una **restricción** sobre I es una función $\varrho : I \rightarrow \{0, 1, \star\}$.

¿Qué significa esta definición? Intuitivamente, lo que se busca con ϱ es obtener funciones a partir de fijar valores de algunas entradas sobre otras funciones más genéricas. Aquellos inputs tales que $\varrho(i) = 0$ o $\varrho(i) = 1$ se fijan a dichos valores respectivamente, mientras que aquellos tales que $\varrho(i) = \star$ se dejan sin fijar. Así, dada una función f , obtenemos otra función $f|_{\varrho}$ con dominio $\{0, 1\}^{\varrho^{-1}(\star)}$, que se define como $(f|_{\varrho})(y) = f(x)$, donde $x \in \{0, 1\}^I$ cumple que $x(i) = \varrho(i)$, si $i \in \varrho^{-1}(\{0, 1\})$, y $x(i) = y(i)$, si $i \in \varrho^{-1}(\star)$.

Definición 3.4.3. Sea I un conjunto (el dominio de una función) y sean $p, q \in [0, 1]$. Denotamos por $\mathcal{R}_I^{p,q}$ la restricción aleatoria $\varrho : I \rightarrow \{0, 1, \star\}$ tal que los valores $\varrho(i)$ están independiente e idénticamente distribuidos de forma que

$$\begin{aligned} \Pr[\varrho(i) = \star] &= p \\ \Pr[\varrho(i) = 1] &= (1 - p)q \\ \Pr[\varrho(i) = 0] &= (1 - p)(1 - q) \end{aligned}$$

A continuación enunciaremos dos resultados muy técnicos que serán útiles posteriormente. Omitimos las demostraciones, pues son excesivamente farragosas, y no aportan demasiado de cara a la comprensión del estudio.

Lema 3.4.1. *Supongamos que $f_n : \{0, 1\}^I \rightarrow \{0, 1\}$ es una sucesión de funciones computadas con circuitos de profundidad d y tamaño $O(n^t)$. Entonces, para cualquier $\delta > 0$ pequeño y $l > 0$ grande, existe cierta constante $c = c(d, t, \delta, l)$ tal que*

$$\Pr_{\varrho \in \mathcal{R}_{[n]}^{n^{-\delta}, 1/2}} [|\mathbb{S}(f|_{\varrho})| > c] = O(1/n^l).$$

En este lema, aunque sea complicado de procesar, reside un resultado importante. Lo más relevante de esto es que hemos dicho que existe una constante $c = c(d, t, \delta, l)$, que **no depende de n** . Así, la probabilidad $O(1/n^l)$ se hace ínfimamente pequeña a medida que n crece. ¿Qué implica esto? Que al tomar una función $f \in AC^0$ y aplicarle una restricción aleatoria, es casi seguro que el output solo dependerá de una cantidad muy pequeña de inputs.

Ahora, desviémonos un momento del artículo, pues resulta útil que nos detengamos más en este punto. Este último resultado es, de una vez por todas, donde se manifiesta la necesidad de imponer la restricción de que los circuitos tengan profundidad constante. De nuevo, no entraremos a intentar demostrar este resultado, pues requiere de una cantidad adicional de desarrollo que queda fuera del alcance de nuestro análisis. No obstante, es interesante que nos remontemos al origen de este resultado.

La demostración de este lema es similar a otra de un resultado más clásico, que afirma que el problema de PARIDAD no pertenece a AC^0 . Este resultado es un resultado tremendamente relevante en el campo de los circuitos booleanos de profundidad constante, y su demostración dio lugar a una técnica (empleada en la demostración del lema que hemos enunciado) propuesta por Johan Håstad [9], llamada *Lema de conmutación (Switching lemma)*. Aunque no vayamos a desarrollar la prueba formal de este resultado, sí que es interesante estudiar la idea que este emplea. Supongamos que tenemos un circuito como el siguiente:

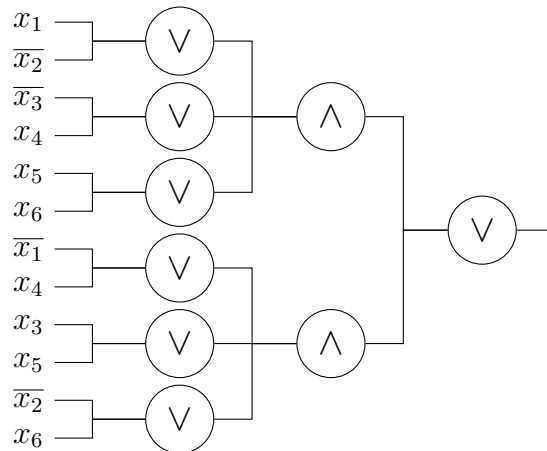


Figura 3.3: Circuito con niveles de AND y OR alternos

Fijándonos en este circuito, vemos que la puerta AND de arriba computa

$$(x_1 \wedge \overline{x_2}) \vee (\overline{x_3} \wedge x_4) \vee (x_5 \wedge x_6).$$

Aplicando propiedades de lógica proposicional, esto se puede expresar como

$$(x_1 \wedge \overline{x_3} \wedge x_5) \vee (x_1 \wedge \overline{x_3} \wedge x_6) \vee \dots \vee (\overline{x_2} \wedge x_4 \wedge x_6).$$

Así, hemos transformado una “AND de ORs” en una “OR de ANDs”. De forma similar, la segunda puerta AND computa

$$(\overline{x_1} \wedge x_4) \vee (x_3 \wedge x_5) \vee (\overline{x_2} \wedge x_6).$$

Haciendo la misma transformación que hicimos antes obtenemos

$$(\overline{x_1} \wedge x_3 \wedge \overline{x_2}) \vee (\overline{x_1} \wedge x_3 \wedge x_6) \vee \dots \vee (x_4 \wedge x_5 \wedge x_6).$$

Por tanto, tras la aplicación de esta transformación a ambas puertas AND nos queda lo siguiente:

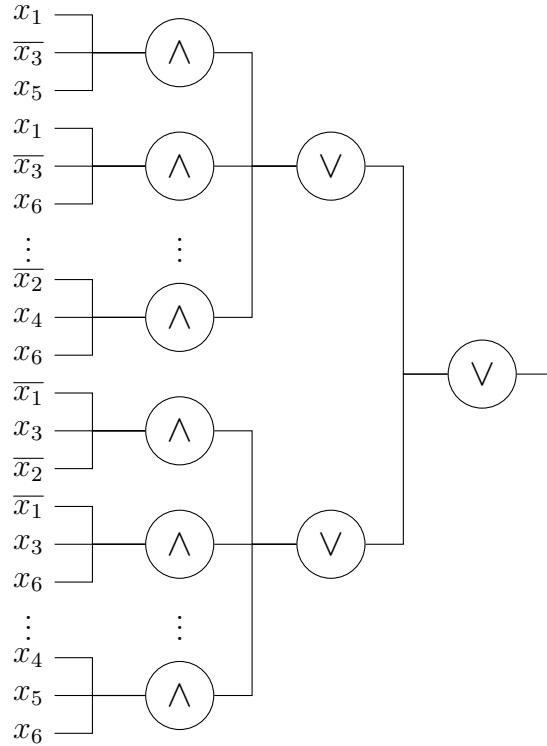


Figura 3.4: Circuito de niveles alternos tras conmutación de niveles de AND y OR

Y ahora, vemos que los dos últimos niveles del circuito están formados por puertas OR. Así, podemos colapsar estos dos niveles en un único nivel, con una única puerta OR (de mayor aridad), quedando de la siguiente forma:

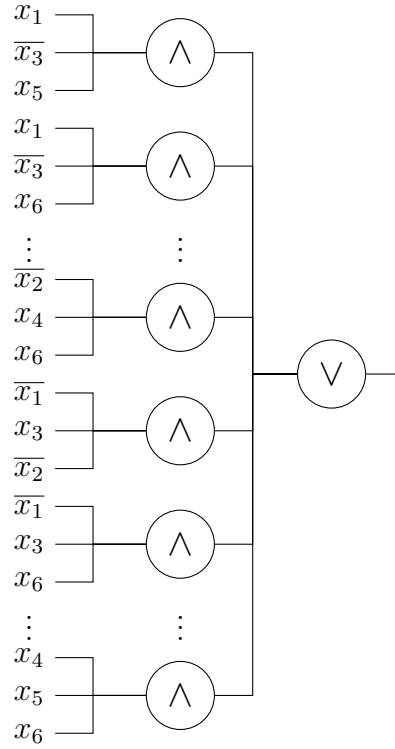


Figura 3.5: Circuito tras colapso de niveles de OR

Con esto, hemos conseguido pasar del circuito original a uno equivalente con un nivel menos de profundidad. Más generalmente, podemos decir que hemos transformado una forma normal conjuntiva (FNC) en una forma normal disyuntiva (FND).

¿Cuál es el problema de esta transformación? Idealmente, lo que querríamos lograr es pasar de una FNC *de cláusulas pequeñas* a una FND *de cláusulas pequeñas*. Sin embargo, esta transformación que hemos hecho lleva, en general, de una FNC de cláusulas pequeñas a una FND de cláusulas grandes. En particular, aplicar esta transformación de forma reiterada a un circuito de tamaño polinómico cualquiera hace que su tamaño deje de ser polinómico, por lo que no nos sirve en general.

Es aquí donde entran los circuitos de profundidad constante. En líneas generales, al aplicar una restricción aleatoria a una función en AC^0 , se cumple con alta probabilidad que la fórmula en cuestión se simplifica a una más simple de profundidad menor (por ejemplo, una FNC de cláusulas cortas a una FND de cláusulas también cortas). Aplicando esto suficientes veces, uno acaba llegando a una función constante o “muy simple” (que depende de pocos inputs). Y este fenómeno es el que no puede ocurrir para ciertas funciones, como la de PARIDAD.

Da igual la restricción que uno tome, al aplicarla sobre la función de PARIDAD, la función resultante sobre los inputs libres es, o bien la propia función de PARIDAD, o su negación. Entonces, no es posible que a base de restricciones aleatorias se llegue a una función “tan simple”, pues la función de PARIDAD siempre depende de todos sus inputs. Esto lleva a que PARIDAD no está en AC^0 . En ese sentido, $Clique_{n,k}$ tiene el mismo comportamiento.

Con esto, volvamos a los resultados del artículo. El siguiente lema es una generalización del lema anterior.

Lema 3.4.2. *En las mismas condiciones del lema anterior, para cualquier $\alpha, \delta > 0$ pequeños y $l > 0$ grande, existe una constante $c = c(d, t, \alpha, \delta, l)$ tal que*

$$\Pr_{\varrho \in \mathcal{R}_{[n]}^{n-(\alpha+\delta), n-\alpha}} [|\mathbb{S}(f \upharpoonright_{\varrho})| > c] = O(1/n^l).$$

Con esto, lo que hemos ganado esencialmente es mayor flexibilidad a la hora de elegir las probabilidades con las que se restringen las variables a 0, 1 o $\star$.

Ahora, vamos a llevar el concepto de las restricciones aleatorias un paso más cerca del contexto de los grafos.

Definición 3.4.4. *Dada una función f con dominio en $\mathcal{G}_n$ (una función de grafos) y una restricción $\varrho : \binom{[n]}{2} \rightarrow \{0, 1, \star\}$, nos referimos a los elementos de $\mathbb{S}(f \upharpoonright_{\varrho})$ como **aristas sensibles**, y definimos el conjunto de **vértices sensibles** como*

$$\mathbb{V}(f \upharpoonright_{\varrho}) = \{i \in [n] : \exists j \in [n] \text{ tal que } \{i, j\} \in \mathbb{S}(f \upharpoonright_{\varrho})\}.$$

Este conjunto es el de los vértices que son extremo de alguna arista sensible.

Por su propia definición, se desprende inmediatamente que $|\mathbb{V}(f \upharpoonright_{\varrho})| \leq 2|\mathbb{S}(f \upharpoonright_{\varrho})|$.

Definición 3.4.5. *Dados un grafo H , $q \in [0, 1]$ y $n \geq |V_H|$, denotamos por $\mathcal{GR}_n^q(H)$ la restricción aleatoria $\varrho : \binom{[n]}{2} \rightarrow \{0, 1, \star\}$ tal que, dado $e \in E_H$, $\varrho(e) = \star$, mientras que dado $e \notin E_H$, $\Pr[\varrho(e) = 0] = q$ y $\Pr[\varrho(e) = 1] = 1 - q$.*

Sencillamente, con esta restricción aleatoria lo que queremos es que queden variables las aristas del grafo, y que todas aquellas aristas restantes queden fijadas a 0 o 1.

Con esto, la última proposición técnica de este apartado, que también dejaremos sin demostrar, es la siguiente.

Proposición 3.4.3. *Sea $f : \mathcal{G}_n \rightarrow \{0, 1\}^{n^\beta}$ una función en AC^0 , con cierto $\beta \geq 0$. Sea H un grafo, y sea $0 < \alpha < \text{thres}(H)$. Entonces*

$$\Pr_{\varrho \in \mathcal{GR}_n^{n-\alpha}(H)} [|\mathbb{V}(f \upharpoonright_{\varrho})| = |V_H|] = n^{\alpha|E_H| + (\beta-1)|V_H| + o(1)}.$$

Esta es la última herramienta que nos hace falta para, finalmente, demostrar el lema 3.3.6.

Demostración del lema 3.3.6. Suponemos que $\alpha < \frac{2}{k-1}$, pues en caso contrario el resultado es trivial.

Sea $f : \{0, 1\}^{\binom{[n]}{2}} \rightarrow \{0, 1\}^{n^\beta}$ perteneciente a AC^0 . Sea $G \in \text{ER}(n, n^{-\alpha})$ y sea $A \in \binom{[n]}{k}$.

Tomamos $\varrho_A^G : \binom{[n]}{2} \rightarrow \{0, 1, \star\}$ una restricción de grafos aleatoria que deja libres $(\star)$ las aristas de $\binom{A}{2}$, asocia a 1 las aristas de $E_G \setminus \binom{A}{2}$, y a 0 el resto (básicamente

estamos “des-restringiendo” G , dejando libres las aristas con ambos extremos en A). Dado que las aristas de G están o no con probabilidad $n^{-\alpha}$, y estamos dejando libres todas las aristas correspondientes a un conjunto de k vértices (como un clique K_k), esta restricción tiene distribución $\mathcal{GR}_n^{n^{-\alpha}}(K_k)$.

Veamos ahora que $\mathbb{T}^{f,G}(A) \subseteq \mathbb{V}(f \upharpoonright_{\varrho_A^G})$. Dado $a \in \mathbb{T}^{f,G}(A)$, existe $B \subseteq A$ tal que $f(G \cup K_B) \neq f(G \cup K_{B \setminus \{a\}})$. Queremos ver que alguna de las aristas de B que incide en a debe ser sensible. Supongamos lo contrario.

En tal caso, sean $b_1, \dots, b_{l_a}$ dichas aristas. Por lo que hemos supuesto,

$$f(G \cup K_B) = f(G \cup \{b_1, \dots, b_{l_a}\}) = f(G \cup \{b_1, \dots, b_{l_a-1}\}).$$

Repitiendo este proceso l_a veces, obtenemos

$$f(G \cup K_B) = f(G \cup K_{B \setminus \{a\}}),$$

pues hemos eliminado todas las aristas de B que inciden en a , pero esto es una contradicción. Por tanto, $a \in \mathbb{V}(f \upharpoonright_{\varrho_A^G})$.

Ahora que tenemos esto, lo que podemos deducir es que, si se cumple que $\mathbb{T}^{f,G}(A) = A$, como $\mathbb{T}^{f,G}(A) \subseteq \mathbb{V}(f \upharpoonright_{\varrho_A^G}) \subseteq A$ y $|A| = k$, entonces $|\mathbb{V}(f \upharpoonright_{\varrho_A^G})| = k$.

Por tanto, empleando que $\alpha < \frac{2}{k-1} = \text{thres}(K_k)$, como vimos en el ejemplo 3.2.1, junto con la proposición 3.4.3, deducimos que

$$\begin{aligned} \Pr_{G,A} [\mathbb{T}^{f,G}(A) = A] &\leq \Pr_{G,A} [|\mathbb{V}(f \upharpoonright_{\varrho_A^G})| = k] \\ &= \Pr_{\varrho \in \mathcal{GR}_n^{n^{-\alpha}}(K_k)} [|\mathbb{V}(f \upharpoonright_{\varrho})| = k] \\ &= n^{\alpha|E_H| + (\beta-1)|V_H| + o(1)} \\ &= n^{\alpha \binom{k}{2} + (\beta-1)k + o(1)}. \end{aligned}$$

□

Con esto demostrado y las nuevas herramientas desarrolladas, estamos en posición para pasar al último apartado, en la que demostramos las versiones fuertes de los teoremas principales.

3.5. Demostración de las versiones fuertes de los teoremas principales

En este punto, el autor hace mención al resultado que, a diferencia de antes, se usará ahora para demostrar los resultados principales. Si recordamos el lema 3.3.3, en la demostración de las versiones débiles de los teoremas principales (3.3.8 y 3.3.9) se aprovechó el resultado (3) de dicho lema. En esta ocasión vamos a usar el resultado (2), que es más fuerte. Afortunadamente, la complejidad adicional no será excesiva, pues lo que hemos visto hasta ahora simplifica mucho la tarea. Introducimos a continuación unas pocas definiciones y resultados, y las demostraciones principales serán similares a las de las versiones débiles.

Definición 3.5.1. Sean $a, b, c \in \mathbb{N}$ tales que $a \leq \min(b, c)$ y $b + c - a \geq 1$. Denotamos por $H_{a,b,c}$ el grafo resultante de tomar un clique de tamaño b y otro de tamaño c y juntarlos, de forma que los vértices compartidos formen un clique de tamaño a .

Ejemplo 3.5.1. Los siguientes grafos son todos del tipo $H_{a,b,c}$:

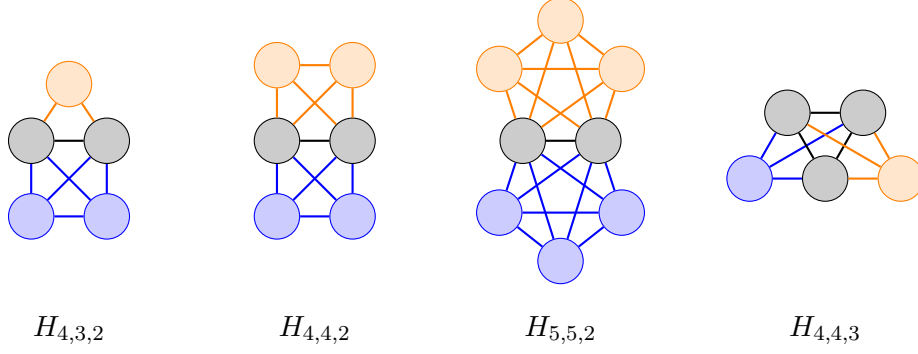


Figura 3.6: Ejemplos de grafos de tipo $H_{a,b,c}$

Una cuestión a observar acerca de este tipo de grafos es que cualquier subgrafo de $H_{a,b,c}$ es también de la forma $H_{a',b',c'}$ para ciertos valores $a' \leq a$, $b' \leq b$ y $c' \leq c$.

Lema 3.5.1. Para cualquier a, b, c , $\text{thres}(H_{a,b,c}) > \frac{1+\sqrt{2}}{\sqrt{2} \max(b,c)}$.

Demostración. Sea $s = \max(b, c)$. El caso de $s = 1$ es trivial, pues en tal caso $H_{a,b,c}$ no tiene aristas, por lo que $\text{thres}(H_{a,b,c}) = \infty$. Así, supongamos $s \geq 2$.

Por definición,

$$\text{thres}(H_{a,b,c}) = \min_{H_{a',b',c'} \subseteq H_{a,b,c}} \frac{|V_{H_{a',b',c'}}|}{|E_{H_{a',b',c'}}|}.$$

Los subgrafos $H_{a',b',c'}$ cumplen que $b' \leq b$ y $c' \leq c$. Si relajamos esta condición, y únicamente nos centramos en que se cumpla que $b', c' \leq s$, tenemos que

$$\min_{H_{a',b',c'} \subseteq H_{a,b,c}} \frac{|V_{H_{a',b',c'}}|}{|E_{H_{a',b',c'}}|} \geq \min_{\substack{a',b',c' \in \mathbb{N} \\ a' \leq b', c' \leq s \\ b' + c' - a' \geq 1}} \frac{b' + c' - a'}{\binom{b'}{2} + \binom{c'}{2} - \binom{a'}{2}}.$$

Ahora, como el denominador crece exponencialmente con respecto a los valores b' y c' , este último valor es mínimo cuando b' y c' toman el valor máximo posible, que es s . En este caso, como se ha de cumplir $b' + c' - a' \geq 1$, se tiene que $a' \leq s - 1$.

Así,

$$\begin{aligned} & \min_{\substack{a',b',c' \in \mathbb{N} \\ a' \leq b', c' \leq s \\ b' + c' - a' \geq 1}} \frac{b' + c' - a'}{\binom{b'}{2} + \binom{c'}{2} - \binom{a'}{2}} \geq \min_{a' \in \{0, \dots, s-1\}} \frac{2s - a'}{2 \binom{s}{2} - \binom{a'}{2}} \\ &= \min_{a' \in \{0, \dots, s-1\}} \frac{4s - 2a'}{2s^2 - a^2 + (a - 2s)} \geq \min_{x \in [0, s]} \frac{4s - 2x}{2s^2 - x^2 + (x - 2s)} \\ &> \min_{x \in [0, s]} \frac{4s - 2x}{2s^2 - x^2} = \min_{x \in [0, s]} \frac{1}{s} \frac{2(2 - \frac{x}{s})}{2 - \frac{x^2}{s^2}} \end{aligned}$$

Finalmente, tomando $\lambda = \frac{x}{s} \in [0, 1]$, se puede comprobar que $f(\lambda) = \frac{1}{s} \frac{2(2-\lambda)}{2-\lambda^2}$ tiene por mínimo $\frac{1}{s} \left(1 + \frac{1}{\sqrt{2}}\right)$, como queríamos demostrar. $\square$

Definición 3.5.2. Sea $s \in \mathbb{N}$. Denotamos por W_s el conjunto de tuplas de la forma (a, b, c) tales que $a \leq \min(b, c)$, $\max(b, c) \leq s$ y $b + c - a \geq s + 1$.

Lema 3.5.2. Sea $f = (f_n)_{n \in \mathbb{N}}$ una secuencia de funciones $f_n : \{0, 1\}^{\binom{n}{2}} \rightarrow \{0, 1\}^{n^\beta}$ computables con circuitos de profundidad constante y tamaño polinómico (AC^0), con cierto $\beta \geq 0$. Sea $s \in \mathbb{N}$ y $(a, b, c) \in W_s$, y tomemos $0 < \alpha \leq \frac{1}{s}$. Entonces, dado $G \in \text{ER}(n, n^{-\alpha})$ y dos conjuntos aleatorios $B \in \binom{[n]}{b}$ y $C \in \binom{[n]}{c}$ tales que $|B \cap C| = a$ se cumple que

$$\Pr \left[\mathbb{T}^{f,G}(B) = B \text{ y } \mathbb{T}^{f,G}(C) = C \right] = n^{\alpha \left(\binom{b}{2} + \binom{c}{2} - \binom{a}{2} \right) + (\beta-1)(b+c-a) + o(1)}.$$

Demostración. Tomemos la restricción de grafos $\varrho_{B,C}^G : \binom{[n]}{2} \rightarrow \{0, 1, \star\}$ tal que, dado $e \in \binom{[n]}{2}$, su imagen es $\star$ si $e \in \binom{B}{2} \cup \binom{C}{2}$, es 1 si $e \in E_G \setminus \left(\binom{B}{2} \cup \binom{C}{2} \right)$, y es 0 en caso contrario. Es decir, estamos dejando libres las aristas de $\binom{B}{2} \cup \binom{C}{2}$ y dejando el resto de aristas de G como están. De forma similar a lo que vimos en la demostración de 3.4.3, esta restricción tiene distribución $\mathcal{GR}_n^{n^{-\alpha}}(H_{a,b,c})$.

De nuevo, por un argumento idéntico al que vimos en la demostración de 3.4.3, todos los elementos de $\mathbb{T}^{f,G}(B) \cup \mathbb{T}^{f,G}(C)$ deben ser vértices sensibles de $f \upharpoonright_{\varrho_{B,C}^G}$, con lo que se cumple

$$\mathbb{T}^{f,G}(B) \cup \mathbb{T}^{f,G}(C) \subseteq \mathbb{V}(f \upharpoonright_{\varrho_{B,C}^G}) \subseteq B \cup C.$$

Ahora, fijémonos en que, por el lema 3.5.1, $\alpha \leq \frac{1}{s} < \text{thres}(H_{a,b,c})$, por lo que podemos emplear la proposición 3.4.3. Haciendo uso de este resultado, obtenemos que

$$\begin{aligned} \Pr_{G,B,C} \left[\mathbb{T}^{f,G}(B) = B \text{ y } \mathbb{T}^{f,G}(C) = C \right] &\leq \Pr_{G,B,C} \left[|\mathbb{V}(f \upharpoonright_{\varrho_{B,C}^G})| = |B \cup C| \right] \\ &\leq \Pr_{\varrho \in \mathcal{GR}_n^{n^{-\alpha}}(H_{a,b,c})} \left[|\mathbb{V}(f \upharpoonright_{\varrho})| = |\mathbb{V}_{H_{a,b,c}}| \right] \\ &= n^{\alpha |E_{H_{a,b,c}}| + (\beta-1) |V_{H_{a,b,c}}| + o(1)} \\ &= n^{\alpha \left(\binom{b}{2} + \binom{c}{2} - \binom{a}{2} \right) + (\beta-1)(b+c-a) + o(1)}. \end{aligned}$$

$\square$

Corolario 3.5.3. Tomemos f, β, s, α como en el lema 3.5.2, y sea k una constante independiente de n . Así, dado un grafo $G \in \text{ER}(n, n^{-\alpha})$ y un conjunto aleatorio $A \in \binom{[n]}{k}$, se cumple que

$$\Pr \left[\left| \mathbb{T}_{<s>}^{f,G}(A) \right| > s \right] = n^{\gamma + o(1)},$$

donde $\gamma = \max_{(a,b,c) \in W_s} \alpha \left(\binom{b}{2} + \binom{c}{2} - \binom{a}{2} \right) + (\beta-1)(b+c-a)$.

Demostración. Esta demostración emplea argumentos similares a los del corolario 3.3.7, aunque adelantamos que son significativamente más complicados.

Para empezar, por el lema 3.3.3 (2),

$$\Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ A \in \binom{[n]}{k}}} \left[\left| \mathbb{T}_{<s>}^{f,G}(A) \right| > s \right] = \Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ A \in \binom{[n]}{k}}} \left[\bigvee_{\substack{B, C \in \binom{A}{\leq s} \\ |B \cup C| > s}} \mathbb{T}^{f,G}(B) = B \text{ y } \mathbb{T}^{f,G}(C) = C \right].$$

Ahora lo que tenemos es una unión de sucesos. Similar a lo que hicimos en la demostración de 3.3.7, podemos acotar esta probabilidad superiormente, y recuperar la igualdad a través del término $o(1)$. Entonces, para deshacernos de esta unión de sucesos, tenemos que sumar todas las probabilidades por separado. Como en la unión se incluyen todas las posibles elecciones de B y C , pues siempre serán subconjuntos de cierto A , basta con que elijamos B y C .

Sean b y c los tamaños de B y C respectivamente. Como $|B \cup C| > s$ y $\max(b, c) \leq s$, basta que tomemos $(a, b, c) \in W_s$, de forma que a sea el tamaño de la intersección. Elegida esta tupla, tenemos que elegir a vértices en los que intersecan B y C , para lo que hay $\binom{k}{b+c-a}$ elecciones posibles. Una vez elegidos los vértices, hay que determinar cuáles pertenecen a B , cuáles a C y cuáles a ambos. Para esto hay $\frac{(b+c-a)!}{a!(b-a)!(c-a)!}$ posibilidades.

Con todo esto, podemos acotar la probabilidad de la unión de sucesos como

$$\begin{aligned} & \Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ A \in \binom{[n]}{k}}} \left[\bigvee_{\substack{B, C \in \binom{A}{\leq s} \\ |B \cup C| > s}} \mathbb{T}^{f,G}(B) = B \text{ y } \mathbb{T}^{f,G}(C) = C \right] \\ & \leq \sum_{(a,b,c) \in W_s} \binom{k}{b+c-a} \frac{(b+c-a)!}{a!(b-a)!(c-a)!} \Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ B \in \binom{[n]}{b}, C \in \binom{[n]}{c} \\ |B \cap C| = a}} \left[\mathbb{T}^{f,G}(B) = B \text{ y } \mathbb{T}^{f,G}(C) = C \right]. \end{aligned}$$

Ahora, empleando el lema 3.5.2,

$$\begin{aligned} & \sum_{(a,b,c) \in W_s} \binom{k}{b+c-a} \frac{(b+c-a)!}{a!(b-a)!(c-a)!} \Pr_{\substack{G \in \text{ER}(n, n^{-\alpha}) \\ B \in \binom{[n]}{b}, C \in \binom{[n]}{c} \\ |B \cap C| = a}} \left[\mathbb{T}^{f,G}(B) = B \text{ y } \mathbb{T}^{f,G}(C) = C \right] \\ & = \sum_{(a,b,c) \in W_s} \binom{k}{b+c-a} \frac{(b+c-a)!}{a!(b-a)!(c-a)!} n^{\alpha \left(\binom{b}{2} + \binom{c}{2} - \binom{a}{2} \right) + (\beta-1)(b+c-a) + o(1)}. \end{aligned}$$

Finalmente, quedándonos con los exponentes máximos, al igual que hicimos al demostrar el corolario 3.3.7, y dándonos cuenta de que la cantidad $\binom{k}{b+c-a} \frac{(b+c-a)!}{a!(b-a)!(c-a)!}$ es constante, concluimos que

$$\sum_{(a,b,c) \in W_s} \binom{k}{b+c-a} \frac{(b+c-a)!}{a!(b-a)!(c-a)!} n^{\alpha \left(\binom{b}{2} + \binom{c}{2} - \binom{a}{2} \right) + (\beta-1)(b+c-a) + o(1)} = n^{\gamma + o(1)}.$$

□

Ahora sí, tenemos los recursos suficientes para demostrar las versiones fuertes de los teoremas principales.

Demostración del teorema 3.2.2. Sea $s = \lceil 2t - 1 \rceil$, y fijémonos que $\alpha = \frac{1}{2t-1} \leq \frac{1}{s}$. Se puede comprobar que se cumplen las siguientes desigualdades:

$$\max(\alpha - 2, \alpha \binom{s}{2} - s) < 0,$$

$$\max_{(a,b,c) \in W_s} \alpha \left(\binom{b}{2} + \binom{c}{2} - \binom{a}{2} \right) - (b + c - a) < -t,$$

y, de forma similar a lo que vimos en su momento, la tupla (a, b, c) que maximiza el último valor es $(s - 1, s, s)$.

A partir de este punto, el razonamiento es totalmente análogo al que se hizo para demostrar la versión débil de este teorema. Lo primero que tenemos que hacer entonces es tomar cierta constante $\beta > 0$ tal que $\gamma < -t$, donde

$$\gamma = \max_{(a,b,c) \in W_s} \alpha \left(\binom{b}{2} + \binom{c}{2} - \binom{a}{2} \right) + (\beta - 1)(b + c - a).$$

A continuación, tomamos una sucesión de circuitos $\mathbf{C} = (\mathbf{C}_n)_{n \in \mathbb{N}}$ con aridad n^β , profundidad $O(1)$ y tamaño $O(n^t)$ que compute las funciones f_n . Por el corolario 3.5.3 tenemos que, para cualquier nodo $\nu \in \mathbf{C}$,

$$\Pr \left[\left| \mathbb{T}_{<s>}^{\mathbf{C}_\nu, G}(A) \right| > s \right] = n^{\gamma+o(1)} = n^{-t-\Omega(1)}.$$

Entonces, tomando la unión de todos los nodos,

$$\Pr \left[\bigvee_{\nu} \left| \mathbb{T}_{<s>}^{\mathbf{C}_\nu, G}(A) \right| > s \right] = O(n^t) \cdot n^{\gamma+o(1)} = n^{-\Omega(1)}.$$

Aparte, por el corolario 3.3.7 (1) tenemos que

$$\Pr \left[\mathbb{T}_{<s>}^{\mathbf{C}, G}(A) \neq \emptyset \right] = n^{\max(\alpha+2, \alpha \binom{s}{2} + s) + o(1)} = n^{-\Omega(1)}.$$

Por tanto, es asintóticamente casi seguro que $\mathbb{T}_{<s>}^{\mathbf{C}, G}(A) = \emptyset$ y que $\left| \mathbb{T}_{<s>}^{\mathbf{C}_\nu, G}(A) \right| \leq s$, para todos los nodos $\nu \in \mathbf{C}$. Por tanto, empleando el lema 3.3.4, obtenemos que es asintóticamente casi seguro que $\mathbf{C}(G) = \mathbf{C}(G \cup K_A)$. $\square$

Demostración del teorema 3.2.3. El resultado es trivial para $k \leq 3$, por lo que asumimos $k \geq 4$.

Sea $\mathbf{C} = (\mathbf{C}_n)_{n \in \mathbb{N}}$ una secuencia de circuitos de $\binom{n}{2}$ entradas con profundidad $O(1)$ y tamaño $O(n^t)$ para $t = \frac{k}{4} > \frac{1}{2}$. Sea $\alpha = \frac{1}{2t-1}$, que nótese que cumple $\alpha = \frac{2}{k-2} > \frac{2}{k-1} = \text{thres}(K_k)$.

Ahora, tomando cierto grafo aleatorio $G \in \text{ER}(n, n^{-\alpha})$ y cierto conjunto $A \in \binom{[n]}{k}$, el teorema 3.2.2 nos dice que es asintóticamente casi seguro que $\mathbf{C}(G) = \mathbf{C}(G \cup K_A)$. Por otro lado, el lema 3.2.1 (1) nos dice que es asintóticamente casi seguro

que G no contiene ningún clique de tamaño k . Como $G \cup K_A$ sí contiene un clique de tamaño k , esto nos dice que C no puede estar definiendo la propiedad de que un grafo contenga un clique de tamaño k . Por tanto, no existe sucesión de circuitos de profundidad constante que determine si un grafo tiene un clique de tamaño k con profundidad constante y tamaño $O(n^{\frac{k}{4}})$. $\square$

Capítulo 4

Estudio experimental

En este capítulo nos dedicaremos a hacer un estudio experimental del problema de $Clique_{n,k}$. Como ya adelantamos anteriormente, vamos a continuar la línea de investigación que propuso Jaime Jacobo Romo González en su trabajo de fin de grado [12].

Recomendamos enérgicamente leer dicho trabajo, pero en cualquier caso aquí explicaremos, de forma más resumida, todos los resultados que son relevantes para nuestro estudio.

4.1. Introducción a las métricas

En [12] se definieron unos tipos de funciones, llamadas métricas, que reflejaban los desarrollos realizados en las notas [2] acerca del estudio de Razborov [11]. Las métricas son una forma de asignar a cualquier función booleana una especie de puntuación con base en ciertas propiedades. Lo que se pretende reflejar con esta puntuación es “cómo de buena es una función”.

En el contexto de intentar computar $Clique_{n,k}$ con un circuito booleano, imaginemos que tenemos un circuito C que lo computa. Si tomamos cualquier nodo interno del circuito, podemos pensar en ese nodo como sumidero del subcircuito que queda antes de él, al cual llamaremos C' y que computará cierta función booleana f . Se puede pensar que a más cercano sea ese nodo al sumidero de C , más se parecerá f a la función de $Clique_{n,k}$.

Con esa idea, parece que cada puerta lógica que se añade a un circuito puede aportar de cara a que la función computada sea cada vez más cercana a la función de $Clique_{n,k}$, y eso es lo que se quiere reflejar con las métricas: según añadimos puertas a un circuito, podemos puntuar cuánto mejora la función resultante.

La cuestión al definir una métrica concreta será especificar a qué nos referimos con “mejorar” o “parecerse” a $Clique_{n,k}$, pero hay una serie de propiedades que, de forma general, resultan útiles a la hora de definir una métrica:

1. Las funciones pequeñas (por ejemplo, las funciones que son sencillamente una entrada) deben tener puntuaciones bajas.

2. La métrica tiene que estar acotada. Es decir, debe existir una puntuación máxima.
3. La puntuación máxima de la métrica debe ser muy alta. Más concretamente, queremos que sea un valor exponencial con respecto a n y k .
4. La función de $Clique_{n,k}$ debe alcanzar esa puntuación máxima.

Estas propiedades nos permiten reflejar bastante bien la idea que queríamos asentar. Las funciones más pequeñas (las entradas) están muy lejos de computar $Clique_{n,k}$, así que tienen puntuaciones muy bajas. Además, a medida que el circuito crece y va avanzando más hacia computar $Clique_{n,k}$, se espera que la puntuación aumente, reflejando un mayor parecido con la función final. Este crecimiento no tiene por qué ser monótono, pues puede ser que en ocasiones el mejor crecimiento del circuito requiera empeorar la métrica temporalmente para que mejore más tarde. La cuestión es que, si una función no alcanza la puntuación máxima, entonces no es la función de $Clique_{n,k}$. Dicho de otra forma, alcanzar la puntuación máxima es una condición necesaria, aunque no tiene por qué ser suficiente.

A partir de aquí, se buscaría poder acotar el crecimiento máximo en la puntuación que aporta cada puerta lógica en un circuito. Si este crecimiento fuera pequeño entonces, dado que hemos pedido que la puntuación máxima sea exponencial, podría llegar a verse que hace falta también una cantidad exponencial de puertas para obtener dicha puntuación máxima, lo cual conllevaría a que cualquier circuito que compute $Clique_{n,k}$ debe tener tamaño exponencial, con lo que se demostraría que $\mathcal{P} \neq \mathcal{NP}$.

4.1.1. Métrica sintáctica

En este apartado vamos a definir la métrica sintáctica, cuya comprensión resulta fundamental para entender todo lo que vendrá posteriormente. Esta métrica fue originalmente definida en el trabajo de Jaime Jacobo Romo González [12].

Recordemos que un grafo G de n vértices se puede representar con $\binom{n}{2}$ variables, cada una representando si una arista está o no presente en G .

Ahora, lo primero que nos interesa es entender la función que computa $Clique_{n,k}$. Dados dos vértices u y v en un grafo (supongamos $u < v$), denotemos por $x_{u,v}$ la variable de entrada que determina si la arista que une estos dos vértices está presente en el grafo.

En un clique de tamaño k hay tantas aristas como parejas de vértices, que es $\binom{k}{2} = \frac{k(k-1)}{2}$. Así, la función más sencilla para determinar si un grafo contiene un clique concreto es hacer la conjunción de todas las variables correspondientes a las aristas de ese clique.

Por ejemplo, la función que determina si en un grafo está presente el clique de tamaño 3 formado por los vértices del 2 al 4 sería $x_{2,3} \wedge x_{2,4} \wedge x_{3,4}$, que escribiremos como $x_{2,3}x_{2,4}x_{3,4}$ por simplificar la notación.

De forma general, dado un conjunto de vértices X de tamaño k , la función que

determina si los vértices de X forman un clique (de tamaño k) en un grafo es

$$\bigwedge_{\substack{i,j \in X \\ i < j}} x_{i,j}.$$

Por tanto, para ciertos valores de n y $k \leq n$, la función que computa $Clique_{n,k}$ es

$$f_{clique} = \bigvee_{\substack{X \subseteq \{1, \dots, n\} \\ |X|=k}} \bigwedge_{\substack{i,j \in X \\ i < j}} x_{i,j}.$$

Sencillamente consiste en hacer la disyunción de cláusulas tales que cada una reconoce uno de los posibles cliques. Como en un grafo de n vértices puede haber hasta $\binom{n}{k}$ cliques, pues es el número de conjuntos de k vértices que se pueden formar, ese es también el número de cláusulas que tiene f_{clique} .

Ejemplo 4.1.1. Para $n = 4$ y $k = 3$, la fórmula que determina si en un grafo de tamaño 4 hay un clique de tamaño 3 es

$$f_{clique} = x_{1,2}x_{1,3}x_{2,3} \vee x_{1,2}x_{1,4}x_{2,4} \vee x_{1,3}x_{1,4}x_{3,4} \vee x_{2,3}x_{2,4}x_{3,4}$$

Cada cláusula tiene $\binom{3}{2} = 3$ literales y en total hay $\binom{4}{3} = 4$ cláusulas.

Observamos que esta expresión de f_{clique} está en forma normal disyuntiva (FND). No obstante, esto no es una característica especial de esta función. Si bien es cierto que en este caso resulta muy natural expresar f_{clique} de esta forma, todas las funciones booleanas se pueden expresar en FND, hecho que emplearemos ahora para definir la métrica sintáctica.

Supongamos que tenemos cierta función booleana f expresada en FND. Una forma de valorar cuánto se parece f a f_{clique} es comparar ambas funciones cláusula a cláusula. Si las cláusulas de f se parecen a f_{clique} , entonces f se parecerá a f_{clique} , y viceversa. Esto es lo que motiva la siguiente definición.

Definición 4.1.1. [12] Sean f y f_{clique} una función booleana en forma normal disyuntiva y la función que computa $Clique_{n,k}$, respectivamente. Sea m el número de cláusulas en f , con lo que denotamos por f_i a la i -ésima cláusula de f y por $f_{clique,j}$ la j -ésima cláusula de f_{clique} . Sea $r = \max(m, \binom{n}{k})$, y supongamos que ambas funciones tienen r cláusulas, aunque para ello haya que extenderlas con cláusulas vacías. Denotemos por $|f_i \cap f_{clique,j}|$ el número de literales en común que tienen f_i y $f_{clique,j}$. Sea S_r el grupo simétrico de r elementos (las biyecciones/permutaciones de r elementos). Con esto, definimos la métrica sintáctica μ_x como

$$\mu_x(f) = \max_{\sigma \in S_r} \left\{ \sum_{i=1}^r |f_i \cap f_{clique, \sigma(i)}| \right\}.$$

Aunque la definición no es exactamente trivial, lo que se busca es encontrar la mejor asignación entre cláusulas de f y f_{clique} , de forma que la cantidad de literales compartidos entre cláusulas sea máximo. Veamos un ejemplo para entender mejor como funciona la métrica.

Ejemplo 4.1.2. Recordemos la fórmula del ejemplo 4.1.1. Probemos a calcular la métrica sintáctica de varias funciones.

Sea $f = x_{1,2}x_{2,3} \vee x_{1,4}x_{3,4}x_{1,3}$. El siguiente diagrama refleja cuáles son las mejores asignaciones para cada cláusula:

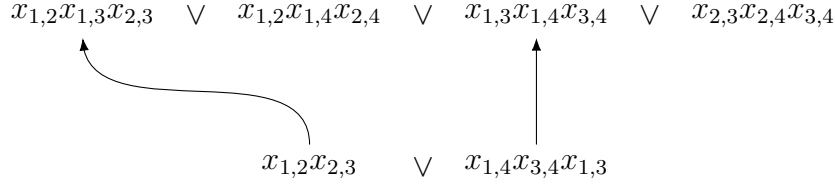


Figura 4.1: Asignaciones óptimas compatibles y de suma máxima

Vemos que esta asignación consigue $|f_1 \cap f_{\text{clique},1}| = 2$ y $|f_2 \cap f_{\text{clique},3}| = 3$. Como las cláusulas asignadas de f_{clique} son distintas, esta asignación es viable (se consigue con cierta permutación de $\mathcal{S}_4$), por lo que $\mu_x(f) = 2 + 3 = 5$. Una cosa a notar en este ejemplo es que se cumplen dos propiedades positivas de cara a obtener una puntuación alta: la asignación óptima de cada cláusula logra que coincidan todos los literales, y estas asignaciones óptimas de cada cláusula son compatibles entre sí. Veamos un par de ejemplos que ilustran que esto no tiene por qué ocurrir en general.

Sea $g = x_{1,2}x_{1,3}x_{3,4} \vee x_{2,3}x_{2,4}$. De nuevo, dibujamos las asignaciones ideales para cada cláusula:

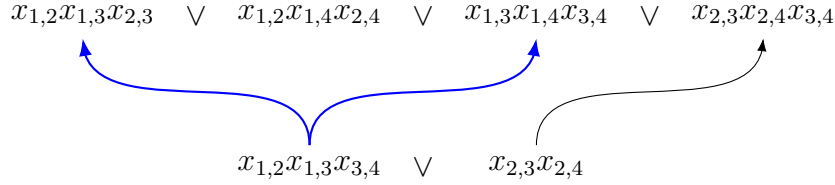


Figura 4.2: Asignaciones óptimas múltiples de sumas no máximas

Las mejores asignaciones de la primera cláusula son las marcadas en azul. Vemos que cualquiera nos vale, pues son compatibles con la mejor asignación de la segunda cláusula. Sin embargo, en ambos casos tenemos $|g_1 \cap f_{\text{clique},1}| = |g_1 \cap f_{\text{clique},3}| = 2$, que es menos que el número de literales en g_1 . En este caso, $\mu_x(g) = 2 + 2 = 4$.

Por último, sea $h = x_{1,2}x_{1,3}x_{2,3} \vee x_{1,2}x_{1,4} \vee x_{1,2}x_{2,4}$. Dibujamos las mejores asignaciones de cada cláusula:

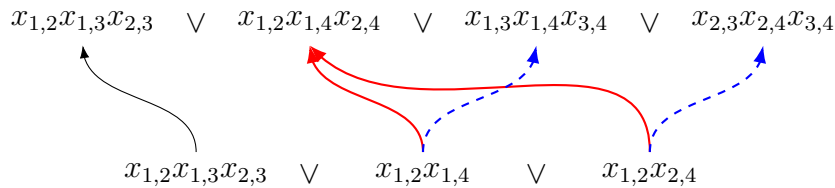


Figura 4.3: Asignaciones óptimas incompatibles

En este caso, las asignaciones óptimas de h_2 y h_3 , marcadas en rojo, no son compatibles, pues ambas apuntan a $f_{\text{clique},2}$. Por tanto, por haber colisión entre las mejores asignaciones posibles, una de estas dos cláusulas de h debe ser asignada a otra cláusula de f_{clique} . Se pueden hacer, por ejemplo, la asignaciones $h_2 \rightarrow f_{\text{clique},3}$ o $h_3 \rightarrow f_{\text{clique},4}$, las líneas azules discontinuas, pues en ambos casos las cláusulas comparten un literal. Por tanto, $\mu_x(h) = 3 + 2 + 1 = 6$.

En vista de estos ejemplos, vemos que calcular la métrica sintáctica no es tan sencillo como contar los literales de una función, ni resulta viable intentar una aproximación voraz. En cualquier caso, es claro por lo que hemos visto hasta ahora que este problema se puede pensar como un problema de asignación. Podemos imaginarnos las cláusulas de una función f y las de f_{clique} como los nodos de un grafo bipartito, donde las aristas van entre cláusulas de f y f_{clique} , y el valor de cada arista es el número de literales que comparten las cláusulas que une. Con este planteamiento, el valor de μ_x se puede calcular en tiempo polinómico mediante el algoritmo húngaro [10]. Los detalles de la implementación no son excesivamente relevantes para nuestro estudio, pero se pueden consultar en [12].

Ahora, recordemos que una propiedad que interesa de las métricas es que el valor máximo sea exponencial con respecto a n . La puntuación máxima que puede obtener una función f es resultado de hacer una asignación perfecta con todas las cláusulas de f_{clique} , es decir, que para cada cláusula de f_{clique} haya una cláusula de f a la que esté asignada (distinta a la que esté asignada al resto de cláusulas de f_{clique}), y que dicha cláusula de f contenga todos los literales de la cláusula de f_{clique} . Esta función tiene tantas cláusulas como cliques de tamaño k puede haber en un grafo de tamaño n , lo cual es $\binom{n}{k}$, y cada cláusula tiene $\binom{k}{2}$ literales, por lo que $\mu_x(f_{\text{clique}}) = \binom{n}{k} \binom{k}{2}$. Viene bien insistir en que alcanzar la puntuación máxima de la métrica sintáctica (la puntuación de f_{clique}) no es condición suficiente para que una función compute $\text{Clique}_{n,k}$. Por ejemplo, cualquier función reducida de la forma $f_{\text{clique}} \vee g$, independientemente de la función g , tendrá la puntuación máxima (las cláusulas de g no aportan nada).

Con esto, es interesante tener una idea de cómo pueden comportarse las puertas lógicas de un circuito con respecto a esta métrica. En [12] se vieron dos resultados iniciales muy interesantes:

Proposición 4.1.1. *Se cumplen las siguientes relaciones:*

1. Dadas dos funciones booleanas f y g , se cumple que $\mu_x(f \vee g) \leq \mu_x(f) + \mu_x(g)$.
2. Dado un circuito C que computa una función f usando únicamente puertas OR y NOT, se cumple que $|C| \geq \mu_x(f) - 1$.

El primer resultado, que se puede pensar como una desigualdad triangular, nos viene a decir que el crecimiento de la métrica que se puede esperar con puertas OR es, como mucho, lineal. La importancia del segundo resultado es que, si no disponemos de la puerta AND, entonces cualquier circuito que compute $\text{Clique}_{n,k}$ debe ser de tamaño exponencial.

Estos dos resultados juntos nos muestran que, si hay alguna esperanza de computar $Clique_{n,k}$ con circuitos de tamaño polinómico, esta reside en la capacidad de las puertas AND de hacer que la puntuación crezca más rápidamente.

¿Y realmente podemos esperar esto de las puertas AND? En principio, las cotas no lo impiden. Si tenemos dos funciones f y g en FND, es decir, $f = \bigvee_{i=1}^{m_1} f_i$ y $g = \bigvee_{j=1}^{m_2} g_j$, y r es una cota superior del tamaño de las cláusulas f_i y g_j , como $f \wedge g = \bigvee_{i=1}^{m_1} \bigvee_{j=1}^{m_2} (f_i \wedge g_j)$, el valor de la métrica se ve acotado por $2rm_1m_2$, pues se forman m_1m_2 cláusulas de tamaño, como máximo, $2r$. Como $\mu_x(f) \leq rm_1$ y $\mu_x(g) \leq rm_2$, y en general $2rm_1m_2 \not\leq rm_1 + rm_2$, no se cumple una desigualdad como la que se cumple en el caso de la puerta OR. Lo que sí se cumple en general es que $\mu_x(f \wedge g) \leq 2\mu_x(f)\mu_x(g)$. Por tanto, la puerta AND puede llegar a generar un crecimiento cuadrático en la métrica. Esto, a priori, abre la posibilidad a que efectivamente un circuito pueda llegar al valor máximo de la métrica empleando un número polinómico de puertas.

Por ello, el objetivo de los experimentos será observar experimentalmente si las puertas AND son así de productivas en el cálculo de $Clique_{n,k}$ o si, de forma contraria, su productividad decae drásticamente a medida que la función computada se parece más a f_{clique} . Anticipamos que lo que se observará principalmente es lo segundo, por lo que nos centraremos en estudiar otros fenómenos asociados al crecimiento de la métrica sintáctica y su posible vínculo con el drástico decaimiento de la productividad de las puertas AND.

4.1.2. Consideraciones previas a los experimentos

4.1.2.1. Simplificación de las funciones

Antes de pasar a los experimentos, debemos mencionar un resultado muy importante (y de uso muy común en el campo de los circuitos booleanos) de cara a simplificar las tareas siguientes.

Proposición 4.1.2. *Sea C un circuito booleano que computa una función f . Entonces, existe otro circuito C' que computa f tal que $|C'|$ es polinómico con respecto a $|C|$ y todas las puertas NOT de C' se encuentran en el primer nivel del circuito.*

Aunque no lo demostremos, la idea trata de duplicar el circuito (uno con variables sin negar y otro con las variables negadas) y interconectarlos aprovechando las leyes de De Morgan. La relevancia de esta proposición es que, sin pérdida de generalidad, podemos suponer que los circuitos están compuestos únicamente de puertas AND y OR, y que las entradas del circuito son las variables y sus negaciones, y lo que hace esta suposición válida es que el tamaño del circuito crece de forma polinómica y no exponencial al hacer esta transformación, que es lo que queremos evitar.

4.1.2.2. Objeto de estudio

El problema con el que estaremos trabajando va a ser $Half-Clique_n$, que, como ya mencionamos previamente, es un problema $\mathcal{NP}$ -completo. Como en este problema el tamaño del clique es siempre $k = \lfloor \frac{n}{2} \rfloor$, solo tenemos que elegir el valor de n . Para

n pequeño, hasta 4, los cliques serían de tamaño como mucho 2, que sencillamente es una arista, lo cual es trivial.

Por otro lado, tengamos en cuenta que este problema tiene tamaño de entrada $\binom{n}{2}$, una por cada posible arista. A poco que n se hace grande, como el número de aristas es cuadrático con respecto a n , los costes computacionales crecen considerablemente, así como el número de funciones booleanas (en vista de la proposición 2.2.1), con lo que se hace totalmente inviable pensar en representar las funciones como cadenas de 1s y 0s que reflejen la salida de cada una de las 2^n posibles entradas. Sin ir más lejos, si tomamos $n = 10$ tendremos 45 variables de entrada, lo que haría que representar una función de esta manera requiera de $2^{45} \approx 10^{13}$ bits, que es excesivo. Incluso si prescindimos de esta representación, los costes computacionales con la representación como forma normal disyuntiva, que mencionaremos a continuación, son también prohibitivos de cara a hacer experimentos de tamaños suficientemente significativos.

Finalmente, $n = 8$ es el primer valor de n con el que cada clique (de tamaño $k = 4$) requiere de más aristas que vértices, que con base en intuición y experimentación previa, da la sensación de poder ser relevante. Por tanto, estos son los valores de n y k que fijaremos para los experimentos siguientes, $n = 8$ y $k = 4$, lo que implica que tendremos $\binom{8}{2} = 28$ entradas para representar las aristas.

4.1.2.3. Representación de funciones

Dado que vamos a estar trabajando constantemente con la métrica sintáctica, esto invita directamente a representar las funciones en su forma normal disyuntiva. Como vamos a hacer los experimentos en Python, la representación de funciones será como lista de listas de enteros, donde cada lista de enteros representará una cláusula, y cada entero un literal que representará una arista.

Para utilizar enteros para la representación de las aristas, estas se ordenarán en orden lexicográfico, y se les asignará a cada una la posición que ocupan en este orden. Es decir, se ordenarán como se muestra a continuación:

$$\begin{array}{ll} \{1, 2\} & \leftrightarrow 1 \\ \{1, 3\} & \leftrightarrow 2 \\ & \vdots \\ \{6, 8\} & \leftrightarrow 27 \\ \{7, 8\} & \leftrightarrow 28 \end{array}$$

En vista a la proposición 4.1.2, todas las puertas NOT se pueden poner en el primer nivel, así que para representar un literal negado sencillamente multiplicaremos el entero correspondiente por -1 .

Así, la lista $[[1, -12, 23], [19, 10], [-4, -15]]$ representa a la función en forma normal disyuntiva $x_{1,2}\bar{x}_{2,7}x_{5,6} \vee x_{4,5}x_{2,5} \vee \bar{x}_{1,5}x_{3,5}$.

Con esto, se simplifica bastante el manejo de las funciones al hacer conjunciones y disyunciones:

- La aplicación de la puerta OR se convierte en concatenar listas.

- La aplicación de la puerta AND se convierte en concatenar todos los pares de cláusulas.
- En ambos casos, pueden eliminarse cláusulas por absorción, cuestión que resultará abordable aprovechando esta representación.

Aunque se ha reaprovechado mucho código del que se usó en [12] para el manejo de la métrica sintáctica y de esta representación de funciones, se han modificado y añadido muchas cosas para facilitar los desarrollos adicionales necesarios para nuestro estudio (como el cambio que se comentará posteriormente acerca de la reducción de formas normales disyuntivas). Todos los detalles de implementación se pueden consultar en [8].

4.2. Experimentos iniciales

En [12] se hicieron diversos experimentos que parecían mostrar el siguiente fenómeno: a partir de ciertos valores de la métrica, la combinación de funciones con puertas AND dejaba de ser lo suficientemente productiva.

Replicando experimentos como este, podemos ver gráficas como la siguiente:

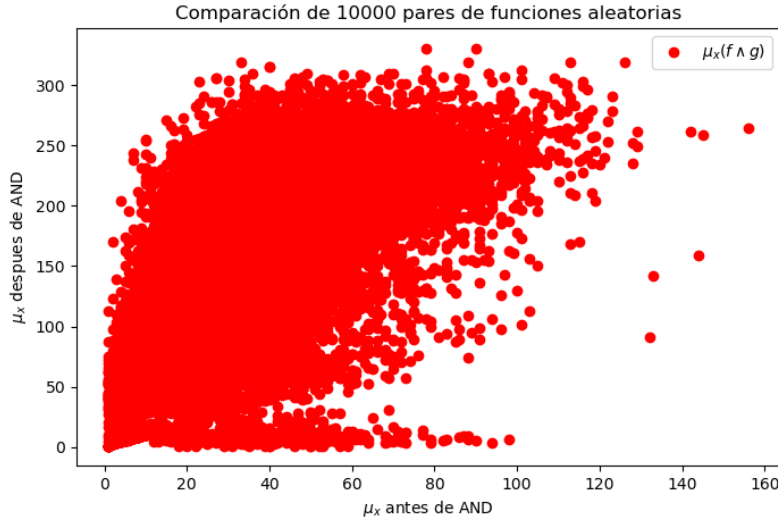


Figura 4.4: Crecimiento de μ_x con funciones aleatorias

Esta gráfica se ha realizado tomando funciones aleatorias f y g y graficando, para cada par de funciones, los puntos $(\mu_x(f), \mu_x(f \wedge g))$ y $(\mu_x(g), \mu_x(f \wedge g))$. Vemos en ella que, a partir de cierto valor de μ_x , el crecimiento que se consigue con las puertas AND se estanca y no es superior al de las puertas OR (al menos no significativamente, pues queda muy lejos de ser un crecimiento cuadrático). Recordemos que alcanzar la puntuación máxima de la métrica sintáctica es una condición necesaria para computar $Clique_{n,k}$, que en el caso de $n = 8$ y $k = 4$ es 420, y sin embargo vemos en la gráfica que las funciones aleatorias ni siquiera alcanzan una puntuación de 350. La intuición que surge de esto es que, si a partir de cierto punto no

podríamos aprovecharnos del posible crecimiento cuadrático de la métrica con las puertas AND, quizá seguiría siendo necesaria una cantidad exponencial de puertas para obtener la puntuación máxima.

Y fijémonos en una cosa: aunque el crecimiento máximo de la puntuación mediante el uso de las puertas AND sí fuera suficientemente alto, esto no sería condición suficiente para que se pueda obtener la puntuación máxima con un circuito de tamaño pequeño. Veamos esto con más detalle.

Un fenómeno más débil que se puede estudiar es si concatenar los crecimientos máximos posibles de la puntuación hasta llegar a la puntuación máxima requiere de una sucesión de crecimientos larga o corta. Por ejemplo, podemos imaginar que tenemos dos funciones f_1 y g_1 en el rango de puntuaciones $[1, 10]$ tales que la puntuación $\mu_x(f_1 \wedge g_1) = p_1$ es la máxima para funciones con puntuación en este intervalo. A continuación, existirán otras dos funciones f_2 y g_2 con puntuaciones $\leq p_1$ tales que la puntuación $\mu_x(f_2 \wedge g_2) = p_2$ es la máxima que se puede obtener para funciones con puntuación $\leq p_1$. Siguiendo con este proceso, obtendríamos una sucesión de parejas de funciones $f_i, g_i, 1 \leq i \leq M$ tales que $\mu_x(f_i), \mu_x(g_i) \leq p_{i-1}$ y $\mu_x(f_i \wedge g_i) = p_i$ es el valor máximo de μ_x que se puede obtener para funciones con puntuaciones $\leq p_{i-1}$, tomando como p_0 un valor pequeño (como 10), y obteniendo como p_M el valor máximo de la métrica, es decir, $p_M = \binom{n}{k} \binom{k}{2}$.

Básicamente, lo que estamos haciendo con este proceso es coger las funciones más productivas posibles, juntarlas y, limitándonos a la puntuación obtenida, coger las siguientes funciones más productivas posibles. En cierto modo, esto es hacer trampas, porque al pasar de funciones f_i y g_i a f_{i+1} y g_{i+1} , no estamos teniendo cuidado de que estas últimas dos funciones sean resultado directo de emplear las dos anteriores. Podría ser que f_i fuera computable con un circuito pequeño y f_{i+1} requiera de uno muy grande.

Por tanto, si viéramos que esta sucesión de funciones “ideales” es corta, o lo que es lo mismo, que M es pequeño, no quedaría demostrado que se puede conseguir la puntuación máxima con un circuito de tamaño polinómico. No obstante, la intención es precisamente la contraria. Recordemos que la intuición de la que partimos es que este último fenómeno no puede ocurrir. Por ello, la utilidad de esta simplificación de hacer trampas es que, si viéramos que esta sucesión ideal es muy larga, es decir, que incluso haciendo trampas y tomando siempre funciones óptimas M es un valor muy grande (idealmente, que la relación que tenga con n sea exponencial), entonces es seguro que no existiría un circuito de tamaño polinómico que pudiera alcanzar la puntuación máxima.

Por tanto, la idea original era estudiar si, permitiendo hacer trampas, era aun así necesaria una sucesión de parejas de funciones muy larga.

Un resultado que reafirma la idea de que alcanzar el valor máximo de la métrica con una cantidad pequeña de puertas y sin hacer trampas puede ser imposible viene de simular *Half-Clique*₈. En [12], como último resultado experimental, se empleó un algoritmo genético para simular el problema. El algoritmo consiste en tomar como población inicial las entradas (las funciones formadas por un único literal), y en cada generación generar nuevas funciones tomando parejas de funciones de la población actual y mezclándolas con puertas OR y AND, siendo la función de fitness la propia métrica μ_x , la cual limita las funciones que sobreviven entre cada

generación. Por la forma en que se obtienen las funciones y la ausencia de una función de mutación, esta simulación nos proporciona funciones obtenidas sin hacer las trampas que mencionábamos previamente, pues cada función se obtiene por una combinación con OR o AND de dos funciones que a su vez se habían obtenido de esa misma manera, hasta llegar a las entradas (la población inicial).

Hemos modificado y ampliado el código que se usó para esto, y hemos probado con un mayor número de generaciones, tamaños de población y métodos de selección, para probar métodos con elitismo y sin él. En todos los casos los resultados son similares a los que se obtuvieron en dicho trabajo.

No obstante, de cara a nuestros experimentos, hemos añadido un cambio relevante, y es acerca de la reducción de formas normales disyuntivas. En el trabajo mencionado las reducciones solo se hacían por absorción, pues hacer reducciones de cualquier otro tipo es significativamente más complicado. Afortunadamente, hemos contado con una librería, llamada PyEDA [1], con la cual se pueden hacer reducciones mucho mejores. Se pueden consultar todos los detalles de implementación en [8].

La relevancia surge principalmente de que, por cuestiones prácticas, es necesario limitar la longitud (número de cláusulas) de las formas normales disyuntivas, pues en caso contrario los costes computacionales se vuelven inabarcables. Con mejores reducciones, es posible conseguir muchas más funciones manteniendo los mismos límites. Así, en nuestros experimentos las puntuaciones obtenidas con el algoritmo genético llegan a ser mayores, pero, en cualquier caso, sigue ocurriendo el mismo fenómeno que se ve en [12]: el crecimiento de la métrica se estanca. Véase la gráfica de uno de nuestros experimentos, en este caso con puertas AND y OR.

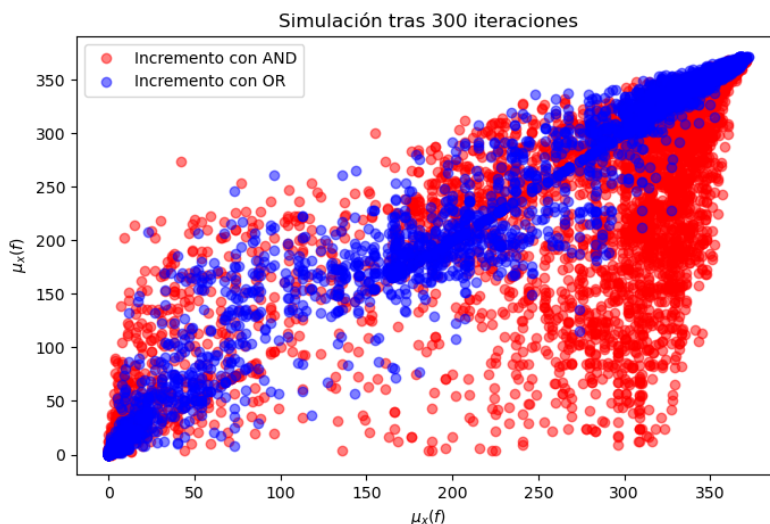


Figura 4.5: Simulación de 300 iteraciones

Al igual que antes, el eje de las abscisas representa la puntuación de una función originalmente, y el eje de las ordenadas la puntuación tras mezclarse cada función con alguna otra. Los puntos azules son los resultantes de juntar funciones con una puerta OR, y los rojos los resultantes de la puerta AND. Interesa mencionar que todos los puntos de la gráfica surgen de una misma ejecución en la que se ha mantenido una

población común para ambas puertas, de forma que las funciones resultantes se obtienen mediante el uso de ambas puertas.

De nuevo, vemos que las puertas AND dejan de ser suficientemente productivas a partir de cierto punto. Este resultado es especialmente interesante, puesto que podemos pensar en las funciones resultantes del algoritmo genético como “funciones pequeñas”. Dado que en este algoritmo partimos de las entradas y las vamos progresivamente combinando, con un límite al número de generaciones e iteraciones por generación, las funciones surgen de una combinación natural de funciones con puertas AND y OR (es decir, como si se fuera generando un circuito booleano) y podemos acotar el tamaño máximo de estas funciones. Empleando una selección aleatoria o con poco elitismo podemos esperar que la mayoría de funciones que se generan con este algoritmo sean producto de un circuito relativamente pequeño. Por la aleatoriedad de la función de selección, es cierto que si tuviéramos una cantidad de tiempo ilimitada podría computarse cualquier función booleana de 28 entradas. No obstante, es de esperar que la mayoría de las funciones sean computables con circuitos relativamente pequeños. Nos referiremos a estas funciones como **funciones simuladas**, y serán muy importantes dentro de poco.

Con todo esto, la vía de estudio original iba a consistir en lo siguiente: centrándonos en $Half-Clique_8$, donde la puntuación máxima es $\binom{8}{4}\binom{4}{2} = 420$, dividir el intervalo de puntuaciones $[0, 420]$ en diversos subintervalos, y en cada uno de ellos buscar las parejas de funciones más productivas posibles (mediante, por ejemplo, otro algoritmo genético) y estudiar sus propiedades.

Sin embargo, a base de hacer pruebas y experimentos con otras poblaciones iniciales en ciertos intervalos (haciendo así trampas como las que hemos mencionado previamente, pues dichas funciones no tenían por qué surgir de una generación natural como la de la simulación anterior), vimos que las intuiciones de las que partíamos no eran exactamente correctas: resulta que sí existen funciones que, al mezclarse, resultan ser muy productivas, y permiten llegar a la puntuación máxima, haciendo trampas, mucho más rápido de lo que esperábamos. Pero, ¿de dónde surgen estas funciones?

A la hora de buscar funciones muy productivas, la pregunta natural que surge es “¿cómo se puede favorecer un incremento máximo de la métrica?”. La cuestión es que, según se ve a la hora de hacer experimentos, la puntuación resultante de combinar dos funciones con la puerta AND casi siempre se aleja muchísimo del crecimiento cuadrático máximo que se podría esperar por las cotas elaboradas anteriormente. Entonces, otra forma de pensar en la misma cuestión es “¿cómo se puede obtener funciones con la menor pérdida de puntuación sobre la máxima posible?”. Ahora lo que veremos son un tipo de funciones que precisamente logran esto.

Vimos en el ejemplo 4.1.2 que, al calcularse $\mu_x(f)$, como esto consiste en el cálculo de una mejor asignación, hay casos en los que algunas cláusulas de f no pueden asignarse con la cláusula de f_{clique} con la que más literales comparten, situación a la que nos referimos con la idea de colisión. Pues precisamente veremos que las funciones que más evitan las colisiones son las que resultan obtener mejores valores de μ_x al combinarse con otras funciones.

Definición 4.2.1. Sea $f = \bigvee_{i=1}^m f_i$. Para $1 \leq i \leq m$, definimos

$$\mathcal{M}_i(f) = \left\{ j \in \left\{ 1, \dots, \binom{n}{k} \right\} \mid |f_i \cap f_{\text{clique},j}| \text{ es máximo} \right\}.$$

Así, definimos

$$\mathcal{M}(f) = \left\{ \varphi : \{1, \dots, m\} \rightarrow \left\{ 1, \dots, \binom{n}{k} \right\} \mid \varphi(i) \in \mathcal{M}_i(f), \forall i \in \{1, \dots, m\} \right\}.$$

Lo que se pretende recoger con esta definición es, para cada función booleana f , el conjunto de emparejamientos φ entre cláusulas de f y f_{clique} tales que el número de literales compartidos entre cláusulas es máximo. La cuestión es que estos emparejamientos no tienen por qué ser asignaciones válidas a la hora de calcular $\mu_x(f)$, pues puede ser que varias cláusulas de f se emparejen con la misma cláusula de f_{clique} .

Definición 4.2.2. Decimos que una función booleana $f = \bigvee_{i=1}^m f_i$ es **pseudoaleatoria** si existe alguna función $\varphi \in \mathcal{M}(f)$ inyectiva tal que $|f_i \cap f_{\text{clique},\varphi(i)}|$ es igual al número de literales sin negar en f_i , $\forall i \in \{1, \dots, m\}$.

Esta definición representa la idea que queríamos reflejar de “pérdida mínima de puntuación”. Que exista una función en $\mathcal{M}(f)$ que sea inyectiva significa que dicha función representa un emparejamiento óptimo que es válido a la hora de calcular $\mu_x(f)$ con el algoritmo húngaro. Es por ello que “se consigue toda la puntuación que se podría llegar a obtener”. Se puede pensar que estas funciones son aquellas tales que cada una de las cláusulas contiene un “trozo de clique” distinto a las demás.

El nombre de estas funciones surge de cómo se descubrieron originalmente. En breve expondremos una forma de obtener funciones pseudoaleatorias de cualquier puntuación. La cuestión es que una versión muy similar de dicha función se creó originalmente para obtener funciones aleatorias de puntuaciones determinadas, sin tener en mente cuestiones relacionadas con la estructura de las funciones resultantes, y en ella se toman “trozos de clique” porque facilita la obtención de puntuaciones concretas. Fue más tarde cuando nos dimos cuenta de que, aunque haya una gran aleatoriedad en la generación de estas funciones, llamar a las funciones resultantes “aleatorias” no resulta del todo preciso. Estas funciones están **muy orientadas a computar clique**. El hecho de que cada cláusula contenga un “trozo de clique” es un fenómeno demasiado restrictivo como para hablar de estas funciones como realmente aleatorias. De ahí el nombre de pseudoaleatorias, “bastante aleatorias, pero no del todo”.

Ejemplo 4.2.1. Volvamos a ver las funciones del ejemplo 4.1.2.

En el caso de f , la asignación ideal de cada cláusula era única. Por tanto, $\mathcal{M}(f) = \{\varphi_f\}$, donde $\varphi_f(1) = 1$ y $\varphi_f(2) = 3$. φ_f es inyectiva, por lo que f es pseudoaleatoria.

En el caso de g , las asignaciones ideales no son únicas, pues g_1 comparte una cantidad máxima de literales tanto con $f_{\text{clique},1}$ como con $f_{\text{clique},3}$. Por tanto, $\mathcal{M}(g) = \{\varphi_g^1, \varphi_g^2\}$, donde $\varphi_g^1(1) = 1$, $\varphi_g^2(1) = 3$ y $\varphi_g^1(2) = \varphi_g^2(2) = 4$, y ambas funciones son

inyectivas. No obstante, $|g_1 \cap f_{\text{clique}, \varphi_g^1(1)}| = |g_1 \cap f_{\text{clique}, \varphi_g^2(1)}| = 2 < 3$, que es el número de literales sin negar en g_1 , por lo que g no es pseudoaleatoria.

Finalmente, para h las asignaciones ideales son únicas. Así, $\mathcal{M}(h) = \{\varphi_h\}$, donde $\varphi_h(1) = 1$, $\varphi_h(2) = 2$ y $\varphi_h(3) = 2$. Sin embargo, en este caso la función no es inyectiva, por lo que h no es pseudoaleatoria.

Ahora, de cara a poder experimentar con este tipo de funciones, nos interesa tener una función que, dada una puntuación p , nos permita obtener una función pseudoaleatoria f tal que $\mu_x(f) = p$. ¿Y cómo obtenemos una función que cumpla con la definición de pseudoaleatoria? Aunque la definición parezca algo enrevesada, este tipo de funciones son realmente fáciles de obtener. Para esto, basta con elegir un subconjunto de cláusulas de f_{clique} , y tomar de cada una de ellas un subconjunto de literales. Por la propia forma en que generamos estas funciones, es claro que las funciones resultantes serán pseudoaleatorias; basta asignar a cada cláusula de f la cláusula de f_{clique} de la que proviene.

De cara a elegir el número de cláusulas, este depende directamente del valor p . Por un lado, dado que cada cláusula de f_{clique} (en el caso de $n = 8$ y $k = 4$) contiene $\binom{k}{2} = 6$ literales, el mínimo número de cláusulas que puede tener f es $\lceil \frac{p}{6} \rceil$. Por otro lado, el número de cláusulas máximo de f está acotado, por un lado, por el número de cláusulas en f_{clique} (pues, de haber más, ninguna asignación podría ser inyectiva por el *Principio del palomar*), y, por otro lado, por p (pues cada cláusula contendrá, al menos, un literal). Por tanto, el número máximo de cláusulas es $\min\left(\binom{n}{k}, p\right) = \min(70, p)$. Con esto, basta tomar un número entre estas cotas.

Luego, determinado el número de cláusulas m , se tomará dicho número de cláusulas aleatorias de f_{clique} . Para cada una de estas hay que determinar el número de literales que se tomará. Para esto, bastará descomponer p en una suma de m números positivos menores que $\binom{k}{2} = 6$, pues es el número de literales en cada cláusula de f_{clique} . Por ejemplo, si quisiéramos obtener una función con $p = 13$ y $m = 3$, podemos descomponer p como $13 = 5 + 2 + 6$, siendo cada sumando el número de literales a tomar de las cláusulas elegidas.

Aun así, queremos aportar algo más de variedad a estas funciones. Según el proceso anterior, las funciones resultantes no tendrían ningún literal negado. Esto nos colocaría en el contexto de los circuitos monótonos, en el cual solo se permite el uso de puertas AND y OR. En este ámbito ya hemos mencionado que Razborov demostró que cualquier circuito que compute $\text{Clique}_{n,k}$ debe tener tamaño superpolinómico [11], por lo que estudiar este caso solo reforzaría los resultados que ya conocemos. Así, en el proceso anterior, también añadiremos a cada cláusula de f cierta cantidad de literales negados, por salirnos de dicho ámbito y aportar cierta variedad a las funciones. Dado que las variables negadas no suman ni restan puntuación bajo μ_x , esto no hace que las funciones pierdan la condición de pseudoaleatoriedad.

El único problema que surge con este procedimiento es que, al tomar subconjuntos de cláusulas de f_{clique} , puede que se produzcan reducciones por absorción, conllevando a una puntuación menor a la esperada. No obstante, se ha comprobado experimentalmente que esto ocurre relativamente poco. Sumado al hecho de que este proceso es muy rápido, basta repetirlo unas pocas veces hasta que no ocurra este efecto. El funcionamiento se puede ver en el siguiente pseudocódigo:

Algoritmo 1 Algoritmo para obtener f pseudoaleatoria tal que $\mu_x(f) = p$.

Entrada: Puntuación p .

Salida: f pseudoaleatoria tal que $\mu_x(f) = p$.

```

1:  $fallo \leftarrow True$ 
2:  $f \leftarrow []$ 
3: while  $fallo$  do
4:    $fallo \leftarrow False$ 
5:    $m \leftarrow n$  aleatorio en  $[\lceil \frac{p}{6} \rceil, \min(70, p)]$ 
6:    $Clausulas \leftarrow$  subconjunto aleatorio de  $\{f_{clique,1}, \dots, f_{clique,70}\}$  de tamaño  $m$ 
7:    $Tams \leftarrow$  partición aleatoria de  $p$  en  $m$  sumandos
8:   for  $i \in Tams$  do
9:      $Literales \leftarrow$  subconjunto aleatorio de  $Clausulas[i]$  de tamaño  $Tams[i]$ 
10:     $LiteralesNeg \leftarrow$  subconjunto aleatorio de  $Clausulas[i] \setminus Literales$ 
11:    for  $x \in LiteralesNeg$  do
12:       $x \leftarrow \neg x$ 
13:    end for
14:     $f = f \cup [Literales \cup LiteralesNeg]$ 
15:  end for
16:   $f \leftarrow reducir(f)$ 
17:  if  $\mu_x(f) \neq p$  then
18:     $fallo \leftarrow True$ 
19:  end if
20: end while
21: return  $f$ 

```

Los detalles del código se pueden consultar en [8].

Pudiendo ya obtener funciones pseudoaleatorias con las puntuaciones que queramos, queremos ver cómo se comportan al combinarlas con puertas AND. Vemos esto en la siguiente gráfica, en la que comparamos sus crecimientos con puertas AND al que se obtiene con funciones simuladas.

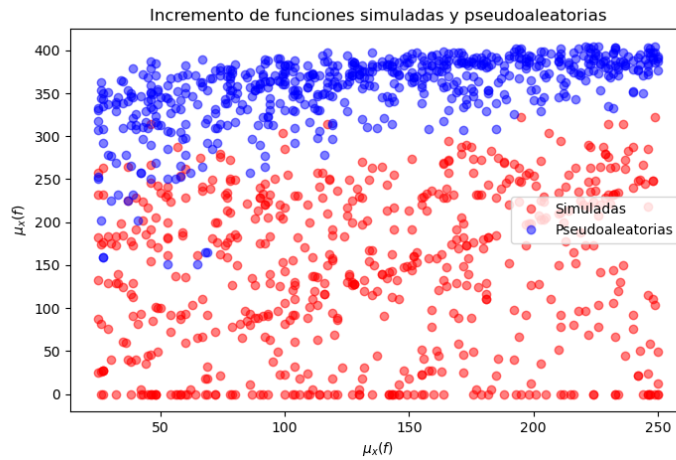


Figura 4.6: Crecimiento de μ_x con funciones simuladas y pseudoaleatorias

Esta gráfica se ha generado con 300 iteraciones del siguiente procedimiento: tomamos dos valores $p_1, p_2 \in [25, 250]$, obtenemos dos funciones simuladas f_1 y f_2 y dos funciones pseudoaleatorias g_1 y g_2 tales que $\mu_x(f_1) = \mu_x(g_1) = p_1$ y $\mu_x(f_2) = \mu_x(g_2) = p_2$ y mostramos en la gráfica los puntos $(p_1, \mu_x(f_1 \wedge f_2))$ y $(p_2, \mu_x(f_1 \wedge f_2))$ en rojo, y los puntos $(p_1, \mu_x(g_1 \wedge g_2))$ y $(p_2, \mu_x(g_1 \wedge g_2))$ en azul.

Vemos en esta gráfica que el crecimiento obtenido de las funciones pseudoaleatorias es muchísimo mayor al de las funciones simuladas. Esto efectivamente rompe con la hipótesis inicial de que, incluso haciendo trampas, harían falta secuencias de funciones muy largas para alcanzar la puntuación máxima de μ_x . Haciendo pruebas, se ha visto que haciendo trampas como las que explicamos anteriormente, pero empleando únicamente funciones pseudoaleatorias, basta con secuencias de unas 5 parejas de funciones pseudoaleatorias para alcanzar dicha puntuación.

No obstante, este tipo de funciones siguen siéndonos muy útiles para nuestro estudio. En vista de este resultado, otra idea que se plantea es la siguiente: ¿Se podrán computar este tipo de funciones con circuitos de tamaño polinómico? De ser así, podría ser que el resultado previo nos llevara a que efectivamente se puede obtener la puntuación máxima con circuitos relativamente pequeños, a través de funciones pseudoaleatorias. Sin embargo, nuestras sospechas van en la dirección contraria. Para explicarlas, volvamos a la elección del nombre de “funciones pseudoaleatorias”.

Aunque estas funciones parezcan tener un grado de aleatoriedad enorme (número de cláusulas aleatorio, subconjuntos de literales aleatorios, ...), ya mencionamos que realmente sería algo impreciso hablar de estas funciones como completamente aleatorias. El hecho de que los literales de cada cláusula se obtengan tomando subconjuntos de literales de cláusulas de f_{clique} limita mucho las funciones que se pueden generar. Sí, son aleatorias, pero no tanto. Estas funciones “están generadas para funcionar muy bien”, pues los literales sin negar de cada cláusula están tomados de forma que todos se refieran a aristas de un mismo clique. Por ejemplo, en ninguna función pseudoaleatoria nos podemos encontrar una cláusula como $x_{1,2}x_{3,4}x_{2,5}$, pues las aristas a las que refieren dichos literales involucran a 5 vértices, que son más de los que puede haber en un clique de tamaño 4.

Estas restricciones nos hicieron pensar en cómo de viable podría resultar el intentar computar este tipo de funciones, y hay ciertas cuestiones que nos hicieron pensar que estas funciones no pueden ser producto de circuitos booleanos pequeños.

4.2.1. Conjeturas acerca de las funciones pseudoaleatorias

Como hemos mencionado, a través de ver estas funciones pensamos en ciertas conjeturas, principalmente dos, que nos hicieron pensar que probablemente estas funciones son “pedir demasiado” y que, por muy buenas que sean, es probable que sean demasiado difíciles de computar como para que nos ayuden a alcanzar el valor máximo de μ_x con circuitos relativamente pequeños.

Veamos, de una manera informal, de qué tratan estas conjeturas, para que luego sea más fácil intentar formalizarlas.

Conjetura 1. *Las funciones pseudoaleatorias reflejan unos niveles de diversidad en sus fórmulas que no pueden ser resultantes de circuitos booleanos pequeños.*

Para entender esta idea, recordemos que, por lo visto anteriormente, si hubiera alguna forma de obtener el valor máximo de μ_x con un circuito de tamaño polinómico, esto tiene que pasar a través de aprovechar el crecimiento cuadrático que puede aportar la puerta AND. Por tanto, es de esperar que cualquier circuito pequeño con un valor alto de la métrica se aproveche de este fenómeno.

No obstante, aprovechar la puerta AND tiene sus consecuencias. Una cuestión resultante de la aplicación de la puerta AND a dos funciones booleanas es que “se reaprovecha mucho de cada FND”.

Más formalmente, sean dos funciones $f = \bigvee_{i=1}^{m_1} f_i$ y $g = \bigvee_{j=1}^{m_2} g_j$. Vamos a fijarnos en alguna cláusula concreta de f , que llamaremos f_{i_0} . Esta cláusula aparece únicamente una vez en la forma normal disyuntiva de f . Sin embargo, fijémonos ahora en lo que ocurre al hacer el AND de estas dos funciones. La fórmula resultante, que ya hemos visto anteriormente, es $f \wedge g = \bigvee_{i=1}^{m_1} \bigvee_{j=1}^{m_2} (f_i \wedge g_j)$. Hacer el AND de dos funciones en forma normal disyuntiva es esencialmente hacer una multiplicación, o convolución, de ambas fórmulas. Entonces, en un caso ideal, podría verse la cláusula f_{i_0} ligada a todas las cláusulas de g . Por ejemplo, tomando $m_1 = 2$ y $m_2 = 4$, y fijándonos en $i_0 = 2$,

$$f \wedge g = (f_1 \vee f_2) \wedge (g_1 \vee g_2 \vee g_3 \vee g_4) = f_1 g_1 \vee f_2 g_1 \vee f_1 g_2 \vee f_2 g_2 \vee f_1 g_3 \vee f_2 g_3 \vee f_1 g_4 \vee f_2 g_4.$$

Vemos que f_2 aparece 4 veces en la fórmula resultante. Esto realmente es una cota superior al número de apariciones, pues puede ser que la fórmula de la derecha de la igualdad se pueda reducir. En cualquier caso, en general, aplicar la puerta AND a dos FND hace que se reutilicen muchas de las cláusulas. Esto, de cara a que μ_x crezca rápidamente es muy bueno, porque esto es lo que hace que el crecimiento cuadrático sea teóricamente posible. No obstante, creemos que esto puede salir al precio de obtener fórmulas muy sobredefinidas muy pronto, es decir, que contengan más literales de los que aparecen en las cláusulas de f_{clique} .

Pensemos, por ejemplo, en la cláusula $x_{1,2}x_{1,3}$. Esta cláusula está formada por únicamente dos literales. Estos dos literales hacen referencia a aristas que inciden en los vértices 1, 2 y 3. De cara a computar $Clique_{n,k}$, si queremos que las cláusulas no estén sobredefinidas (es decir, que involucren a más de 4 vértices), esta cláusula solo podría juntarse con literales que involucren a estos tres vértices y a uno distinto. En el momento en que esta cláusula se combine con una cláusula que involucre a dos vértices distintos a los tres que hemos mencionado, la cláusula va a estar sobredefinida. Por ejemplo $x_{1,2}x_{1,3} \wedge x_{4,5} = x_{1,2}x_{1,3}x_{4,5}$ ya involucra a demasiados vértices.

Más inmediatos son casos de cláusulas como $x_{1,2}x_{4,5}$. Estos dos literales ya involucran a cuatro vértices, que son los que hacen falta para cada clique. Por tanto, de cara a que no se sobredefina esta cláusula, solo podrá juntarse con literales que refieran a los mismos vértices. En concreto, los literales $x_{1,4}$, $x_{1,5}$, $x_{2,4}$ y $x_{2,5}$ son los que harían falta para completar una cláusula de f_{clique} .

El problema que surge de esto es que, de cada a obtener la propia forma normal disyuntiva de f_{clique} , las cláusulas sobredefinidas se van a tener que eliminar en algún momento. Para esto hay esencialmente dos formas:

1. Absorción: $x_{1,2}x_{1,3} \vee x_{1,2}x_{1,3}x_{4,5} = x_{1,2}x_{1,3}$.
2. Simplificación: $x_{1,2}x_{3,4}x_{5,6} \vee x_{1,2}\overline{x_{3,4}}x_{5,6} \vee x_{1,2}x_{3,4}\overline{x_{5,6}} \vee x_{1,2}\overline{x_{3,4}}\overline{x_{5,6}} = x_{1,2}$.

La reducción por absorción implica que las cláusulas que se eliminan no eran útiles para empezar. Por otro lado, la reducción por simplificación requiere de muchas cláusulas muy concretas. En ambos casos, parece que la sobredefinición de cláusulas es un problema difícil de evitar.

Ahora, esto no ocurre con las funciones pseudoaleatorias. Estas funciones vienen directamente de tomar subconjuntos de cláusulas bien definidas y, como mucho, añadir algunos literales negados. Esto precisamente, en vista de lo que hemos explicado, parece muy difícil de obtener como resultado de mezclar funciones relativamente grandes con puertas AND. Claro es que hay una forma trivial de calcular estas funciones, la misma que para calcular f_{clique} : basta computar cada cláusula por separado y luego juntarlas todas con puertas OR. El problema de esto es que el número de cláusulas que hacen falta al final es $\binom{8}{4} = 70$, por lo que este método no se aprovecha del crecimiento cuadrático de las puertas AND.

Con todo esto, es difícil imaginarse que estas funciones aprovechen el crecimiento de la AND, y por tanto, no sean significativamente más fáciles de computar de lo que es f_{clique} .

Pasamos ahora a la segunda conjetura, que tiene mucha relación con la que acabamos de ver:

Conjetura 2. *Las funciones computables con circuitos booleanos pequeños tienden a estar sesgadas.*

¿A qué nos referimos con que una función esté sesgada? En cierto modo, lo que queremos decir es que “no trata a todos los inputs por igual”.

Por ejemplo, si nos fijamos en cada input por separado, y tomamos la misma cantidad de inputs que f_{clique} , es decir, $\binom{8}{2} = 28$, la función $x_1 \vee x_2 \vee \dots \vee x_{28}$ está muy poco sesgada, pues puedes intercambiar cualquier pareja de inputs y la función seguiría siendo la misma.

Este caso puede dar la idea de que tratar a todos los inputs por igual no es algo tan complicado. A fin de cuentas, esa función es trivialmente computable con 27 puertas OR. En general, para un número arbitrario de vértices en un grafo, harían falta $\binom{n}{2} - 1 = \frac{n(n-1)}{2} - 1$ puertas OR, que es cuadrático con respecto a n , no exponencial.

No obstante, esta función es un caso muy concreto. En general, la grandísima mayoría de funciones que vamos a manejar no van a tener un único literal por cada cláusula. A medida que el número de literales por cláusula crece, hacen falta más puertas para obtener el mismo efecto.

Esto supone un problema cuando pasamos de querer tratar por igual a todos los inputs a querer tratar por igual a todos los posibles subconjuntos de inputs. Una idea

razonable en el campo de los circuitos booleanos, que se explora en el trabajo [5], es pensar que cualquier circuito que trate a todos los subconjuntos de inputs por igual, se defina tal fenómeno como se defina, debe ser de tamaño exponencial, pues el número de subconjuntos en un conjunto de tamaño n es 2^n .

Por tanto, cuando se trabaja con funciones que se pueden computar con circuitos relativamente pequeños, se espera que la función esté sesgada, de forma que “ciertos inputs importen más que otros”. Todo esto queda sujeto a que se defina, en cada caso concreto, qué significa la importancia de cada input.

Ahora, ¿cómo se aplica esto al caso de f_{clique} ? Esta función, aunque no trate a todos los subconjuntos de inputs por igual, sí que trata a todos los cliques por igual. Se puede pensar que ningún clique es más importante que los demás. Tener información sobre las aristas del clique formado por los vértices 1, 2, 3 y 4 aporta lo mismo que si tuviéramos la misma información acerca del clique formado por los vértices 5, 6, 7 y 8. En general, la FND de f_{clique} es “muy simétrica”, y está muy poco sesgada.

La cuestión es que, aunque no estemos tratando por igual a todos los subconjuntos de inputs de tamaño $\binom{k}{2} = 6$, sí estamos tratando por igual a todos los subconjuntos de dicho tamaño que se refieren a un mismo clique. Como el número de cliques también es superpolinómico, da a pensar que esta función también requiere de una cantidad exponencial de puertas para ser computada.

En cuanto a las funciones pseudoaleatorias, estas no tienen por qué estar tan poco sesgadas como lo está f_{clique} , pero sospechamos que van a seguir estando mucho menos sesgadas que las funciones computadas con circuitos pequeños.

Con esto, quedan explicadas informalmente las conjeturas principales que tenemos acerca de estas funciones. A partir de aquí, lo que pretendemos hacer es definir ciertas medidas que sean capaces de reflejar los fenómenos que hemos tratado hasta ahora. Así, no solo pretendemos reafirmar nuestras conjeturas acerca de las funciones pseudoaleatorias. Aparte de esto, tenemos la esperanza de que estas medidas sean capaces de diferenciar fuertemente las funciones computables con circuitos pequeños del resto de funciones posibles, pues, de ser así, el estudio de estas medidas podría resultar de gran utilidad.

4.3. Medidas

4.3.1. Medición del solapamiento

Como comentábamos en nuestra primera conjetura, una cuestión que esperamos que ocurra en funciones computadas con circuitos pequeños es que se reutilice bastante información entre las cláusulas. Al contrario, parece que en las funciones pseudoaleatorias se manifiesta una gran diversidad de cláusulas y unas combinaciones de literales que no parecen propias de las funciones resultantes de circuitos pequeños.

Una forma que proponemos de medir este fenómeno es observar el promedio de literales que comparten las distintas cláusulas de cada función. Definimos así la siguiente medida:

Definición 4.3.1. Sea $f = \bigvee_{i=1}^m f_i$ una función booleana en forma normal disyuntiva con $m \geq 2$. Definimos la **medida de solapamiento** η_s de f como

$$\eta_s(f) = \frac{2}{m(m-1)} \sum_{1 \leq i < j \leq m} \frac{|f_i \cap f_j|}{\max(|f_i|, |f_j|)}.$$

Lo que esta medida refleja es la cantidad promedio de información que se comparte entre cláusulas de una misma función. Los sumandos del sumatorio son cocientes para normalizar los valores, pues más que la cantidad de literales que se comparten, nos interesa más fijarnos en la proporción de estos, de forma que pueda haber funciones con solapamientos muy grandes o muy pequeños independientemente del tamaño de las cláusulas.

De cara al uso de esta medida, tanto en el caso de estudio de *Half-Clique*₈ como el caso de otros valores de n y k , resulta interesante conocer un par de propiedades. En particular, vamos a estudiar los valores límite de esta medida, así como el valor que obtiene la función que computa *Clique* _{n,k} , pues será un valor a tener como referencia más adelante.

Proposición 4.3.1. Dada una función $f = \bigvee_{i=1}^m f_i$ en forma normal disyuntiva con $m \geq 2$ se cumple que $\eta_s(f) \in [0, 1]$.

Demostración. Dado que η_s se define como suma y producto de términos no negativos, es inmediato que $\eta_s(f) \geq 0$. Además, este valor se alcanza. Basta tomar una función cuyas cláusulas no tengan literales en común, como $x_{1,2}x_{3,4} \vee x_{5,6}x_{7,8}$.

Aparte, puesto que hay m cláusulas, se pueden formar $\binom{m}{2} = \frac{m(m-1)}{2}$ parejas. Por tanto, dada una pareja de cláusulas f_i y f_j ,

$$\begin{aligned} |f_i \cap f_j| &\leq \max(|f_i|, |f_j|) \\ \Rightarrow \frac{|f_i \cap f_j|}{\max(|f_i|, |f_j|)} &\leq 1 \\ \Rightarrow \sum_{1 \leq i < j \leq m} \frac{|f_i \cap f_j|}{\max(|f_i|, |f_j|)} &\leq \frac{m(m-1)}{2} \\ \Rightarrow \eta_s(f) = \frac{2}{m(m-1)} \sum_{1 \leq i < j \leq m} \frac{|f_i \cap f_j|}{\max(|f_i|, |f_j|)} &\leq \frac{2}{m(m-1)} \frac{m(m-1)}{2} \leq 1. \end{aligned}$$

□

Proposición 4.3.2. Dados n y k (tamaño de grafo y de clique, respectivamente), sea f_{clique} la función que computa *Clique* _{n,k} en forma normal disyuntiva, es decir,

$$f_{\text{clique}} = \bigvee_{\substack{X \subseteq \{1, \dots, n\} \\ |X|=k}} \bigwedge_{\substack{i, j \in X \\ i < j}} x_{i,j}.$$

El valor de la medida de solapamiento de f_{clique} es

$$\eta_s(f_{\text{clique}}) = \frac{1}{\left(\binom{n}{k} - 1\right) \binom{k}{2}} \left(\sum_{x=2}^k \binom{x}{2} \binom{k}{x} \binom{n-k}{k-x} - \binom{k}{2} \right).$$

Demostración. Dado que cada cláusula de f_{clique} representa la conjunción de aristas referentes a un clique, y en total hay $\binom{n}{k}$ cliques posibles, expresemos $f_{clique} = \bigvee_{i=1}^{\binom{n}{k}} f_i$.

Cada cláusula f_i tiene tantos literales como aristas hay en un clique de tamaño k , es decir, $\binom{k}{2}$. Por tanto,

$$\eta_s(f_{clique}) = \frac{2}{\binom{n}{k} \left(\binom{n}{k} - 1 \right)} \frac{1}{\binom{k}{2}} \sum_{1 \leq i < j \leq \binom{n}{k}} |f_i \cap f_j|. \quad (4.1)$$

Centrémonos entonces en calcular $\sum_{1 \leq i < j \leq \binom{n}{k}} |f_i \cap f_j|$. Este sumatorio calcula la intersección de cada pareja de cláusulas distintas de f_{clique} , sin repetición, por lo que hay en total $\binom{\binom{n}{k}}{2}$ sumandos. Un sumatorio más fácil de calcular es

$$\sum_{\substack{1 \leq i \leq \binom{n}{k} \\ 1 \leq j \leq \binom{n}{k}}} |f_i \cap f_j|.$$

Este último sumatorio calcula la intersección de todas las parejas de cláusulas con repetición e incluyendo la intersección de cada cláusula consigo misma. Cada par de cláusulas distintas aparece dos veces ($|f_i \cap f_j|$ y $|f_j \cap f_i|$) y cada cláusula aparece intersecada consigo mismo una vez. Por tanto, se cumple que

$$\sum_{1 \leq i < j \leq \binom{n}{k}} |f_i \cap f_j| = \frac{1}{2} \left(\sum_{\substack{1 \leq i \leq \binom{n}{k} \\ 1 \leq j \leq \binom{n}{k}}} |f_i \cap f_j| - \sum_{i=1}^{\binom{n}{k}} |f_i \cap f_i| \right). \quad (4.2)$$

Calculemos primero $\sum_{\substack{1 \leq i \leq \binom{n}{k} \\ 1 \leq j \leq \binom{n}{k}}} |f_i \cap f_j|$. Podemos expresar este sumatorio como

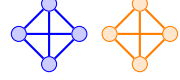
$$\sum_{\substack{1 \leq i \leq \binom{n}{k} \\ 1 \leq j \leq \binom{n}{k}}} |f_i \cap f_j| = \sum_{i=1}^{\binom{n}{k}} \sum_{j=1}^{\binom{n}{k}} |f_i \cap f_j|.$$

Al expresarlo de esta manera, vamos a poder aprovechar que f_{clique} es una función invariante bajo permutación. Es decir, al intercambiar dos variables x_i y x_j , la fórmula sigue siendo la misma. Por tanto, el segundo sumatorio va a ser el mismo para cualquier i . Esto implica que, si fijamos cierto índice i_0 ,

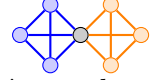
$$\sum_{\substack{1 \leq i \leq \binom{n}{k} \\ 1 \leq j \leq \binom{n}{k}}} |f_i \cap f_j| = \binom{n}{k} \sum_{j=1}^{\binom{n}{k}} |f_{i_0} \cap f_j|. \quad (4.3)$$

Ahora, dada cualquier cláusula f_i , como esta representa un clique y contiene $\binom{k}{2}$ literales, involucra k vértices. Empleando esto, para calcular $\sum_{j=1}^{\binom{n}{k}} |f_{i_0} \cap f_j|$ vamos a distinguir casos, según cuantos vértices involucren en común los cliques representados por f_{i_0} y f_j :

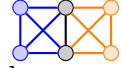
Caso 0: No hay vértices involucrados en común. Claramente, tampoco hay aristas en común, por lo que $|f_{i_0} \cap f_j| = 0$. Por tanto, los cliques con 0 vértices en común aportan 0 al sumatorio.



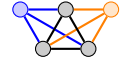
Caso 1: Hay un único vértice involucrado en común. En este caso, por tanto, tampoco hay aristas en común, por lo que $|f_{i_0} \cap f_j| = 0$. Por tanto, los cliques con 1 vértice en común también aportan 0 al sumatorio.



Caso 2: Hay dos vértices involucrados en común. Como dados dos vértices hay una única arista que los conecta, se tiene que $|f_{i_0} \cap f_j| = \binom{2}{2} = 1$. Los vértices en común pueden ser cualquiera de los k en los que se apoya f_{i_0} , por lo que tenemos $\binom{k}{2}$ elecciones posibles. Además, f_j puede referir a cualquier clique que contenga los dos vértices comunes y otros $k - 2$ que no estén en el clique de f_{i_0} , por lo que para determinar f_j hay $\binom{n-k}{k-2}$ opciones. Por tanto, los cliques con 2 vértices comunes aportan al sumatorio $\binom{2}{2} \binom{k}{2} \binom{n-k}{k-2}$.



Caso 3: Hay tres vértices involucrados en común. Dados tres vértices, hay una tres aristas que los conectan, así que se tiene que $|f_{i_0} \cap f_j| = \binom{3}{2} = 3$. Los vértices en común pueden ser cualquiera de los k en los que se apoya f_{i_0} , por lo que tenemos $\binom{k}{3}$ elecciones posibles. Además, f_j puede referir a cualquier clique que contenga los dos vértices comunes y otros $k - 3$ que no estén en el clique de f_{i_0} , por lo que para determinar f_j hay $\binom{n-k}{k-3}$ opciones. Por tanto, los cliques con 3 vértices comunes aportan al sumatorio $\binom{3}{2} \binom{k}{3} \binom{n-k}{k-3}$.



Caso x: En general, si hay x vértices involucrados en común, se tiene que $|f_{i_0} \cap f_j| = \binom{x}{2}$, hay $\binom{k}{x}$ elecciones de vértices posibles y $\binom{n-k}{k-x}$ opciones para determinar f_j . Por tanto, los cliques con x vértices comunes y $x \leq k$ aportan al sumatorio $\binom{x}{2} \binom{k}{x} \binom{n-k}{k-x}$.

Por tanto, finalmente obtenemos que

$$\sum_{j=1}^{\binom{n}{k}} |f_{i_0} \cap f_j| = \sum_{x=2}^k \binom{x}{2} \binom{k}{x} \binom{n-k}{k-x}.$$

Volviendo a la igualdad (4.3),

$$\sum_{\substack{1 \leq i \leq \binom{n}{k} \\ 1 \leq j \leq \binom{n}{k}}} |f_i \cap f_j| = \binom{n}{k} \sum_{x=2}^k \binom{x}{2} \binom{k}{x} \binom{n-k}{k-x}.$$

Ahora, dado que cada cláusula tiene $\binom{k}{2}$ literales,

$$\sum_{i=1}^{\binom{n}{k}} |f_i \cap f_i| = \sum_{i=1}^{\binom{n}{k}} |f_i| = \binom{n}{k} \binom{k}{2}.$$

Con todo esto, volviendo a (4.2), tenemos que

$$\sum_{1 \leq i < j \leq \binom{n}{k}} |f_i \cap f_j| = \frac{\binom{n}{k}}{2} \left(\sum_{x=2}^k \binom{x}{2} \binom{k}{x} \binom{n-k}{k-x} - \binom{k}{2} \right).$$

Finalmente, juntando todo en (4.1), concluimos que

$$\begin{aligned} \eta_s(f_{clique}) &= \frac{2}{\binom{n}{k} \left(\binom{n}{k} - 1 \right)} \frac{1}{\binom{k}{2}} \frac{\binom{n}{k}}{2} \left(\sum_{x=2}^k \binom{x}{2} \binom{k}{x} \binom{n-k}{k-x} - \binom{k}{2} \right) \\ &= \frac{1}{\left(\binom{n}{k} - 1 \right) \binom{k}{2}} \left(\sum_{x=2}^k \binom{x}{2} \binom{k}{x} \binom{n-k}{k-x} - \binom{k}{2} \right). \end{aligned}$$

□

Como consecuencia inmediata de esta proposición obtenemos que, siendo f_{clique} la función que computa *Half-Clique*₈ (es decir, $n = 8$ y $k = 4$), se tiene que

$$\eta_s(f_{clique}) \approx 0,20288055072 \approx 0,2,$$

valor que nos vendrá bien tener como referencia de cara a los experimentos posteriores.

De cara a experimentar con esta medida, su cálculo es muy sencillo. Se ilustra el funcionamiento en el siguiente pseudocódigo:

Algoritmo 2 Cálculo de $\eta_s(f)$.

Entrada: f expresada como lista de cláusulas.

Salida: $\eta_s(f)$.

```

1:  $suma \leftarrow 0$ 
2:  $m \leftarrow$  número de cláusulas en  $f$ 
3: for  $i = 1$  hasta  $m$  do
4:   for  $j = i + 1$  hasta  $m$  do
5:      $suma \leftarrow suma + \frac{|f_i \cap f_j|}{\max(|f_i|, |f_j|)}$ 
6:   end for
7: end for
8: return  $\frac{2}{m(m-1)} \cdot suma$ 
```

Dado que las cláusulas contienen, como mucho, $\binom{n}{2}$ literales, el cálculo de cada intersección se calcula fácilmente en coste $O\left(\binom{n}{2}^2\right)$. Como esto se hace para cada

pareja de cláusulas, si denotamos por m el número de cláusulas en f , el algoritmo se computa con complejidad $O\left(m^2 \binom{n}{2}^2\right)$, aunque en la práctica las cláusulas son generalmente bastante menores. Para más detalles, se puede consultar el código en [8].

4.3.2. Medición de la homogeneidad

Al hablar de nuestra segunda conjetura, comentábamos que era de esperar que en general las funciones computables con circuitos booleanos pequeños tendieran a estar, en cierto modo, sesgadas. Es decir, que no todos los inputs tuvieran el mismo peso a la hora de determinar el output.

La cuestión a la que quedaba sujeto esto era: ¿Qué significa que distintos inputs tengan el mismo peso a la hora de determinar el output? Hay muchas formas de definir esto, y en ocasiones pueden depender o aprovecharse de la representación que se haga de las funciones. Por ejemplo, en el trabajo de fin de grado de Enrique Carro Garrido y José Ramón Cobián Fernández [5] se definió la medida de ecuanimidad, que trata la importancia de las variables según efecto que tienen en la tabla de verdad de la función.

Otra forma de pensar en esto, que va muy ligada a la primera conjetura que planteábamos y se aprovecha de la representación que hacemos de las funciones, trata acerca del número de apariciones de cada literal en una forma normal disyuntiva. Al hablar de la segunda conjetura, vimos un ejemplo de función que no tenía ningún tipo de sesgo. En ese caso se puede apreciar que todos los literales aparecen el mismo número de veces. Se podría pensar que dicha fórmula es muy homogénea.

Sin embargo, partiendo de nuestra primera conjetura, si esperamos que cualquier función en forma normal disyuntiva resultante de un circuito pequeño reutilice mucha información, es también fácil imaginar que los literales involucrados en dicha información aparecerán más veces que otros, dando lugar a formas normales disyuntivas poco homogéneas y con cierta disparidad en el número de apariciones de cada variable. Formalizamos esta idea con la siguiente medida.

Definición 4.3.2. Sea $f : \mathcal{G}_n \rightarrow \{0, 1\}$ una función de grafos expresada en forma normal disyuntiva como $f = \bigvee_{i=1}^m f_i$. Para cada variable $x_{u,v}$, con $u < v$, definimos $\nu_{u,v} = |\{i \in \{1, \dots, m\} : x_{u,v} \in f_i \text{ o } \overline{x_{u,v}} \in f_i\}|$, es decir, el número de apariciones del literal $x_{u,v}$ en la forma normal disyuntiva, sin importar si está negado. Sea $\nu = \left(\nu_{1,2}, \nu_{1,3}, \dots, \nu_{\binom{n}{2}-1, \binom{n}{2}}\right)$. Definimos la **medida de homogeneidad** η_h de f como

$$\eta_h(f) = \text{Var}(\nu),$$

es decir, la varianza en el número de apariciones de cada literal en f .

Una definición equivalente: sea $\mu = \frac{1}{\binom{n}{2}} \sum_{1 \leq i < j \leq \binom{n}{2}} \nu_{i,j}$ la media del número de apariciones de cada literal. Entonces

$$\eta_h(f) = \frac{1}{\binom{n}{2}} \sum_{1 \leq i < j \leq \binom{n}{2}} (\nu_{i,j} - \mu)^2.$$

Veamos, como hicimos con la medida de solapamiento, algunas propiedades acerca de esta medida.

Proposición 4.3.3. *Sea $f : \mathcal{G}_n \rightarrow \{0, 1\}$ una función de grafos expresada en forma normal disyuntiva como $f = \bigvee_{i=1}^m f_i$, y sea $T = \sum_{i=1}^m |f_i|$ la suma de las longitudes de las cláusulas. Se cumple que*

$$0 \leq \eta_h(f) \leq T^2 \frac{\binom{n}{2} - 1}{\binom{n}{2}^2}.$$

Demostración. Que $0 \leq \eta_h(f)$ es evidente, por ser una varianza. Lo interesante es la cota superior.

Para empezar, observamos que sumar la longitud de las cláusulas es lo mismo que sumar el número de apariciones de todos los literales. Así, podemos expresar $T = \sum_{1 \leq i < j \leq \binom{n}{2}} \nu_{i,j}$, con lo que la media del número de apariciones también se puede expresar como $\mu = \frac{T}{\binom{n}{2}}$.

Ahora, desarrollando la fórmula de $\eta_h(f)$,

$$\begin{aligned} \eta_h(f) &= Var(\nu) = \frac{1}{\binom{n}{2}} \cdot \sum_{1 \leq i < j \leq \binom{n}{2}} (\nu_{i,j} - \mu)^2 \\ &= \frac{1}{\binom{n}{2}} \left(\sum_{1 \leq i < j \leq \binom{n}{2}} \nu_{i,j}^2 - 2\mu \cdot \sum_{1 \leq i < j \leq \binom{n}{2}} \nu_{i,j} + \binom{n}{2} \mu^2 \right) \\ &= \frac{1}{\binom{n}{2}} \cdot \sum_{1 \leq i < j \leq \binom{n}{2}} \nu_{i,j}^2 - 2\mu^2 + \mu^2 \\ &= \frac{1}{\binom{n}{2}} \cdot \sum_{1 \leq i < j \leq \binom{n}{2}} \nu_{i,j}^2 - \mu^2. \end{aligned}$$

En vista de este desarrollo, maximizar $\eta_h(f)$ o $\sum_{1 \leq i < j \leq \binom{n}{2}} \nu_{i,j}^2$ son cuestiones equivalentes, pues μ^2 no cambia según como se distribuyan los $\nu_{u,v}$, ya que la suma de todos ellos seguirá siendo T en cualquier caso. Lo que queremos demostrar es que la máxima disparidad/varianza se alcanza cuando una única variable aparece muchas veces y las demás muy pocas. Como caso límite, podemos imaginar que solo aparezca una variable. Es decir, que una variable aparezca T veces y el resto aparezcan 0 veces.

De forma más general, tenemos el siguiente problema de optimización:

$$\begin{aligned} &\text{máx} \sum_{i=1}^M x_i^2 \\ &\text{sujeto a } \sum_{i=1}^M x_i = T \\ &x_i \geq 0, \forall i \in \{1, \dots, M\} \end{aligned}$$

Afortunadamente, no hará falta usar herramientas de programación no lineal para resolver este problema. Lo que sí nos viene bien es que, con este planteamiento, vemos que el conjunto de los valores que satisfacen las restricciones es compacto, y la función a maximizar $f(x_1, \dots, x_M) = x_1^2 + \dots + x_M^2$ es continua, por lo que efectivamente existe máximo.

Ahora queremos ver que, como mencionamos previamente, el máximo de esta suma de cuadrados se alcanzará cuando una de las variables vale T y el resto 0.

Supongamos que este no es el caso. Entonces existe otro punto $x = (x_1, \dots, x_M) \in \mathbb{R}^M$ tal que al menos dos de los valores son estrictamente positivos y en el cual se alcanza el valor máximo de f . Denotemos como x_j y x_k dichos valores. Tomemos ahora un nuevo punto x' tal que $x_i = x'_i$, si $i \neq j, k$, $x'_j = x_j + x_k$ y $x'_k = 0$. Claramente este nuevo punto satisface las restricciones. Por hipótesis, $f(x) \geq f(x')$. Sin embargo,

$$\begin{aligned} f(x') - f(x) &= \sum_{i=1}^M x_i'^2 - \sum_{i=1}^M x_i^2 = x_j'^2 + x_k'^2 - x_j^2 - x_k^2 \\ &= (x_j + x_k)^2 - x_j^2 - x_k^2 = 2x_jx_k > 0, \end{aligned}$$

lo cual contradice la hipótesis. Por tanto, el máximo efectivamente se alcanza cuando una de las variables vale T y el resto 0. Así, el máximo es T^2 .

Finalmente, volviendo a nuestro problema original, deducimos que

$$\begin{aligned} \eta_h(f) &= \frac{1}{\binom{n}{2}} \cdot \sum_{1 \leq i < j \leq \binom{n}{2}} \nu_{i,j}^2 - \mu^2 \leq \frac{T^2}{\binom{n}{2}} - \mu^2 \\ &= \frac{T^2}{\binom{n}{2}} - \left(\frac{T}{\binom{n}{2}} \right)^2 = T^2 \frac{\binom{n}{2} - 1}{\binom{n}{2}^2}. \end{aligned}$$

□

Proposición 4.3.4. *Dados n y k (tamaño de grafo y de clique, respectivamente), sea f_{clique} la función que computa $\text{Clique}_{n,k}$ en forma normal disyuntiva, es decir,*

$$f_{\text{clique}} = \bigvee_{\substack{X \subseteq \{1, \dots, n\} \\ |X|=k}} \bigwedge_{\substack{i, j \in X \\ i < j}} x_{i,j}.$$

El valor de la medida de homogeneidad de f_{clique} es

$$\eta_h(f_{\text{clique}}) = 0.$$

Demostración. El literal $x_{u,v}$ representa la arista que se apoya en los vértices u y v . Fijados estos dos vértices, hace falta elegir $k-2$ vértices para formar un clique, por lo que esta arista pertenece a $\binom{n-2}{k-2}$ cliques. Por tanto, $x_{u,v}$ aparece en esa misma cantidad de cláusulas de f_{clique} .

Esta cantidad de apariciones es la misma para todos los literales, por lo que la varianza del número de apariciones es 0, como queríamos demostrar. □

Al igual que hicimos con la medida de solapamiento, mostramos a continuación un pseudocódigo muy sencillo que ilustra cómo calcular esta medida:

Algoritmo 3 Cálculo de $\eta_h(f)$.

Entrada: f expresada como lista de cláusulas.

Salida: $\eta_h(f)$.

```

1:  $A \leftarrow \binom{n}{2}$ 
2:  $veces \leftarrow [0, \dots, 0]$  de tamaño  $A$ 
3: for cláusula  $c \in f$  do
4:   for literal  $\ell \in c$  do
5:      $veces[|\ell|] \leftarrow veces[|\ell|] + 1$ 
6:   end for
7: end for
8:  $\mu \leftarrow \frac{1}{A} \sum_{i=1}^A veces[i]$ 
9:  $varianza \leftarrow \frac{1}{A} \sum_{i=1}^A (veces[i] - \mu)^2$ 
10: return  $varianza$ 

```

En vista de este pseudocódigo, si denotamos de nuevo por T la suma de las longitudes de las cláusulas, esta función se calcula en complejidad $O(T)$. Para más detalles de implementación, se puede consultar el código en [8].

4.3.3. Medición de grados de libertad

La anterior medida parece ser útil para diferenciar funciones que pueden computarse con circuitos pequeños de las que no parecen cumplir lo mismo. Sin embargo, nos gustaría encontrar una medida más orientada al problema de $Clique_{n,k}$. Aunque definimos la medida en el contexto de funciones de grafos, esta medida es claramente extrapolable a funciones booleanas con cualquier tipo de input. Vamos entonces a elaborar una medida más orientada al problema que tenemos entre manos, siguiendo en la línea de las conjeturas previamente enunciadas.

En la segunda conjetura mencionamos que, intuitivamente, f_{clique} no distingue entre cliques. Todos aportan la misma información, ninguno es más importante que los demás. Entonces, al tratar con funciones en general, ¿cómo podemos manejar la idea de “información aportada por un clique”?

Un recurso que pensamos que podría ser útil es trabajar con ideas similares a las que se usan en el estudio teórico del capítulo 3. En ese estudio se utilizan conceptos como $\mathbb{T}^{f,G}$, las restricciones de funciones y los inputs sensibles, y se nos ocurrió que podríamos extrapolar estos recursos del campo teórico al experimental. Además, la lectura del trabajo de fin de grado de Enrique Carro Garrido y José Ramón Cobián Fernández [5] fue muy útil para pensar en formas de definir la idea de “aportar información”. En este trabajo definieron conceptos muy interesantes relacionados con las restricciones de funciones, como el concepto de ecuanimidad que mencionamos previamente, aunque no podremos hacer uso de ellos porque los costes computacionales en nuestro caso de estudio serían inabarcables.

Para introducir la siguiente idea, veamos un ejemplo antes:

Ejemplo 4.3.1. Sea $f = x_1x_7 \vee x_2x_8 \vee x_3x_9 \vee x_4 \vee x_5 \vee x_6$.

Tomemos las tuplas $X = (x_1, x_2, x_3)$ e $Y = (x_4, x_5, x_6)$. Cada una de estas tuplas contiene tres variables de f . Dado que a cada variable se puede asignar valor 0 o 1, cada una de estas tuplas puede tener $2^3 = 8$ valores distintos. Ahora, si fijamos el valor de una de estas tuplas y restringimos la función a dichos valores, la restricción quedará en función del resto de variables.

Por ejemplo, si tomamos $X = (0, 0, 0)$, y restringimos f nos queda una nueva función

$$f|_{X=(0,0,0)} = (0 \wedge x_7) \vee (0 \wedge x_8) \vee (0 \wedge x_9) \vee x_4 \vee x_5 \vee x_6 = x_4 \vee x_5 \vee x_6.$$

Hagamos esto con los 8 valores posibles de X :

- $X = (0, 0, 0) \rightarrow f|_{X=(0,0,0)} = x_4 \vee x_5 \vee x_6.$
- $X = (0, 0, 1) \rightarrow f|_{X=(0,0,1)} = x_9 \vee x_4 \vee x_5 \vee x_6.$
- $X = (0, 1, 0) \rightarrow f|_{X=(0,1,0)} = x_8 \vee x_4 \vee x_5 \vee x_6.$
- $X = (0, 1, 1) \rightarrow f|_{X=(0,1,1)} = x_8 \vee x_9 \vee x_4 \vee x_5 \vee x_6.$
- $X = (1, 0, 0) \rightarrow f|_{X=(1,0,0)} = x_7 \vee x_4 \vee x_5 \vee x_6.$
- $X = (1, 0, 1) \rightarrow f|_{X=(1,0,1)} = x_7 \vee x_9 \vee x_4 \vee x_5 \vee x_6.$
- $X = (1, 1, 0) \rightarrow f|_{X=(1,1,0)} = x_7 \vee x_8 \vee x_4 \vee x_5 \vee x_6.$
- $X = (1, 1, 1) \rightarrow f|_{X=(1,1,1)} = x_7 \vee x_8 \vee x_9 \vee x_4 \vee x_5 \vee x_6.$

Vemos que todas las restricciones de f a los distintos valores de X son distintas. Esto, por supuesto, no tiene por qué ser siempre el caso. Veamos lo que ocurre con los 8 posibles valores de Y :

- $Y = (0, 0, 0) \rightarrow f|_{Y=(0,0,0)} = x_1x_7 \vee x_2x_8 \vee x_3x_9.$
- $Y = (0, 0, 1) \rightarrow f|_{Y=(0,0,1)} = x_1x_7 \vee x_2x_8 \vee x_3x_9 \vee 0 \vee 0 \vee 1 = \top.$
- $Y = (0, 1, 0) \rightarrow f|_{Y=(0,1,0)} = \top.$
- $Y = (0, 1, 1) \rightarrow f|_{Y=(0,1,1)} = \top.$
- $Y = (1, 0, 0) \rightarrow f|_{Y=(1,0,0)} = \top.$
- $Y = (1, 0, 1) \rightarrow f|_{Y=(1,0,1)} = \top.$
- $Y = (1, 1, 0) \rightarrow f|_{Y=(1,1,0)} = \top.$
- $Y = (1, 1, 1) \rightarrow f|_{Y=(1,1,1)} = \top.$

En este caso, a diferencia del anterior, nos encontramos con que los 8 valores de Y solo han dado lugar a dos restricciones distintas.

Una forma de interpretar esto es como “grados de libertad”. La tupla X aporta 8 grados de libertad, pues según la información que tenemos de él nos lleva a 8 posibles funciones, “8 posibles situaciones”. Sin embargo, Y solo nos ha proporcionado 2 grados de libertad. Tener información sobre este conjunto ha acotado mucho las posibles situaciones en las que nos podemos hallar.

Ahora, como ya introducimos anteriormente, f_{clique} no debería hacer distinción entre cliques. Todos importan lo mismo, así que todos deberían aportarnos la misma cantidad de información o, como hemos mencionado en el ejemplo, la misma cantidad de grados de libertad. Así, definamos con detalle esta idea.

Definición 4.3.3. Sea $f : \{0, 1\}^m \rightarrow \{0, 1\}$ una función booleana. Sea X una tupla de variables cualquiera de tamaño v . Denotamos por $\delta(X)$ los **grados de libertad de X en f** , definido como

$$\delta(X) = |\text{res}(X)|,$$

donde $\text{res}(X) = \{f \upharpoonright_{X=(s_1, \dots, s_v)} : s_i \in \{0, 1\}, \forall i \in \{1, \dots, v\}\}$.

Definición 4.3.4. Sea f una función de grafos de tamaño n , y sea $k \leq n$. Denotamos por $\eta_\delta(f)$ la **medida de grados de libertad sobre cliques** de f , definida como la media de los grados de libertad de cada clique en f .

Más formalmente, dado $C \subseteq \{1, \dots, n\}$ un subconjunto de tamaño k (los vértices de un clique de tamaño k), sea $\text{ar}(C)$ una tupla con las $\binom{k}{2}$ variables que representan las aristas apoyadas en los vértices de C . Con esta notación,

$$\eta_\delta(f) = \frac{1}{\binom{n}{k}} \sum_{\substack{C \subseteq \{1, \dots, n\} \\ |C|=k}} \delta(\text{ar}(C)).$$

Observación 4.3.1. En la definición de $\eta_\delta(f)$, dado cualquier conjunto C , el orden de las variables en $\text{ar}(C)$ es irrelevante.

En [5], aparte de la medida de ecuanimidad mencionada previamente, se definió la medida de enredo, que a grandes rasgos trata de buscar la mejor partición de los inputs en dos mitades, de forma que el conocimiento de los valores de una mitad limite mucho la información necesaria acerca de la otra mitad.

En nuestro caso, buscar una partición en mitades no pareció que fuera a ser la mejor opción. $\text{Clique}_{n,k}$ es un problema en el que parece muy difícil hacer una aproximación de divide y vencerás, pues los cliques se mezclan mucho entre sí (dos cliques pueden compartir $k - 1$ vértices y ser distintos, situación en la que compartirían $\binom{k-1}{2}$ variables). Sin embargo, esta medida que hemos definido se puede ver como una aproximación al enredo en el que las particiones a usar son los propios cliques. Por tanto, se espera que, dado un clique, cada uno de sus estados nos conduzca a una función distinta, teniendo así la cantidad máxima de grados de libertad.

A continuación vemos un par de resultados que demuestran que este es el caso para Half-Clique_n .

Proposición 4.3.5. *Sea f una función de grafos de tamaño n , y sea $k \leq n$. Se cumple que*

$$\eta_\delta(f) \in \left[1, 2^{\binom{k}{2}}\right].$$

Demostración. Para cualquier subconjunto $C \subseteq \{1, \dots, n\}$ de tamaño k , $\text{res}(\text{ar}(C))$ es un conjunto no vacío. Así, $|\delta(\text{ar}(C))| \geq 1$, por lo que

$$\eta_\delta(f) = \frac{1}{\binom{n}{k}} \sum_{\substack{C \subseteq \{1, \dots, n\} \\ |C|=k}} \delta(\text{ar}(C)) \geq \frac{1}{\binom{n}{k}} \sum_{\substack{C \subseteq \{1, \dots, n\} \\ |C|=k}} 1 = \frac{1}{\binom{n}{k}} \cdot \binom{n}{k} = 1.$$

Por otro lado, el caso en el que $\delta(\text{ar}(C))$ es máximo se obtiene cuando todos los posibles valores de $\text{ar}(C)$ dan lugar a distintas restricciones. Como C tiene tamaño k , $\text{ar}(C)$ está formado por $\binom{k}{2}$ variables, por lo que este número máximo es $2^{\binom{k}{2}}$. Así

$$\eta_\delta(f) = \frac{1}{\binom{n}{k}} \sum_{\substack{C \subseteq \{1, \dots, n\} \\ |C|=k}} \delta(\text{ar}(C)) \leq \frac{1}{\binom{n}{k}} \sum_{\substack{C \subseteq \{1, \dots, n\} \\ |C|=k}} 2^{\binom{k}{2}} = \frac{1}{\binom{n}{k}} \cdot \binom{n}{k} 2^{\binom{k}{2}} = 2^{\binom{k}{2}}.$$

□

Proposición 4.3.6. *Sea f_{clique} la función que computa Half-Clique_n ($k = \lfloor \frac{n}{2} \rfloor$). Se cumple que*

$$\eta_\delta(f_{\text{clique}}) = 2^{\binom{k}{2}}.$$

Demostración. Sea C un conjunto de k vértices cualquiera. Queremos demostrar que $\delta(\text{ar}(C)) = 2^{\binom{k}{2}}$, de lo cual se deduce directamente el resultado final.

Así, sean s_1 y s_2 dos posibles valores distintos de $\text{ar}(C)$. Sean $f_1 = f_{\text{clique}} \upharpoonright_{\text{ar}(C)=s_1}$ y $f_2 = f_{\text{clique}} \upharpoonright_{\text{ar}(C)=s_2}$. Veamos que $f_1 \neq f_2$:

Como $s_1 \neq s_2$, sin pérdida de generalidad podemos suponer que hay cierta variable de $\text{ar}(C)$, $x_{u,v}$, que toma el valor 1 con s_1 y 0 con s_2 . Dado que $k = \lfloor \frac{n}{2} \rfloor$, existen $\lfloor \frac{n}{2} \rfloor$ vértices que no son parte de C . Por tanto, podemos tomar $k-2$ de estos vértices, a los que llamaremos $v_1, \dots, v_{k-2}$. De esta forma, $u, v, v_1, \dots, v_{k-2}$ forman un conjunto de k vértices, dos de los cuales son de C . Llamemos a este conjunto C' .

Ahora, la única arista que se apoya en un par de estos vértices y que forma parte de $\text{ar}(C)$ es $\{u, v\}$. Todas las demás aristas tienen un extremo fuera de C , por lo que no son parte de $\text{ar}(C)$. Como C' tiene tamaño k , hay una cláusula c de la forma normal disyuntiva de f_{clique} que está precisamente formada por la conjunción de las variables de $\text{ar}(C')$.

Ahora, al restringir f_{clique} con $\text{ar}(C) = s_1$, obteniendo f_1 , puesto que $x_{u,v}$ vale 1 en s_1 , la cláusula c se restringe a una cláusula c_1 en la que aparecen todos los literales de c excepto $x_{u,v}$. En cambio, al restringir f_{clique} con $\text{ar}(C) = s_2$ y obtener f_2 la cláusula c desaparece, pues el valor de $x_{u,v}$ es 0 y la conjunción da $\perp$.

Cada cláusula de f_{clique} hace referencia a un conjunto de k vértices, y es la única cláusula que refiere a dicho conjunto de vértices. Por tanto, c_1 es la única cláusula de f_1 que hace referencia únicamente a vértices de C' , mientras que ninguna cláusula de f_2 hace referencia a estos vértices. Como f_1 y f_2 son funciones cuyo dominio

es $ar(\{1, \dots, n\}) \setminus ar(C)$ (las aristas que no están apoyadas en dos vértices de C), tomemos la entrada x tal que la variable $x_{i,j}$ vale 1 si i y j son parte de C' , y 0 en caso contrario. Estos valores hacen que c_1 se evalúe a $\top$, por lo que $f_1(x) = 1$. Sin embargo, cada cláusula de f_2 contiene alguna variable cuyo valor es 0, por lo que $f_2(x) = 0$. Por tanto, $f_1 \neq f_2$.

Así, cada valor de $ar(C)$ da lugar a una restricción distinta, por lo que $\delta(ar(C)) = 2^{\binom{k}{2}}$. Aplicando esto a cualquier C , obtenemos el resultado deseado. $\square$

En vista de este resultado, tomando f_{clique} la función que computa *Half-Clique*₈ ($n = 8$ y $k = 4$) se tiene que

$$\eta_\delta(f_{clique}) = 64.$$

Proposición 4.3.7. *Sea $n \geq 2$ y $k = \lfloor \frac{n}{2} \rfloor$ (para situarnos en el contexto de *Half-Clique*_n). Dada $f = x_{u,v}$ una función formada por un único literal, se tiene que*

$$\eta_\delta(f) = 1 + \frac{k(k-1)}{n(n-1)}.$$

Demostración. La variable $x_{u,v}$ hace referencia a una arista apoyada en dos vértices. Por tanto, esta arista forma parte de $\binom{n-2}{k-2}$ cliques. Así, sea C un conjunto de k vértices:

- Si C contiene a u y v , entonces los distintos valores posibles de $ar(C)$ darán valor 0 o 1 a $x_{u,v}$, por lo que $f|_{ar(C)=s} = \top$ o $f|_{ar(C)=s} = \perp$, para cualquier s . Así, $\delta(ar(C)) = 2$.
- Si C es uno de los $\binom{n}{k} - \binom{n-2}{k-2}$ conjuntos de k vértices que no contiene a u o v , entonces $f|_{ar(C)=s} = f$, para cualquier s , pues $x_{u,v}$ no es parte de $ar(C)$. Por tanto, $\delta(ar(C)) = 1$.

Con todo esto,

$$\begin{aligned} \eta_\delta(f) &= \frac{1}{\binom{n}{k}} \left(\binom{n}{k} - \binom{n-2}{k-2} + 2 \binom{n-2}{k-2} \right) = 1 + \frac{\binom{n-2}{k-2}}{\binom{n}{k}} \\ &= 1 + \frac{\frac{(n-2)!}{(k-2)!(n-k)!}}{\frac{n!}{k!(n-k)!}} = 1 + \frac{k(k-1)}{n(n-1)}. \end{aligned}$$

$\square$

Observación 4.3.2. $\eta_\delta(\top) = \eta_\delta(\perp) = 1$, pues ninguna restricción modifica estas funciones.

En vista de estos resultados, fijémonos en lo que tenemos:

- η_δ asigna un valor muy pequeño (< 2) a las funciones pequeñas ($\top$, $\perp$ y los inputs).
- η_δ asigna el valor máximo a la función que computa *Half-Clique*_n.

- El valor máximo de η_δ es exponencial con respecto a n : $2^{\left(\lceil \frac{n}{2} \rceil\right)}$.

Por tanto, podemos hablar de esta función como una *métrica* en el sentido de Razborov (como se hizo en [12] en el estudio de las notas [2]). Debido a ello, vamos a dejar de hablar de η_δ como medida, y usaremos el nombre de **métrica de grados de libertad sobre cliques**, denotada con el símbolo μ_δ para asemejarlo a la métrica sintáctica. Cabe mencionar que, a diferencia de la métrica sintáctica, esta toma valores no enteros.

Ahora, como hicimos anteriormente con las medidas, nos viene bien entender cómo calcular esta métrica de cara a los experimentos. Lo primero que necesitamos es una forma de calcular restricciones de funciones. Esto se ilustra con el siguiente pseudocódigo, que se explica posteriormente.

Algoritmo 4 Restricción de una FND fijando algunos literales.

Entrada: f expresada como lista de cláusulas, **valores** como diccionario de variables a valores.

Salida: FND resultante tras fijar los valores indicados.

```

1: restriccion  $\leftarrow []$ 
2: for cláusula  $c \in f$  do
3:   nueva_c  $\leftarrow []$ 
4:   for literal  $\ell \in c$  do
5:     if  $|\ell| \in \text{valores}$  then
6:       if  $(l > 0 \text{ y } \text{valores}[|\ell|] = 1) \text{ o } (l < 0 \text{ y } \text{valores}[|\ell|] = 0)$  then
7:         continuar con el siguiente literal
8:       else
9:         nueva_c  $\leftarrow [\text{falso}]$ 
10:        salir del bucle de literales
11:      end if
12:    else
13:      nueva_c  $\leftarrow \text{nueva\_c} \cup \ell$ 
14:    end if
15:  end for
16:  if nueva_c =  $[\text{falso}]$  then
17:    continuar con la siguiente cláusula
18:  else if nueva_c está vacía then
19:    return  $[\text{verdadero}]$ 
20:  else
21:    restriccion  $\leftarrow \text{restriccion} \cup \{\text{nueva\_c}\}$ 
22:  end if
23: end for
24: if restriccion está vacía then
25:  return  $[[\text{falso}]]$ 
26: else
27:  return reducir(nueva)
28: end if

```

El cálculo anterior es iterativo. Dada una cláusula C y una de sus variables x ,

veamos lo que ocurre según la cláusula y el valor que se asigna a x .

- Si no se da valor a x , la cláusula permanece inalterada.
- Si $x = 0$ y la variable no aparece negada, la conjunción se vuelve falsa, así que C desaparece. Lo mismo ocurre si $x = 1$ y la variable sí que aparece negada.
- Por lo contrario, si $x = 1$ y la variable no aparece negada (o $x = 0$ y sí que aparece negada), entonces C pasa a ser una nueva cláusula C' en la que aparecen todos los literales menos x . Si x era el único literal, entonces $C' = \top$ y la función entera se convierte en $\top$.
- Si tras hacer estas aplicaciones con todas las variables de la función desaparecen todas las cláusulas, entonces es que todas se han vuelto falsas, por lo que la función se convierte en $\perp$.

Dado que la restricción se calcula recorriendo de forma iterativa las cláusulas, y recorriendo a su vez de forma iterativa los literales de cada cláusula, si denotamos por T a la suma de las longitudes de las cláusulas tenemos que esta función se computa con complejidad $O(T)$, a falta de considerar el coste de la reducción del final.

Con esto implementado, calcular la métrica de grados de libertad es bastante sencillo, como se ve en el siguiente pseudocódigo.

Algoritmo 5 Cálculo de $\mu_\delta(f)$.

Entrada: f expresada como lista de cláusulas.
Salida: $\mu_\delta(f)$

```

1:  $grados \leftarrow []$ 
2: for  $C$  clique de tamaño  $k$  do
3:    $aristas \leftarrow []$ 
4:   for  $i \in C$  do
5:     for  $j \in C, j > i$  do
6:        $aristas \leftarrow aristas \cup \{i, j\}$ 
7:     end for
8:   end for
9:    $restricciones \leftarrow \emptyset$ 
10:  for  $v \in \{0, 1\}^{\binom{k}{2}}$  do
11:     $asignaciones \leftarrow$  diccionario vacío
12:    for  $i = 1.. \binom{k}{2}$  do
13:       $asignaciones[aristas[i]] \leftarrow v[i]$ 
14:    end for
15:     $f\_res \leftarrow restringe\_fnd(f, asignaciones)$ 
16:     $restricciones \leftarrow restricciones \cup \{f\_res\}$ 
17:  end for
18:   $grados \leftarrow grados \cup \{|restricciones|\}$ 
19: end for
20: return  $\frac{1}{\binom{n}{k}} \sum_{i=1}^{\binom{n}{k}} grados[i]$ 
```

Como en el grafo puede haber $\binom{n}{k}$ cliques, cada uno formado por $\binom{k}{2}$ aristas, el coste de este algoritmo es $O(\binom{n}{k}2^{\binom{k}{2}}T)$, a falta de considerar nuevamente el coste de las reducciones. Para ver los detalles concretos de la implementación, se puede consultar todo en [8].

4.4. Experimentos con las medidas

En esta sección, vamos a hacer uso de las medidas que definimos en la sección anterior. Nuestro objetivo con estas medidas es ser capaces de diferenciar claramente las funciones simuladas (las obtenidas como resultado de simular *Half-Clique₈*) del resto de funciones. Así, vamos a exponer gráficas comparando valores de las distintas medidas para tres tipos de funciones: funciones simuladas, pseudoaleatorias y aleatorias.

De primera mano, imaginamos que diferenciar las funciones simuladas de las pseudoaleatorias sería relativamente sencillo. No obstante, nuestras suposiciones iniciales iban en la línea de que diferenciar las funciones simuladas de funciones completamente aleatorias resultaría algo más complicado. Así, interesa ver si estas medidas, pos sí solas o de forma conjunta, resultan útiles para este fin.

4.4.1. Resultados con la medida de solapamiento

Empecemos con la medida de solapamiento. En la siguiente gráfica vamos a ver puntos de la forma $(\mu_x(f), \eta_s(f))$, en azul para las funciones pseudoaleatorias, en verde para las aleatorias y en rojo para las simuladas, para comparar los valores de solapamiento que obtienen los tres tipos de funciones con respecto a sus valores de la métrica sintáctica.

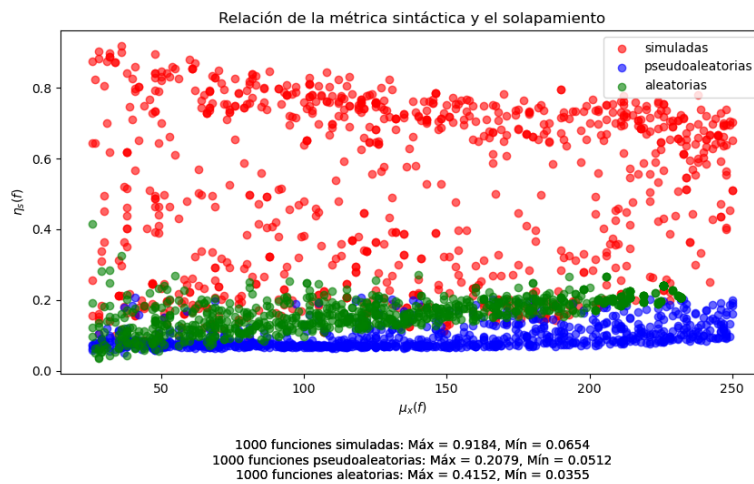


Figura 4.7: Solapamiento de funciones pseudoaleatorias, aleatorias y simuladas

En esta gráfica, para empezar, vemos que los tres tipos de funciones parecen estar segregados en tres niveles. Las funciones pseudoaleatorias tienen valores bajos de

solapamiento, las funciones aleatorias tienen valores más intermedios, y las funciones simuladas alcanzan valores bastante mayores. Además, como era de esperar, las funciones pseudoaleatorias tienen valores cercanos (y generalmente inferiores) a 0,2, que vimos previamente que sería nuestro valor de referencia, por ser el valor de solapamiento de f_{clique} . Si quitamos las funciones aleatorias de la ecuación, nos queda esta gráfica:

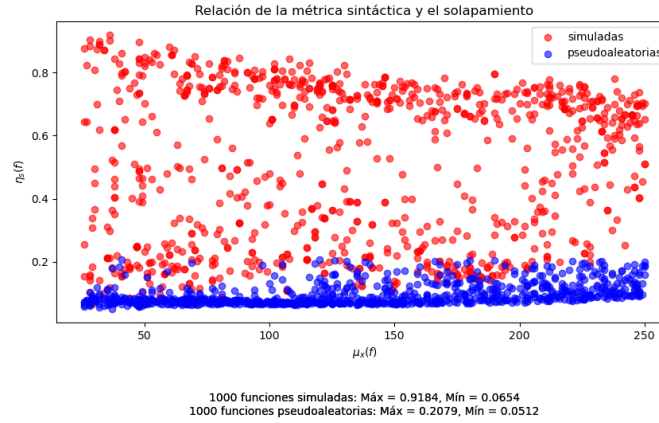


Figura 4.8: Solapamiento de funciones pseudoaleatorias y simuladas

Vemos en esta gráfica una clara diferencia entre los valores obtenidos por ambos tipos de funciones. Ya que la diferenciación que se produce es clara, independientemente del valor de μ_x , podemos emplear otra representación en forma de histograma, para determinar cuántas funciones de cada tipo obtenemos para cada rango de valores de la medida de solapamiento, sin tener en cuenta los valores de la métrica sintáctica. Véase esto en la siguiente figura:

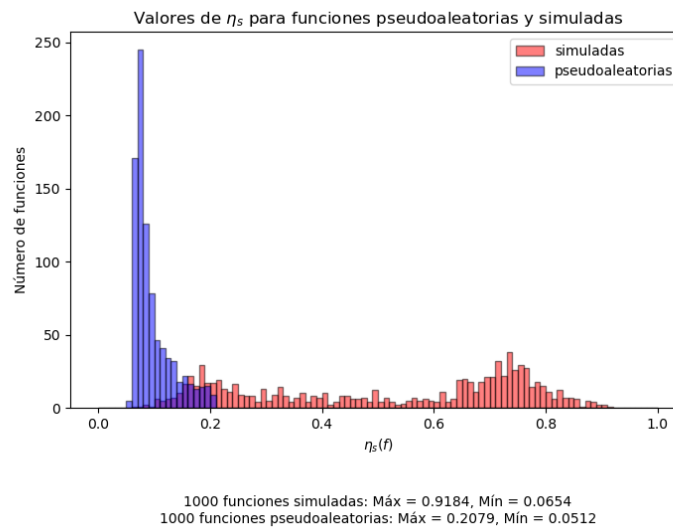


Figura 4.9: Histograma de solapamiento de funciones pseudoaleatorias y simuladas

En vista de esta gráfica, parece que el umbral de 0,2 es casi determinante a la hora de saber si una función es pseudoaleatoria o simulada. De hecho, si hacemos la predicción de “una función es simulada si su solapamiento es mayor a 0,2, y pseudoaleatoria en caso contrario”, obtenemos la siguiente matriz de confusión:

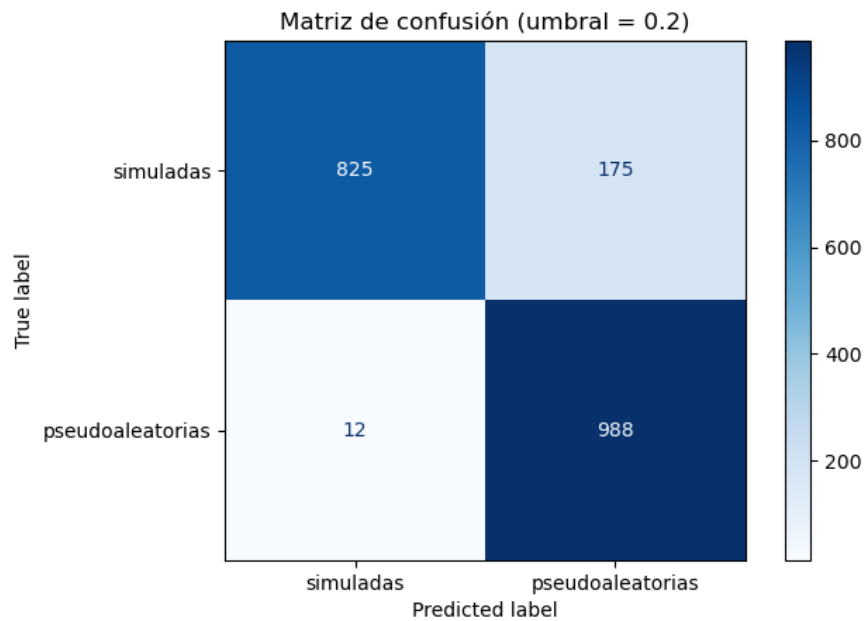


Figura 4.10: Matriz de confusión del umbral de solapamiento

En vista de esta matriz, la predicción tiene una precisión del 90,65 %. Casi la totalidad de funciones pseudoaleatorias tienen valores de solapamiento menores a 0,2, así que quedan bien predichas. Las simuladas sin embargo fallan más, confundándose aproximadamente una de cada seis.

Por supuesto, esta precisión es menor si incluimos en la ecuación las funciones aleatorias, cambiando la predicción a “simulada o no simulada”. No obstante, la precisión obtenida nos indica cierta segregación a través de esta medida. No esperamos que esta medida por sí sola sea suficiente para diferenciar las funciones simuladas del resto, pero nos sirve como punto de partida.

Ahora, ¿cómo evoluciona el valor de esta medida al mezclar funciones a través de puertas AND y OR? Esto se ilustra en la siguiente gráfica. Esta se ha generado tomando parejas de funciones simuladas f y g , y se han graficado en azul los puntos $(\eta_s(f), \eta_s(f \vee g))$ y $(\eta_s(g), \eta_s(f \vee g))$, y en rojo los puntos $(\eta_s(f), \eta_s(f \wedge g))$ y $(\eta_s(g), \eta_s(f \wedge g))$.

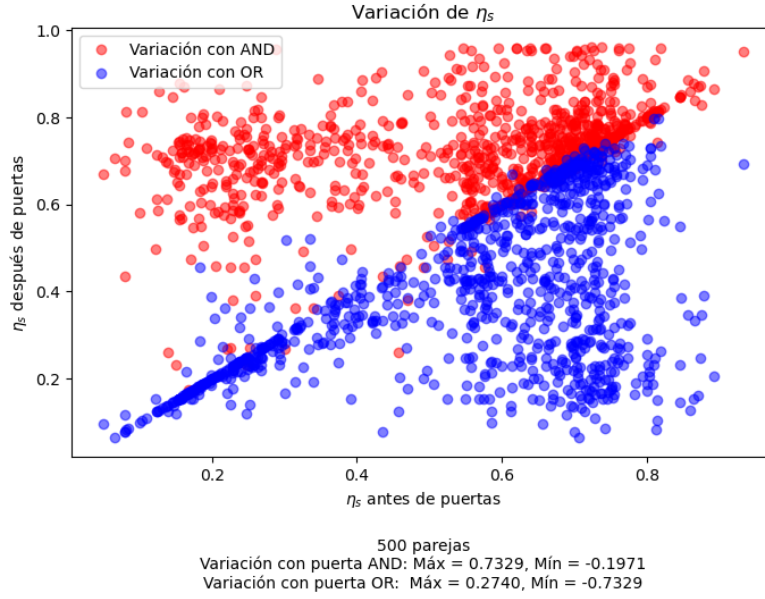


Figura 4.11: Variación de solapamiento de funciones simuladas

Lo que esta gráfica ilustra nos parece de gran interés. Los puntos están divididos muy fuertemente en dos sectores. Casi todos los resultantes de la puerta AND quedan por encima de la diagonal principal (la recta $x = y$), mientras que los de la puerta OR quedan por debajo. Esto significa que, al juntar funciones con la puerta AND, el solapamiento tiende a aumentar, mientras que con la puerta OR tiende a disminuir.

Esto, por un lado, va en la línea de nuestras conjeturas iniciales, pues sospechábamos que al tener fragmentos de funciones en forma normal disyuntiva que se repitieran mucho, el crecimiento multiplicativo de la puerta AND lo único que haría es acentuar esta repetición.

Por otro lado, y aún más interesante, nos indica que, quizá, la puerta AND no es tan útil como desearíamos que fuera. Recordemos que $\eta_s(f_{clique}) \approx 0,2$. Nuestra única esperanza para computar $Half-Clique_8$ con circuitos pequeños residía en el comportamiento de las puertas AND. No obstante, vemos que estas puertas, al contrario que con μ_x , parece que nos alejan cada vez más de obtener el valor de $\eta_s(f_{clique})$. Al contrario, la puerta OR no resultaba suficientemente productiva para μ_x , pero vemos que es precisamente la que nos acerca al valor de $\eta_s(f_{clique})$. Así, parece que mejorar una medida hace que empeore la otra, y esto es problemático, pues alcanzar el valor de f_{clique} para μ_x y η_s son ambas **condiciones necesarias** para computar $Half-Clique_8$.

4.4.2. Resultados con la medida de homogeneidad

Veamos ahora algunas gráficas análogas a las anteriores para la medida de homogeneidad. En primer lugar, comparemos los valores de homogeneidad para las funciones pseudoaleatorias, aleatorias y simuladas:

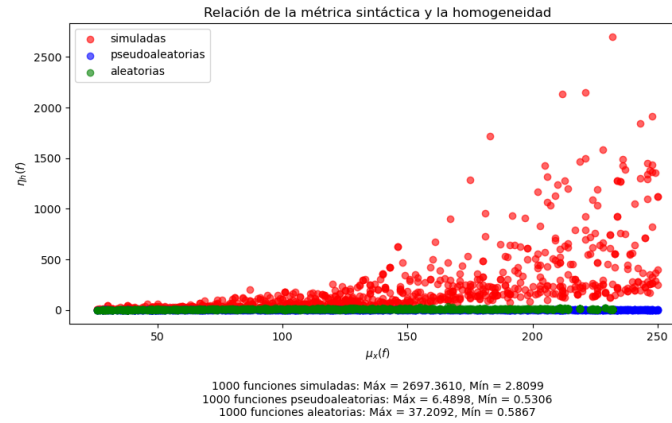


Figura 4.12: Homogeneidad de funciones pseudoaleatorias, aleatorias y simuladas

A primera vista, parece que las funciones simuladas, como sospechábamos que ocurriría, tienen unos valores de η_h muchísimo mayores a los de las funciones pseudoaleatorias. Parece incluso que, a partir de ciertos valores de μ_x , todos los puntos resultantes de funciones simuladas se separan del resto de funciones. Por supuesto, las funciones aleatorias podrían obtener cualquier valor (en particular, podrían generarse funciones simuladas de forma aleatoria), pero en vista de lo densamente concentradas que están las funciones aleatorias en la parte baja de la gráfica, se ve que esto es poco frecuente.

Esto nos resulta bastante útil, pues parece que esta medida, aunque probablemente no sea suficiente por sí sola, sí que puede llegar a ayudar en gran medida a diferenciar las funciones simuladas del resto de funciones booleanas.

La forma en que esta medida varía, sin embargo, parece opuesta a la de la medida de solapamiento. Véase esto en la siguiente gráfica

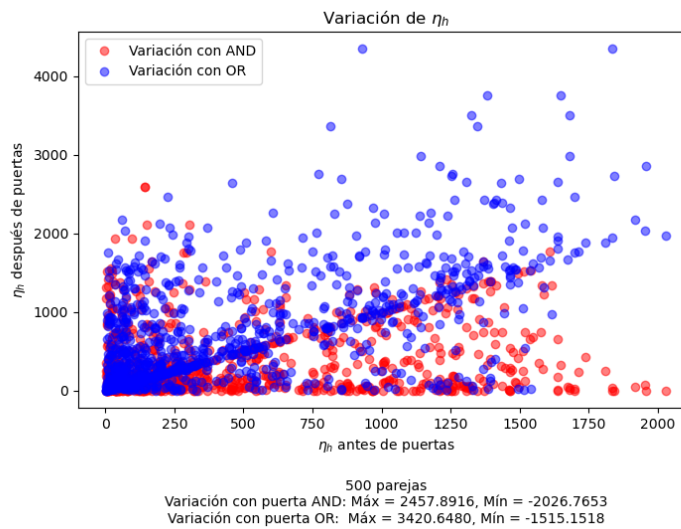


Figura 4.13: Variación de homogeneidad de funciones simuladas

En este caso, aunque las diferencias son menos drásticas que en el caso de la medida de solapamiento, parece que la puerta que más ayuda a disminuir esta medida (acercando los valores a 0, el valor de f_{clique}) es la puerta AND. La puerta OR da lugar a, mayoritariamente, puntos por encima de la diagonal principal. Es decir, la homogeneidad tiende a aumentar con la puerta OR. Por lo contrario, la puerta AND da lugar a más puntos por debajo de la diagonal principal que por encima. En cualquier caso, para valores bajos de homogeneidad, este comportamiento es algo menos claro. Parece entonces que, en el caso de η_h , la puerta AND es particularmente útil para reducir la homogeneidad de funciones con valores muy altos de esta medida. Para funciones con valores más próximos a 0, sin embargo, la utilidad no parece tan clara.

No obstante, incluso si imaginamos que la puerta AND sí resulte útil para valores bajos de esta medida también (a diferencia de la puerta OR, que parece ser menos útil en la mayoría de casos), parece que esto nos sitúa en un callejón sin salida de cara a computar $Half-Clique_s$ con un circuito pequeño: nos interesa llegar a un solapamiento de 0,2 y a una homogeneidad de 0. Para lo primero, en vista de los resultados anteriores, nos conviene más emplear puertas OR, pero para lo segundo, según acabamos de ver, nos conviene más la AND. En ambos casos, además, emplear la puerta contraria parece que nos aleja siempre de nuestro objetivo, similar a lo que pasó antes con μ_x y η_s .

4.4.3. Resultados con la medida de grados de libertad

Vamos a estudiar ahora nuestra última medida/métrica. De nuevo, como hicimos con las anteriores medidas, veamos la gráfica que compara los valores de esta métrica para los tres tipos de funciones, con respecto a la métrica sintáctica.

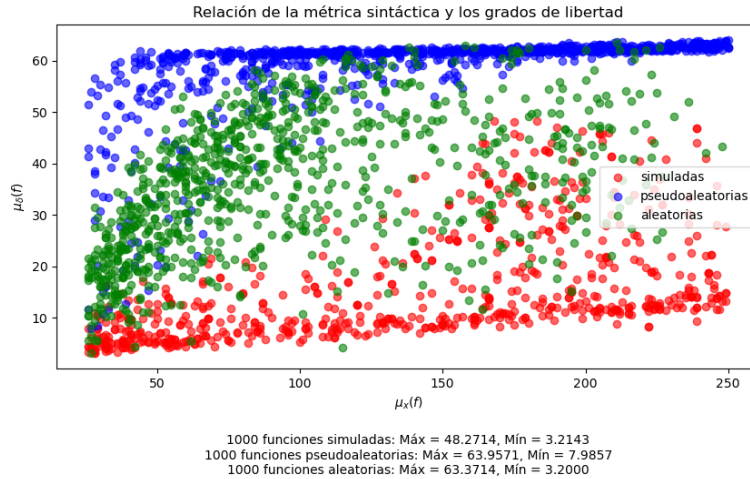


Figura 4.14: Grados de libertad de funciones pseudoaleatorias, aleatorias y simuladas

Una de las primeras cosas que observamos es que las funciones pseudoaleatorias alcanzan valores muy altos de μ_δ a partir de valores muy bajos de μ_x , cosa que

las diferencia claramente de los otros dos tipos de funciones. Además, vemos que las funciones simuladas no son capaces de alcanzar valores tan altos en ningún momento. Vemos que unos cuantos puntos se acercan a valores próximos a 50, aunque inferiores en cualquier caso. Finalmente, las funciones aleatorias parecen ser, una vez más, un punto intermedio entre ambos tipos de funciones. Estas sí alcanzan valores muy altos de μ_δ , aunque en general tienen valores muy dispersos, sobre todo a partir de valores más altos de μ_x , donde se mezclan bastante con las funciones simuladas. De cara a diferenciar funciones simuladas de las demás, esta métrica parece ser particularmente útil para valores bajos de μ_x , al contrario que con η_h . Esto nos da la esperanza de que estas dos últimas medidas funcionen bien de forma conjunta a la hora de intentar diferenciar funciones simuladas del resto de funciones.

Ahora, veamos la evolución de los grados de libertad a través de las puertas AND y OR:

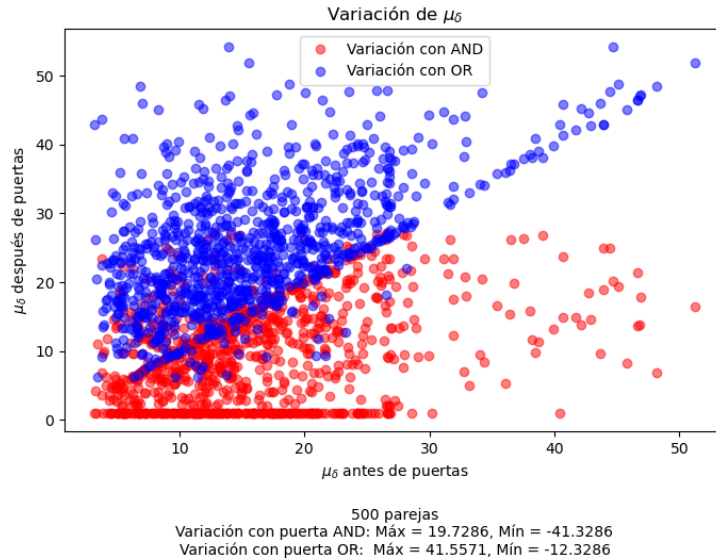


Figura 4.15: Variación de grados de libertad de funciones simuladas

En este caso, las conclusiones parece que son similares a las que sacamos en el caso del solapamiento, lo cual es positivo. Una vez más, la gran mayoría de puntos resultantes de la puerta OR se sitúan por encima de la diagonal principal, mientras que con la puerta AND ocurre lo contrario. Teniendo en cuenta que $\mu_\delta(f_{clique}) = 64$, vemos que las puertas OR nos acercan a este valor, mientras que las puertas AND nos alejan. Así, como ocurrió antes, parece que nos hallamos en una situación comprometida: algunas de las medidas se benefician de la puerta AND, mientras que otras se ven perjudicadas, y lo mismo ocurre con la puerta OR.

En definitiva, parece que es difícil encontrar un balance en el que todas las medidas se vean beneficiadas.

4.5. Predicción de funciones según el valor de las medidas

En vista de los resultados anteriores, parece evidente que las funciones simuladas son inevitablemente distintas a las demás. Ya éramos conscientes de que la cantidad de funciones computables con circuitos pequeños es ínfima con respecto al número total de funciones booleanas, como vimos en el teorema 2.2.3.

Ahora, la esperanza que albergábamos inicialmente era que quizá las medidas desarrolladas en este estudio fueran capaces de diferenciar fuertemente las funciones simuladas del resto de funciones. Más concretamente, la pregunta que nos planteábamos era la siguiente:

¿Son estas medidas capaces de caracterizar el conjunto de las funciones simuladas?

De ser así, un estudio en profundidad acerca de estas medidas podría acercarnos a entender mejor las dificultades que afrontan los circuitos booleanos pequeños a la hora de intentar aproximarse a la función que computa $\text{Clique}_{n,k}$.

Ya vimos en la sección anterior que, aunque es improbable que una medida sola sea capaz de diferenciar los tipos de funciones, varias de ellas parecían ayudar. En particular, la medida de homogeneidad parecía ser bastante útil para valores altos de μ_x , mientras que la medida de grados de libertad parecía ser más útil para valores bajos. La medida de solapamiento parecía no ser muy útil para diferenciar funciones cerca del valor de 0,2, pero sí para valores mayores. De alguna manera, podría ser que el uso conjunto de estas medidas dé resultados mucho más precisos que los que daría cualquiera de ellas por separado.

Con esta idea, lo que haremos ahora consistirá en entrenar un modelo de aprendizaje automático que determine lo siguiente: dados, para cierta función f , los valores $\mu_x(f)$, $\eta_s(f)$, $\eta_h(f)$ y $\mu_\delta(f)$, ¿es f una función simulada?

En vista de las gráficas anteriores, podemos pensar en estos valores como coordenadas en un espacio 4-dimensional. Da la impresión de que, utilizando esta representación, la mayoría de funciones simuladas van a quedar bastante “lejos” del resto de funciones. Además, nos gustaría probar inicialmente con un modelo relativamente fácil de manejar, pues en caso de tener buenos resultados podría ayudar en estudios posteriores a interpretar los resultados. Por estos motivos, vamos a probar con un algoritmo de k-NN (k-Nearest Neighbours).

Para el entrenamiento hemos tomado un dataset con 6000 funciones simuladas, 3000 aleatorias y 3000 pseudoaleatorias. Las primeras 6000 están etiquetadas con la etiqueta “simuladas”, y las demás con la etiqueta “no simuladas”. Por no dar mayor peso a valores muy desproporcionados, como los que se pueden obtener con la medida de homogeneidad, reescalamos los valores al intervalo $[0, 1]$. El algoritmo se hizo en Python, empleando la librería `scikit-learn`, con un 80 % de entrenamiento y un 20 % de test. Con todo esto, los resultados se ilustran en la siguiente matriz de confusión:

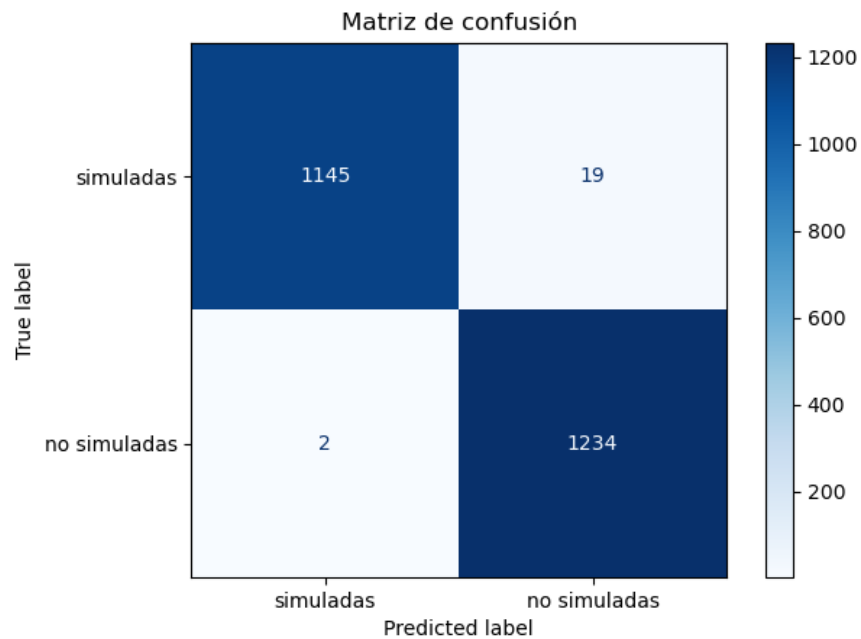


Figura 4.16: Matriz de confusión de predicción de funciones simuladas con k-NN

Vemos que, efectivamente, este modelo acierta con la gran mayoría de las predicciones. Concretamente, la precisión es del 99,12 %. Con esto, reforzamos nuestra idea: puede ser que estas medidas sean capaces de caracterizar al conjunto de funciones simuladas.

Conclusiones y trabajo futuro

Terminamos este trabajo haciendo un balance de los resultados obtenidos.

Para empezar, hemos visto de cerca los resultados de Benjamin Rossman [13] acerca del problema de $\text{Clique}_{n,k}$ y los circuitos booleanos con profundidad constante. Aparte de comprender los resultados, dimos con ciertas herramientas que nos aportaron ideas e intuiciones de cara al estudio experimental, mediante las restricciones de funciones y la sensibilidad a cliques.

En la parte experimental, inicialmente buscamos funciones productivas bajo la métrica sintáctica μ_x , y descartamos la hipótesis que teníamos inicialmente. Vimos que no era cierto que, incluso tomando siempre las funciones más productivas posibles, ignorando si están de alguna forma relacionadas, hiciera falta una gran cantidad de funciones para alcanzar la puntuación máxima. En este sentido, nos dimos cuenta de que las funciones que definimos como pseudoaleatorias hacían que esta hipótesis no se sostuviera.

A través de la definición de este tipo de funciones presentamos unas conjeturas acerca de la diferenciación entre las funciones pseudoaleatorias y las simuladas.

Desarrollamos luego tres medidas, η_s , η_h y η_δ , que daban forma a las ideas en las que se sustentaban las conjeturas. Además, vimos que la última de estas cumplía con los requisitos de las métricas.

Al poner a prueba las medidas, vimos que los resultados estaban muy alineados con nuestras ideas originales. Todas ellas muestran claras diferencias entre las funciones simuladas y el resto de funciones.

Además, un fenómeno muy interesante que observamos fue que el progreso de las distintas medidas parece chocar y resultar difícil de compatibilizar entre sí. Si bien adelantábamos que nuestra esperanza con la métrica sintáctica residía en el crecimiento con las puertas AND, vimos posteriormente que algunas de las métricas tienen un comportamiento opuesto, alejándose del objetivo con las puertas AND y acercándose con las OR. Con esto, la idea que se plantea es que conseguir avanzar en todas las medidas a la vez, que es condición necesaria, parece imposible.

Finalmente, vimos que estas medidas parecen ser muy útiles a la hora de caracterizar las funciones simuladas. Un sencillo modelo de aprendizaje supervisado alcanzó una precisión superior al 99 % a la hora de predecir, con base en las medidas, si una función era o no simulada. Esto nos da mucha esperanza y nos hace pensar

que en los conceptos definidos pueden residir cuestiones relevantes.

Con esto, creemos que un estudio teórico más profundo acerca de las medidas y sus evoluciones a través de los circuitos booleanos podría arrojar resultados interesantes, aunque cierto es que consideramos que un estudio puramente teórico en esta línea puede resultar de notable dificultad.

Otra vía que podría ser interesante investigar va más relacionada con el resultado de Benjamin Rossman [13]. Podría ser útil estudiar el comportamiento de los circuitos booleanos si empleamos como restricción la profundidad máxima, analizando así el comportamiento con las distintas medidas con relación a la profundidad de los circuitos, empleando una metodología que permita generar circuitos (idealmente mínimos) con tamaños y profundidades acotadas.

También se podría probar a emplear otro modelo de aprendizaje supervisado más sofisticado para la predicción del tipo de funciones, como una red neuronal, de forma que se pueda buscar más información acerca de cómo se relacionan estas medidas entre sí.

Finalmente podría ser útil intentar unificar todas las medidas en una combinación de ellas que ilustre que el crecimiento simultáneo de todas las medidas es inviable. Quizá, con la elección correcta de combinación, la hipótesis que descartamos acerca de la métrica sintáctica vuelva a ser viable.

Conclusions and future work

We conclude this work by summarizing the results obtained. To begin with, we have taken a close look at the results of Benjamin Rossman [13] on the problem of Clique_n and constant depth boolean circuits. Apart from understanding the results, we found certain tools that provided us with ideas and insights for the experimental study, through function constraints and clique sensitivity.

In the experimental part, we initially searched for productive functions under the syntactic metric μ_x , and we discarded our initial hypothesis. We saw that it was not true that, even when always taking the most productive functions possible, ignoring whether they are related in any way, a large number of functions were needed to reach the maximum score. In this sense, we realized that the functions we defined as pseudorandom made this hypothesis untenable.

Through the definition of this type of function, we presented some conjectures about the differentiation between pseudorandom and simulated functions.

We then developed three measures, η_s , η_h , and η_δ , which shaped the ideas behind the conjectures. In addition, we saw that the last of these met the requirements of the metrics.

When testing the measures, we saw that the results were very much in line with our original ideas. All of them show clear differences between simulated functions and other functions.

In addition, a very interesting phenomenon we observed was that the progress of the different measures seems to clash and be difficult to reconcile with each other. Although we had anticipated that our hope with the syntactic metric laid in the growth with AND gates, we later saw that some of the metrics behave in the opposite way, moving away from the target with AND gates and approaching it with OR gates. This suggests that it seems impossible to achieve progress in all measures at the same time, which is a necessary condition.

Finally, we saw that these measures seem to be very useful when characterizing simulated functions. A simple supervised learning model achieved an accuracy of over 99 % when predicting, based on the measures, whether a function was simulated or not. This gives us a lot of hope and leads us to believe that there may be relevant insights to be found in the defined concepts.

We believe that a more in-depth theoretical study of the measures and their evolution through boolean circuits could yield interesting results, although we do consider that a purely theoretical study along these lines may prove to be extremely

difficult.

Another approach that could be interesting to explore is more related to the result of Benjamin Rossman [13]. It could be useful to study the behavior of boolean circuits if we use maximum depth as a constraint, thus studying the behavior with different measures in relation to the depth of the circuits, using a methodology that allows us to generate (ideally minimal) circuits with limited sizes and depths.

Another more sophisticated supervised learning model could also be tried for predicting function types, such as a neural network, in order to find more information about how these measures relate to each other.

Finally, it might be useful to try and unify all the measures into a combination that illustrates that the simultaneous growth of all measures is unfeasible. Perhaps, with the right choice of combination, the hypothesis we discarded about the syntactic metric will once again become viable.

Bibliografía

- [1] Librería PyEDA. <https://pyeda.readthedocs.io/en/latest/>, 2018.
- [2] AISENBERG, J. *The Razborov monotone circuit lower bound*. 2013. Disponible en https://mathweb.ucsd.edu/~sbuss/CourseWeb/Math262A_2013F/. Notas del curso *Combinatorics: Circuit Complexity*. Universidad de California en San Diego.
- [3] ALON, N. y SPENCER, J. H. *The Probabilistic Method*. John Wiley & Sons, 3ª edición, 2016. ISBN 978-1-119-06195-3.
- [4] ARORA, S. y BARAK, B. *Computational Complexity: A Modern Approach*. Cambridge University Press, 1ª edición, 2009. ISBN 978-0-521-42426-4 978-0-511-80409-0.
- [5] CARRO GARRIDO, E. y COBIÁN FERNÁNDEZ, J. R. *Analysis of Boolean functions using equanimity and entanglement*. Trabajo de fin de grado, Universidad Complutense de Madrid, Madrid, 2023.
- [6] COOK, S. A. *The Complexity of Theorem-Proving Procedures*. En *Proceedings of the third annual ACM symposium on Theory of computing*, STOC '71, páginas 151–158. Association for Computing Machinery, New York, NY, USA, 1971. ISBN 978-1-4503-7464-4.
- [7] GAREY, M. R. y JOHNSON, D. S. *Computers and Intractability; A Guide to the Theory of NP-Completeness*. W. H. Freeman & Co., USA, 1990. ISBN 978-0-7167-1045-5.
- [8] GONZÁLEZ ARBELO, D. *Repositorio del texto*. <https://github.com/dgonzalezarbelo/Codigo-TFG-Informatica>, 2025.
- [9] HASTAD, J. *Almost Optimal Lower Bounds for Small Depth Circuits*. En *Proceedings of the eighteenth annual ACM symposium on Theory of computing*, STOC '86, páginas 6–20. Association for Computing Machinery, New York, NY, USA, 1986. ISBN 978-0-89791-193-1.
- [10] KUHN, H. W. *The Hungarian method for the assignment problem*. *Naval Research Logistics Quarterly*, vol. 2(1-2), páginas 83–97, 1955. ISSN 1931-9193.

- [11] RAZBOROV, A. A. *Lower bounds on the monotone complexity of some boolean functions.* *Dokl. Akad. Nauk SSSR*, vol. 281(4), páginas 798–801, 1985.
- [12] ROMO GONZÁLEZ, J. J. *Búsqueda de circuitos mínimos para computar el problema CLIQUE.* Trabajo de fin de grado, Universidad Complutense de Madrid, Madrid, 2024.
- [13] ROSSMAN, B. *On the Constant-Depth Complexity of k -Clique.* En *Proceedings of the fortieth annual ACM symposium on Theory of computing*, STOC '08. Association for Computing Machinery, New York, NY, USA, 2008. ISBN 978-1-60558-047-0.
- [14] SIPSER, M. *Introduction to the Theory of Computation.* Cengage Learning, third edition, international edition edición, 2013. ISBN 978-1-133-18779-0 978-1-133-18781-3 978-0-357-67058-3.